COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	AI &DS
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	**CSA0504 - DATABASE MANAGEMENT SYSTEMS**
Academic Year / Batch	2026-2027
Faculty Name	Dr. K. SARAVANAN
Assignment Title	**An Online Examination Database System** using Student(Student_ID, Student_Name, Program_ID, Semester), Course(Course_ID, Course_Code, Course_Name, Credits), Exam(Exam_ID, Course_ID, Exam_Date, Maximum_Mark), Question(Question_ID, Exam_ID, Question_Text, Marks), Submission(Submission_ID, Exam_ID, Student_ID, Start_Time, Submit_Time, Status), Result(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade). Develop the ER model and relational schemas. Identify functional dependencies and normalize the relations. Design indexes for Student_ID and Exam_ID. Write SQL queries for course-wise results and top performers. Explain transaction management during result submission. Discuss concurrency and recovery requirements.
Date of Issue	20-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	**CO1:** Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships. (BL1, BL2) **CO3:** Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3) **CO4:** Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4) **CO5:** Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)
Bloom's Taxonomy Level	L4 – Analyze, L6 – Create, L3 – Apply

SDG Mapping (SDG 1-17)	SDG 4 – Quality Education and SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	The **Online Examination Database System** is highly relevant to educational institutions, universities, certification bodies, and e-learning platforms for managing students, courses, examinations, submissions, and results efficiently. It demonstrates industry practices in **database design, normalization, indexing, SQL query processing, transaction management, concurrency control, and recovery**. The system reduces manual work, improves data accuracy and integrity, enables faster result retrieval, and supports secure and reliable examination processes. From a societal perspective, it promotes **digital education, transparent evaluation, accessibility, and efficient academic administration**.

B. Assignment Problem / Challenge

An Online Examination Database System using Student(Student_ID, Student_Name, Program_ID, Semester), Course(Course_ID, Course_Code, Course_Name, Credits), Exam(Exam_ID, Course_ID, Exam_Date, Maximum_Mark), Question(Question_ID, Exam_ID, Question_Text, Marks), Submission(Submission_ID, Exam_ID, Student_ID, Start_Time, Submit_Time, Status), Result(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade). Develop the ER model and relational schemas. Identify functional dependencies and normalize the relations. Design indexes for Student_ID and Exam_ID. Write SQL queries for course-wise results and top performers. Explain transaction management during result submission. Discuss concurrency and recovery requirements.

Realistic engineering/scientific/computing problem, case, challenge or design requirement related to the Course Outcome.

- The **Online Examination Database System** is a realistic computing problem designed to manage the complete online examination process in an efficient and organized manner. In a traditional examination system, maintaining student information, course details, examination schedules, questions, submissions, marks, and results manually can be difficult and may lead to data duplication, errors, delays, and loss of information. The proposed system uses a **Relational Database**

Management System (RDBMS) to store and manage all examination-related data systematically. The main entities in the system are **Student, Course, Exam, Question, Submission, and Result**. An appropriate **Entity-Relationship (ER) model** must be developed to represent the relationships between these entities. The ER model is then converted into relational schemas using primary keys and foreign keys. The system should maintain accurate student and examination information and provide an organized structure for storing and retrieving records.

- The database design should identify the **functional dependencies** among the attributes and normalize the relations to reduce redundancy and maintain data consistency. For example, Student_ID determines the student name, program, and semester, while Exam_ID determines the corresponding course, exam date, and maximum marks. The relations should be normalized up to **Third Normal Form (3NF)** to avoid insertion, deletion, and update anomalies. Proper normalization ensures that the same information is not unnecessarily repeated in different records. Suitable **indexes on Student_ID and Exam_ID** should also be designed to improve database performance. These indexes help the system quickly retrieve student submissions, examination details, and results, especially when the database contains a large number of records. SQL queries should be developed to generate **course-wise results**, calculate marks, display grades, and identify students who have achieved the highest performance in different examinations or courses.
- **Transaction management** is another important requirement of the system because result submission involves several database operations. When a student completes an examination, the system may need to update the submission status, record the submission time, calculate the total marks, assign the appropriate grade, and store the final result. These operations should be handled as a single transaction using the **ACID properties—Atomicity, Consistency, Isolation, and Durability**. If all operations are successful, the transaction is committed;

if an error occurs, the transaction can be rolled back to prevent incomplete or incorrect result records. Since many students may submit their examinations simultaneously, the system must also handle **concurrency** properly. Concurrency-control techniques such as transaction isolation, locking, unique constraints, and appropriate database controls can prevent problems such as duplicate results, lost updates, dirty reads, and inconsistent data.

- The system must also provide suitable **recovery and security mechanisms** to protect examination information from unexpected failures. Regular database backups, transaction logs, commit and rollback operations, and crash recovery techniques can help restore the database if a system or hardware failure occurs. Security mechanisms such as authentication, authorization, role-based access control, and restricted access to examination results should be implemented to protect sensitive academic information. From a modern computing perspective, the system can be enhanced using **cloud databases, scalable server architecture, automated evaluation, real-time examination monitoring, and performance analytics**. These technologies can support a large number of students taking examinations at the same time while maintaining reliability and performance. Overall, this assignment provides a practical application of important DBMS concepts such as **ER modelling, relational schema design, functional dependencies, normalization, indexing, SQL queries, transaction management, concurrency control, and database recovery**. The proposed system therefore provides a **secure, reliable, efficient, and scalable solution for modern online examination management**.

Assignment Requirements

Detailed Answer

- The assignment requires students to **apply DBMS concepts practically rather than simply reproducing theoretical definitions**. Students should analyze the Online Examination Database System and identify the major problems involved in managing student details, courses, examinations, questions, submissions, and results. They should understand the system requirements and constraints, including data accuracy, security, performance, scalability, concurrent submissions, and reliable result processing. Students are expected to develop the **ER model, relational schemas, functional dependencies, normalization, indexes, and SQL queries** based on the actual requirements of the system. They should justify why particular entities, relationships, keys, constraints, and database structures are selected.
- Students should also be able to **develop and compare alternative database solutions** where appropriate. For example, they may compare a centralized database with a cloud-based or distributed database based on factors such as performance, scalability, availability, cost, and maintenance. They should select suitable indexing techniques for Student_ID and Exam_ID and explain how these decisions improve query performance. Students should use relevant **modern engineering and computing tools**, such as MySQL, PostgreSQL, Oracle Database, ER modelling tools, SQL development environments, and database design tools, to implement and test their proposed solution. The use of these tools should help students move from theoretical database design to a practical working system.
- Students should **interpret the results obtained from SQL queries and database testing** and use those results to evaluate whether the system satisfies its requirements. They should identify top-performing students, generate course-wise results, analyze query performance, and verify that normalization and indexing have improved the database structure and retrieval efficiency. Engineering decisions such as choosing primary keys, foreign keys, normalization level, indexes, transaction isolation,

constraints, and recovery methods should be clearly explained and justified based on technical requirements. Students should also consider important factors such as **security, privacy, reliability, cost, scalability, and maintainability**, because examination results are sensitive academic information and the system may handle a large number of users.

- The assignment should further encourage students to consider **ethical, societal, safety, and sustainability aspects** of the proposed system. Student examination data and results must be protected from unauthorized access, modification, or disclosure, and the system should provide fair and reliable result processing. Ethical considerations include maintaining student privacy, preventing unauthorized changes to marks, and ensuring that automated evaluation does not produce unfair results due to incorrect system design. From a sustainability perspective, efficient database queries, optimized storage, cloud resource management, regular backup strategies, and scalable infrastructure can reduce unnecessary resource usage and operational costs. Therefore, the assignment develops not only database knowledge but also **problem-solving, analytical thinking, engineering decision-making, practical implementation, security awareness, and responsible computing skills** required for designing a modern Online Examination Database System.

C. Problem Statement

Problem / Case / Design Challenge

- The increasing use of online examinations in educational institutions creates a need for a **reliable, secure, and efficient database system** to manage examination-related information. The proposed **Online**

Examination Database System is required to store and manage details of students, courses, examinations, questions, submissions, and results in a structured manner. The system must maintain relationships between students and their examinations, courses and exams, exams and questions, and submissions and results. The main challenge is to design a database that can efficiently handle a large amount of examination data while maintaining **data accuracy, consistency, security, and fast retrieval**.

- The system should be designed using appropriate **ER modelling and relational database concepts**. Students are required to identify the entities, attributes, relationships, primary keys, and foreign keys and convert the ER model into suitable relational schemas. Functional dependencies must be analyzed and the relations must be normalized to an appropriate normal form, preferably **3NF**, to reduce redundancy and avoid insertion, deletion, and update anomalies. Appropriate indexes must be designed for **Student_ID and Exam_ID** to improve the performance of frequently executed queries. SQL queries should be developed to generate **course-wise examination results and identify top-performing students**.
- Another major challenge is ensuring the **reliability of result submission and processing** when many students use the system simultaneously. The system must use transaction management to ensure that submission status, marks, and grades are stored correctly. The **ACID properties** should be considered to maintain consistency and prevent incomplete transactions. Concurrency-control mechanisms must be analyzed to avoid duplicate submissions, lost updates, inconsistent data, and other conflicts. The system should also include suitable **backup and recovery mechanisms** so that examination data and results can be restored after system failures, crashes, or unexpected interruptions.

- The proposed solution should also consider modern computing requirements such as **scalability, security, performance, maintainability, and cost-effectiveness**. Students may evaluate alternative solutions such as centralized and cloud-based database architectures and justify the most suitable approach based on system requirements. Since examination records contain sensitive student information, privacy, authorization, and ethical handling of data must also be considered. The final design should demonstrate how DBMS concepts can be applied to solve a realistic engineering problem and should provide a **secure, scalable, reliable, and efficient Online Examination Database System** capable of supporting students, faculty, and administrators.

D. Requirements and Constraints

1. Functional Requirements

- The Online Examination Database System should provide all the basic functions required to manage an online examination process. The system must store and manage **Student, Course, Exam, Question, Submission, and Result** information. It should allow authorized users to add, update, delete, and retrieve examination records. The system should record student submissions with start time, submit time, and status. It should also calculate and store total marks and grades. SQL queries should be provided to generate course-wise results and identify top-performing students.

2. Performance Requirements

- The database should provide **fast and efficient data retrieval**, even when a large number of students and examination records are stored. Indexes should be created on frequently searched attributes such as **Student_ID and Exam_ID** to improve query performance. The system should respond quickly when students submit examinations or when

faculty members retrieve results. It should also be capable of handling multiple users and simultaneous examination submissions without significant performance degradation.

3. Technical Constraints

- The system should be implemented using a suitable **Relational Database Management System (RDBMS)** such as MySQL, PostgreSQL, or Oracle. The database must use appropriate primary keys, foreign keys, constraints, indexes, and normalized relations. SQL should be used for creating tables, inserting records, retrieving information, and generating reports. The design should preferably follow normalization principles up to **Third Normal Form (3NF)** and should maintain referential integrity between related tables.

4. Cost and Time Constraints

- The system should be developed within the available **academic time and budget**. Open-source database technologies such as MySQL or PostgreSQL can be preferred to reduce software licensing costs. The development should be completed within the assignment schedule while covering database design, implementation, testing, and documentation. The selected solution should provide the required functionality without unnecessary hardware, software, or maintenance expenses.

5. Resource Requirements

The system requires appropriate **hardware, database software, development tools, and technical knowledge**. A computer or server with sufficient processing power, memory, and storage is required to host the database. Database management tools and SQL development environments can be used for implementation and testing. Human resources such as students,

faculty, database developers, and administrators may also be required for designing, maintaining, and using the system.

6. Reliability Requirements

- The examination database must be **highly reliable** because incorrect or lost results can seriously affect students. The system should maintain data consistency during examination submission and result generation. Transactions should use **commit and rollback** mechanisms to prevent incomplete operations. Regular backups and transaction logs should be maintained so that examination data can be recovered after software, hardware, or system failures.

7. Security and Privacy Requirements

- Security and privacy are important because the system stores **student information, examination records, marks, and grades**. Only authorized users should be able to access or modify sensitive information. Authentication and role-based authorization should be implemented for students, faculty, and administrators. Student results should not be accessible to unauthorized users. Appropriate security measures should also protect the database from unauthorized modification, data leakage, and malicious attacks.

8. User Requirements

- The system should be simple and convenient for different types of users. **Students** should be able to access examinations, submit their answers, and view their results. **Faculty members** should be able to manage questions, examinations, and student results. **Administrators** should be able to manage students, courses, examinations, and database operations. The system should provide clear information and minimize unnecessary steps for users.

9. Concurrency Requirements

- The system should support **multiple students submitting examinations at the same time**. Proper concurrency-control techniques should be used to prevent duplicate results, lost updates, dirty reads, and inconsistent data. Transaction isolation and suitable database constraints should ensure that simultaneous transactions do not corrupt examination or result information. A unique constraint on the combination of Exam_ID and Student_ID can help prevent duplicate result records.

10. Safety and Recovery Requirements

- The system should protect examination information from accidental loss and system failures. Automatic backups, transaction logs, recovery procedures, and rollback mechanisms should be provided. If a failure occurs during result submission, the system should either complete the entire transaction or cancel it without leaving incomplete data. Recovery mechanisms should ensure that committed results are not lost and that incomplete transactions do not create incorrect records.

11. Sustainability and Environmental Considerations

- The database system should use computing resources efficiently by avoiding unnecessary data duplication and inefficient queries. **Normalization, indexing, optimized SQL queries, and appropriate storage management** can reduce unnecessary processing and storage requirements. If cloud infrastructure is used, resources can be scaled according to demand instead of continuously maintaining excessive computing capacity. Efficient resource usage can reduce operational costs as well as energy consumption.

12. Ethical Considerations

- The system should handle student examination data in an **ethical and responsible manner**. Student marks and personal information must be kept confidential and should only be accessed for legitimate academic purposes. The system should provide fair and accurate result processing and should prevent unauthorized changes to marks or grades. Automated evaluation, if used, should be properly tested to reduce errors and ensure that students are treated fairly.

E. Student Work

1. Problem Understanding and Formulation

What is the Problem?

The main problem is to design and develop an **Online Examination Database System** that can efficiently manage all information related to students, courses, examinations, questions, submissions, and results. In a traditional examination system, maintaining a large number of records manually can cause data duplication, errors, delays, and difficulty in retrieving results. The proposed database system should provide a structured way to store and manage examination information while maintaining **data accuracy, consistency, security, and reliability**. It should also support multiple students submitting examinations at the same time without creating duplicate or incorrect records.

What are the Expected Outcomes?

The expected outcome is a **well-designed and functional relational database system** for online examination management. The system should include an appropriate ER model, relational schemas, primary keys, foreign keys, functional dependencies, and normalized relations up to 3NF. It should provide suitable indexes for Student_ID and Exam_ID to improve database

performance. SQL queries should be developed to obtain **course-wise results and top-performing students**. The system should also demonstrate proper transaction management during result submission, concurrency control for simultaneous users, and recovery mechanisms for handling system failures. The final database should be efficient, secure, reliable, and easy to maintain.

What Information/Data is Available?

The available data consists of six main entities: **Student, Course, Exam, Question, Submission, and Result**. The Student relation contains Student_ID, Student_Name, Program_ID, and Semester. The Course relation contains Course_ID, Course_Code, Course_Name, and Credits. The Exam relation stores Exam_ID, Course_ID, Exam_Date, and Maximum_Mark. The Question relation contains Question_ID, Exam_ID, Question_Text, and Marks. The Submission relation stores Submission_ID, Exam_ID, Student_ID, Start_Time, Submit_Time, and Status. The Result relation contains Result_ID, Exam_ID, Student_ID, Total_Mark, and Grade. These data items provide the necessary information to design the database and perform examination and result-related operations.

What Assumptions are Made?

The system assumes that every student has a **unique Student_ID**, every course has a unique Course_ID, and every examination has a unique Exam_ID. Each question belongs to one examination, and each examination is associated with a particular course. It is assumed that a student can attend multiple examinations and that an examination can be attended by multiple students. It is also assumed that a student should have only **one final result for a particular examination**, represented by the combination of Exam_ID and Student_ID. Marks are assumed to be within the maximum marks specified for the examination, and grades are generated based on the total marks obtained. Only authorized users are assumed to have permission to modify examination and result information.

What Constraints Must be Satisfied?

The system must satisfy important **database, performance, security, and reliability constraints**. Primary-key values must be unique and cannot be null, while foreign keys must maintain referential integrity between related tables. The database should be normalized to reduce redundancy and avoid data anomalies. Appropriate indexes should be created on Student_ID and Exam_ID for efficient data retrieval. The system must prevent duplicate result records and should maintain consistency when many students submit examinations simultaneously. Transactions must follow ACID properties, and failed transactions should be rolled back safely. Regular backups and recovery mechanisms should protect data from system failures. In addition, student information and examination results must be protected through proper authentication, authorization, privacy, and security measures.

2. Application of Course Knowledge

Principles

The Online Examination Database System applies important **DBMS principles** such as data abstraction, data independence, integrity, consistency, and efficient data management. The database is divided into separate relations such as Student, Course, Exam, Question, Submission, and Result to avoid unnecessary repetition of data. **Primary keys** are used to uniquely identify records, while **foreign keys** establish relationships between tables. Integrity constraints ensure that invalid or inconsistent data cannot be inserted into the database.

Theories

The main DBMS theories applied are **Entity-Relationship modelling, Functional Dependency, Relational Model, Normalization, Transaction**

Management, Concurrency Control, and Recovery. Functional dependencies are identified to determine how attributes depend on candidate keys. For example:

Student_ID → Student_Name, Program_ID, Semester
Course_ID → Course_Code, Course_Name, Credits
Exam_ID → Course_ID, Exam_Date, Maximum_Mark
Question_ID → Exam_ID, Question_Text, Marks
Submission_ID → Exam_ID, Student_ID, Start_Time, Submit_Time, Status
Result_ID → Exam_ID, Student_ID, Total_Mark, Grade

These dependencies are used to design the relations correctly and reduce redundancy.

Mathematical Models

The Online Examination Database System uses mathematical models to calculate and analyze student performance and examination results. The important mathematical models used in the system are **total marks, average marks, percentage, and ranking**.

1. Total Marks

The total marks obtained by a student are calculated by adding the marks obtained in all questions.

Formula:

$$\text{Total Mark} = M_1 + M_2 + M_3 + \ldots + M_n$$

For example, if a student scores 8, 9, 7, 10, and 6 marks:

$$\text{Total Mark} = 8 + 9 + 7 + 10 + 6$$
$$= 40 \text{ marks}$$

2. Percentage Calculation

The percentage of a student can be calculated using:

Percentage = (Total Mark / Maximum Mark) × 100

For example, if the student obtains 80 marks out of 100:

Percentage = (80 / 100) × 100
= 80%

3. Average Marks

The average marks are useful for analyzing the overall performance of students in a course or examination.

Average Mark = Sum of all students' marks / Number of students

For example, if five students obtain 80, 75, 90, 85, and 70 marks:

Average = (80 + 75 + 90 + 85 + 70) / 5
= 400 / 5
= 80 marks

4. Ranking Model

Students can be ranked according to their total marks. The student with the highest total mark receives the first rank.

Rank 1 → Highest Total Mark
Rank 2 → Second Highest Total Mark
Rank 3 → Third Highest Total Mark

SQL can be used to implement ranking:

```
SELECT Student_ID, Student_Name,
    SUM(Total_Mark) AS Total_Marks,
    RANK() OVER (ORDER BY SUM(Total_Mark) DESC) AS Student_Rank
FROM Student S
JOIN Result R ON S.Student_ID = R.Student_ID
```

GROUP BY Student_ID, Student_Name;

5. Relational Algebra Model

The database also applies relational algebra for retrieving information.

Selection:

σ Exam_ID = 101 (RESULT)

This selects results belonging to Exam 101.

Projection:

π Student_ID, Total_Mark (RESULT)

This displays only the Student ID and Total Mark.

Join:

STUDENT ⋈ RESULT

Algorithms

The Online Examination Database System uses algorithms for **result calculation, grade generation, submission processing, and identifying top performers**. These algorithms help automate the examination process and reduce manual calculation errors.

1. Algorithm for Result Calculation and Grade Generation

Algorithm:

1. Start.
2. Read the marks obtained by the student.
3. Calculate the total marks.
4. Calculate the percentage.
5. Determine the grade based on the percentage.

6. Store the total mark and grade in the Result table.
7. Stop.

SQL Coding:

```
SELECT
   Student_ID,
   Total_Mark,
   CASE
      WHEN Total_Mark >= 90 THEN 'A+'
      WHEN Total_Mark >= 80 THEN 'A'
      WHEN Total_Mark >= 70 THEN 'B'
      WHEN Total_Mark >= 60 THEN 'C'
      WHEN Total_Mark >= 50 THEN 'D'
      ELSE 'F'
   END AS Grade
FROM Result;
```

This query automatically assigns a grade based on the student's total marks.

2. Algorithm for Finding Top Performers

Algorithm:

1. Start.
2. Retrieve student details and their results.
3. Calculate the total marks for each student.
4. Sort students in descending order of total marks.
5. Select the students with the highest marks.
6. Display the top performers.
7. Stop.

SQL Coding:

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Mark) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

This query displays the **top 5 performing students** based on their total marks.

3. Algorithm for Processing Exam Submission

Algorithm:

START
↓
Student submits exam
↓
Validate submission
↓
Update submission status
↓
Calculate marks
↓
Generate grade
↓
Store result
↓
COMMIT transaction

↓

END

SQL Coding:

```
START TRANSACTION;

UPDATE Submission
SET Submit_Time = CURRENT_TIMESTAMP,
    Status = 'Submitted'
WHERE Submission_ID = 101;

INSERT INTO Result
(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade)
VALUES
(501, 10, 101, 85, 'A');

COMMIT;
```

If an error occurs:

```
ROLLBACK;
```

Engineering Concepts

Important engineering concepts such as **reliability, scalability, maintainability, performance, security, and fault tolerance** are considered in the database design. Indexing improves the speed of searching records, normalization improves data consistency, and transaction management ensures reliable result submission. The system should also be scalable so that it can support an increasing number of students and examinations. Proper modular design makes the database easier to maintain and modify in the future.

Scientific Concepts

The system uses concepts of **data analysis and measurement** to evaluate student performance. Total marks, average marks, highest marks, lowest marks, and grades can be calculated from the stored examination data. These measurements help faculty understand student performance and identify high-performing or low-performing students. Statistical analysis can also be added in the future to study course-wise performance trends.

Design Methodology

A **structured database design methodology** is followed. First, the problem and requirements are analyzed. Next, entities and relationships are identified and represented using an ER model. The ER model is converted into relational schemas, followed by functional-dependency analysis and normalization. Primary keys, foreign keys, constraints, and indexes are then defined. SQL queries are developed and tested using sample data. Finally, transaction management, concurrency control, security, backup, and recovery requirements are considered.

Normalization Reasoning

Normalization is an important database design technique used in the **Online Examination Database System** to reduce data redundancy, avoid data anomalies, and maintain data consistency. The given system contains separate relations such as **Student, Course, Exam, Question, Submission, and Result**. Each relation is designed so that every attribute stores a single value and each record can be uniquely identified using an appropriate key.

1. First Normal Form (1NF)

A relation is in **1NF** when all attributes contain atomic values and there are no repeating groups. For example, the Student relation is:

STUDENT(Student_ID, Student_Name, Program_ID, Semester)

The functional dependency is:

Student_ID → Student_Name, Program_ID, Semester

Each student has one Student_ID, one name, one program, and one semester. Therefore, the Student relation satisfies **1NF**.

Similarly:

COURSE(Course_ID, Course_Code, Course_Name, Credits)

EXAM(Exam_ID, Course_ID, Exam_Date, Maximum_Mark)

QUESTION(Question_ID, Exam_ID, Question_Text, Marks)

contain atomic values and satisfy 1NF.

2. Second Normal Form (2NF)

A relation is in **2NF** when it is in 1NF and has no partial dependency on a part of a composite key. Most relations in this system have a single-attribute primary key, such as Student_ID, Course_ID, Exam_ID, and Question_ID. Therefore, partial dependency does not occur.

For the Result relation:

RESULT(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade)

If (Exam_ID, Student_ID) is considered the candidate key, the dependency is:

(Exam_ID, Student_ID) → Total_Mark, Grade

The total mark and grade depend on the **complete combination** of Exam_ID and Student_ID, not on only one part. Therefore, there is no partial dependency and the relation satisfies **2NF**.

3. Third Normal Form (3NF)

A relation is in **3NF** when it is already in 2NF and has no transitive dependency. For example, course information should not be repeatedly stored inside the Exam table.

Instead of:

EXAM(Exam_ID, Course_ID, Course_Code, Course_Name, Credits)

the information is separated into:

COURSE(Course_ID, Course_Code, Course_Name, Credits)

EXAM(Exam_ID, Course_ID, Exam_Date, Maximum_Mark)

The dependencies are:

Course_ID → Course_Code, Course_Name, Credits

Exam_ID → Course_ID, Exam_Date, Maximum_Mark

Therefore, course details depend only on Course_ID, while examination details depend on Exam_ID. This removes transitive dependency and reduces repeated data.

4. Final Normalized Relations

After normalization, the database contains the following relations:

STUDENT
(Student_ID, Student_Name, Program_ID, Semester)

COURSE
(Course_ID, Course_Code, Course_Name, Credits)

EXAM
(Exam_ID, Course_ID, Exam_Date, Maximum_Mark)

QUESTION
(Question_ID, Exam_ID, Question_Text, Marks)

SUBMISSION
(Submission_ID, Exam_ID, Student_ID,
Start_Time, Submit_Time, Status)

RESULT
(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade)

5. Justification

The normalized design prevents **duplicate data and update, insertion, and deletion anomalies**. For example, if the name of a course changes, it only needs to be updated in the Course table rather than in every examination record. Similarly, student information is stored only in the Student table and referenced using Student_ID. The use of primary and foreign keys maintains relationships between the tables. Hence, normalization up to **3NF** provides a well-structured, consistent, and maintainable database for the Online Examination Database System.

3. Solution / Design / Methodology

Design

The proposed **Online Examination Database System** is designed as a relational database system to efficiently manage student information, courses, examinations, questions, submissions, and results. The design follows a structured database methodology in which the requirements are first analyzed, followed by identification of entities, attributes, relationships,

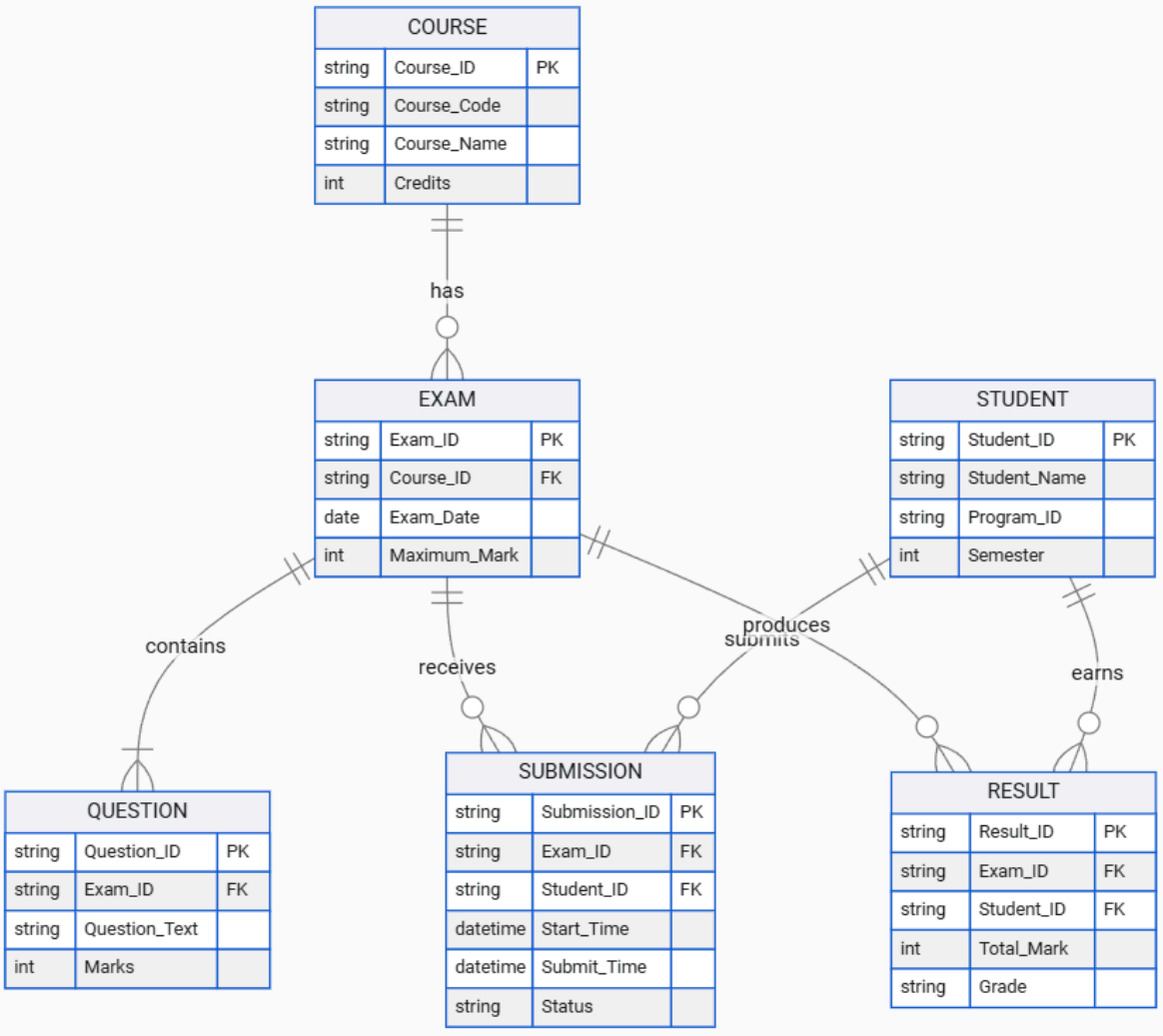
keys, and constraints. The major entities are **Student, Course, Exam, Question, Submission, and Result**. Each entity is represented as a separate table, which helps reduce redundancy and maintain data consistency. Primary keys are used to uniquely identify each record, while foreign keys are used to connect related tables.

The **Student** table stores student details such as Student_ID, Student_Name, Program_ID, and Semester. The **Course** table stores Course_ID, Course_Code, Course_Name, and Credits. The **Exam** table connects an examination with a particular course using Course_ID and stores the examination date and maximum marks. The **Question** table stores individual questions belonging to an examination. The **Submission** table records when a student starts and submits an examination and maintains the submission status. Finally, the **Result** table stores the student's total marks and grade for a particular examination.

Database Design Diagram

Course-Examination ERD

repla



ENTITIES
6

RELATIONSHIPS
6

View Mode chevr Logical chevr

Technical Schema Breakdown

Entity	Primary Key (PK)	Foreign Keys (FK)	Attributes & Types	Relationship Rules
STUDENT	Student_ID	*None*	Student_Name (VARCHAR) Program_ID (VARCHAR) Semester (INT)	1 : N with SUBMISSION 1 : N with RESULT
SUBMISSION	Submission_ID	Exam_ID $\rightarrow$ EXAM Student_ID $\rightarrow$ STUDENT	Start_Time (DATETIME) Submit_Time (DATETIME) Status (VARCHAR)	N : 1 with STUDENT N : 1 with EXAM
EXAM	Exam_ID	Course_ID $\rightarrow$ COURSE	Exam_Date (DATE) Maximum_Mark (INT)	N : 1 with COURSE 1 : N with QUESTION

				1 : N with SUBMISSION **1 : N** with RESULT
COURSE	Course_ID	*None*	Course_Code (VARCHAR) Course_Name (VARCHAR) Credits (INT)	**1 : N** with EXAM
QUESTION	Question_ID	Exam_ID $\rightarrow$ EXAM	Question_Text (TEXT) Marks (INT)	**N : 1** with EXAM
RESULT	Result_ID	Exam_ID $\rightarrow$ EXAM Student_ID $\rightarrow$ STUDENT	Total_Mark (INT) Grade (VARCHAR)	**N : 1** with EXAM **N : 1** with STUDENT

Relationship Design

The **Course–Exam** relationship is one-to-many because one course can have multiple examinations, while each examination belongs to one course. The **Exam–Question** relationship is also one-to-many because an examination can contain multiple questions. The **Student–Submission** relationship allows a student to make examination submissions, while the **Exam–Submission** relationship connects each submission to its examination. The Result table

connects students and examinations and stores the final performance of each student.

Key and Constraint Design

The database uses primary keys such as Student_ID, Course_ID, Exam_ID, Question_ID, Submission_ID, and Result_ID to uniquely identify records. Foreign keys are used to maintain referential integrity between related tables. A **UNIQUE constraint on (Exam_ID, Student_ID)** in the Result table can prevent duplicate results for the same student and examination. Appropriate NOT NULL, CHECK, and data-type constraints can also be applied to ensure valid data.

Design Decision

A **relational database design** is selected because the examination system contains well-defined entities and relationships. The database is normalized up to **3NF**, which reduces unnecessary duplication and improves consistency. Indexes on Student_ID and Exam_ID improve search and query performance. Transaction management, concurrency control, security, and backup mechanisms are included in the design to make the system reliable. This design provides a **structured, secure, efficient, and scalable solution** for managing online examinations.

Algorithm

The Online Examination Database System uses a set of algorithms to manage the examination process from student login to result generation. The algorithm ensures that student submissions are properly recorded, marks are calculated correctly, grades are generated, and results are stored securely in the database. The major steps include validating the student, retrieving the examination, recording the submission, calculating marks, generating the grade, and updating the result.

Algorithm for Online Examination and Result Processing

Step 1: Start the system.

Step 2: Verify the student's Student_ID.

Step 3: Check whether the student is registered for the examination.

Step 4: Retrieve the examination details using Exam_ID.

Step 5: Display the questions associated with the examination.

Step 6: Record the student's start time.

Step 7: Accept and store the student's answers.

Step 8: When the student submits the examination, record the submit time.

Step 9: Change the submission status to **Submitted**.

Step 10: Evaluate the answers and calculate the total marks.

Step 11: Calculate the percentage and determine the grade.

Step 12: Store the total marks and grade in the Result table.

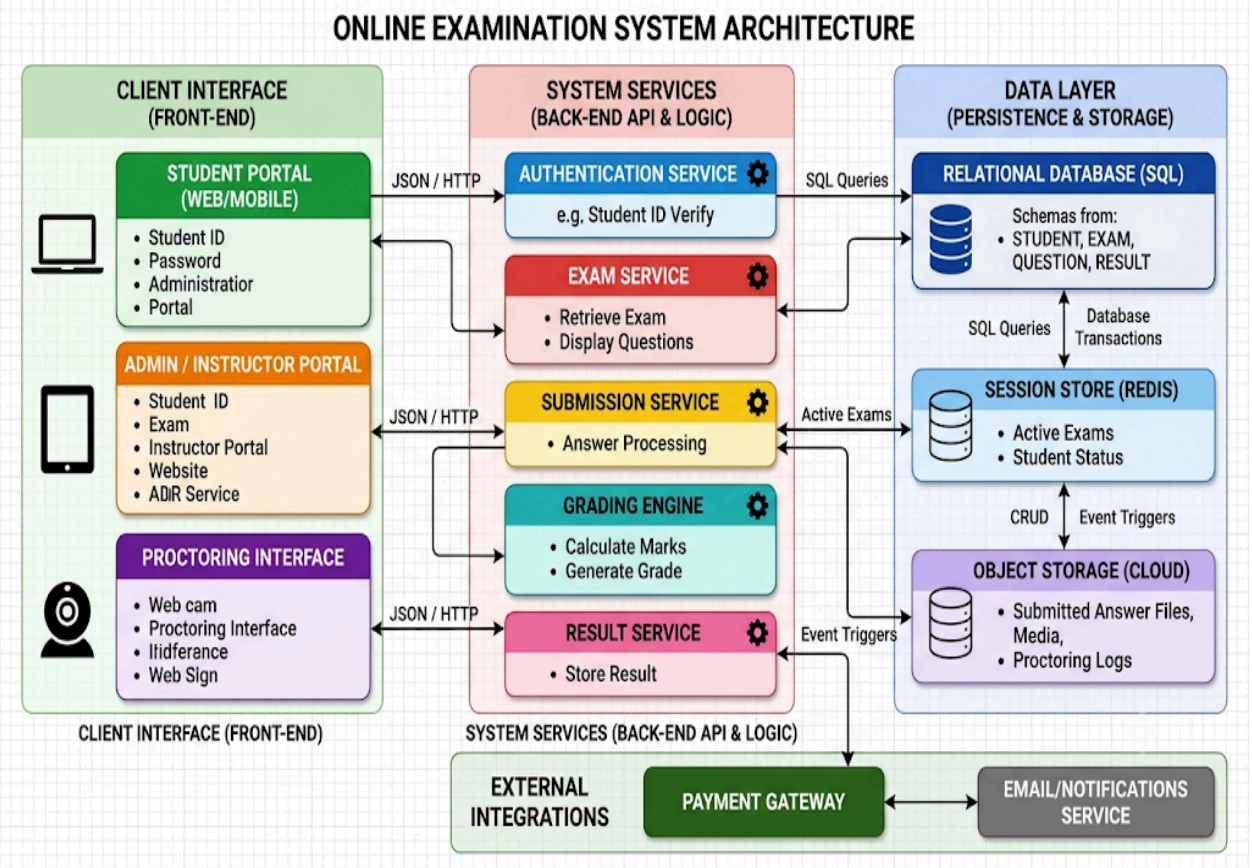
Step 13: Commit the transaction if all operations are successful.

Step 14: If an error occurs, perform rollback.

Step 15: Display the result to the authorized user.

Step 16: Stop.

Algorithm Flow Diagram



TABLE

Step	Action Name	Operation Type	Relational Database / System Action
1	**START**	Lifecycle	Initializes user exam session.
2	**Verify Student_ID**	Authentication	SELECT * FROM STUDENT WHERE Student_ID = ?;
3	**Retrieve Exam_ID**	Query	SELECT * FROM EXAM WHERE Exam_ID = ?;
4	**Display Questions**	Query / Render	SELECT * FROM QUESTION WHERE Exam_ID = ?;
5	**Student Answers**	Client Input	Temporarily buffers user input / response choices.
6	**Submit**	Transaction	INSERT INTO SUBMISSION (...)

	Examination	Start	VALUES (...); (BEGIN TRANSACTION)
7	**Calculate Total Marks**	Aggregation	Evaluates answers against key values to calculate Total_Mark.
8	**Generate Grade**	Business Logic	Maps Total_Mark against grading criteria (e.g., A, B, C, F).
9	**Store Result**	Persistence	INSERT INTO RESULT (Exam_ID, Student_ID, Total_Mark, Grade) VALUES (...);
10	**COMMIT**	Transaction Commit	COMMIT; (Persists submission and result atomically).
11	**END**	Lifecycle	Concludes the examination lifecycle and updates session status.

SQL Implementation

```
START TRANSACTION;

UPDATE Submission
SET Submit_Time = CURRENT_TIMESTAMP,
    Status = 'Submitted'
WHERE Submission_ID = 101;

INSERT INTO Result
(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade)
VALUES
(501, 10, 101, 85, 'A');

COMMIT;
```

If an error occurs during the process:

ROLLBACK;

Top Performer Algorithm

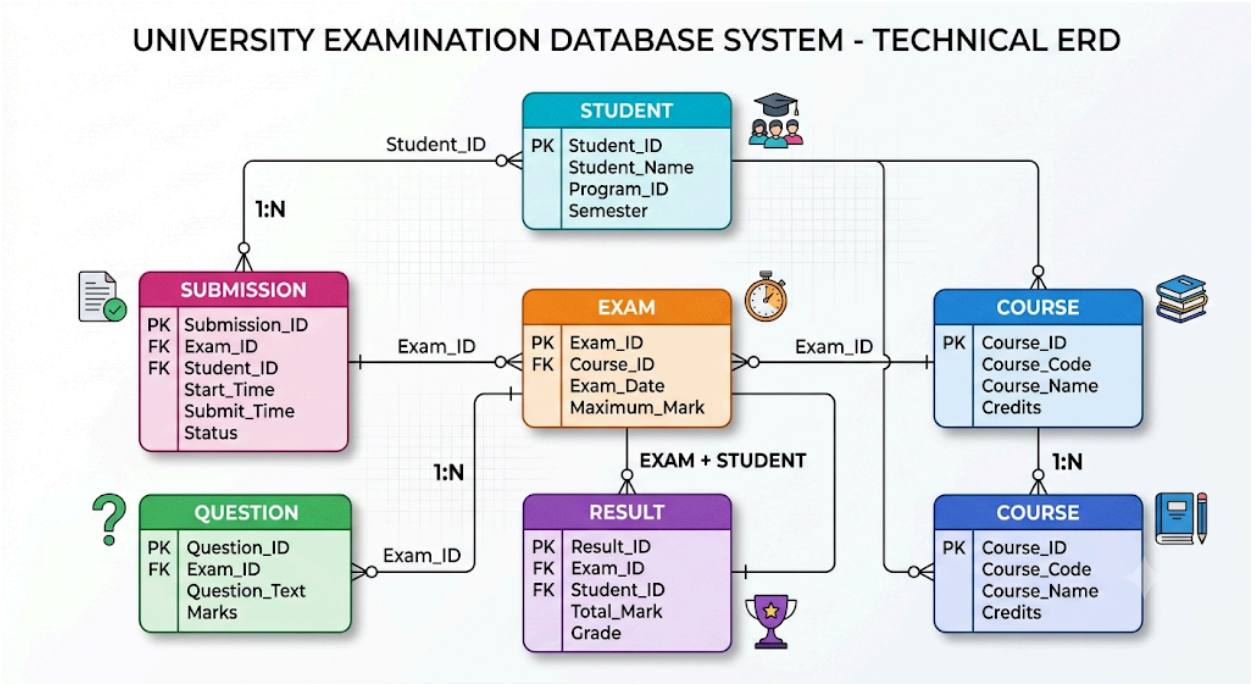
The system can identify top performers by calculating the total marks of each student and arranging them in descending order.

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Mark) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

This algorithm helps the system automatically identify the **top five performing students** based on their total examination marks.

Model

The proposed **Online Examination Database System** follows a **Relational Database Model**. In this model, all examination information is represented using tables, where each table contains related attributes and each record represents one entity. The main tables are **Student, Course, Exam, Question, Submission, and Result**. Primary keys uniquely identify each record, while foreign keys establish relationships between the tables. This model provides a structured way to store, retrieve, and manage examination data.



Model Relationships

The **Student** entity is connected to the **Submission** entity because a student can submit an examination. The **Course** entity is connected to the **Exam** entity because a course can have one or more examinations. Each examination can contain multiple **Questions**, so there is a one-to-many relationship between Exam and Question. The Submission table connects students with examinations and stores the examination attempt details. The **Result** entity connects a student with an examination and stores the final marks and grade.

Model Working

The model starts with student and course information. An examination is created for a particular course, and questions are associated with that examination. When a student attends the examination, a submission record is created with the start time and submission status. After the examination is submitted, the system evaluates the answers and calculates the total marks. The final marks and grade are then stored in the Result table. This process provides a complete connection between the student, examination, questions, submission, and final result.

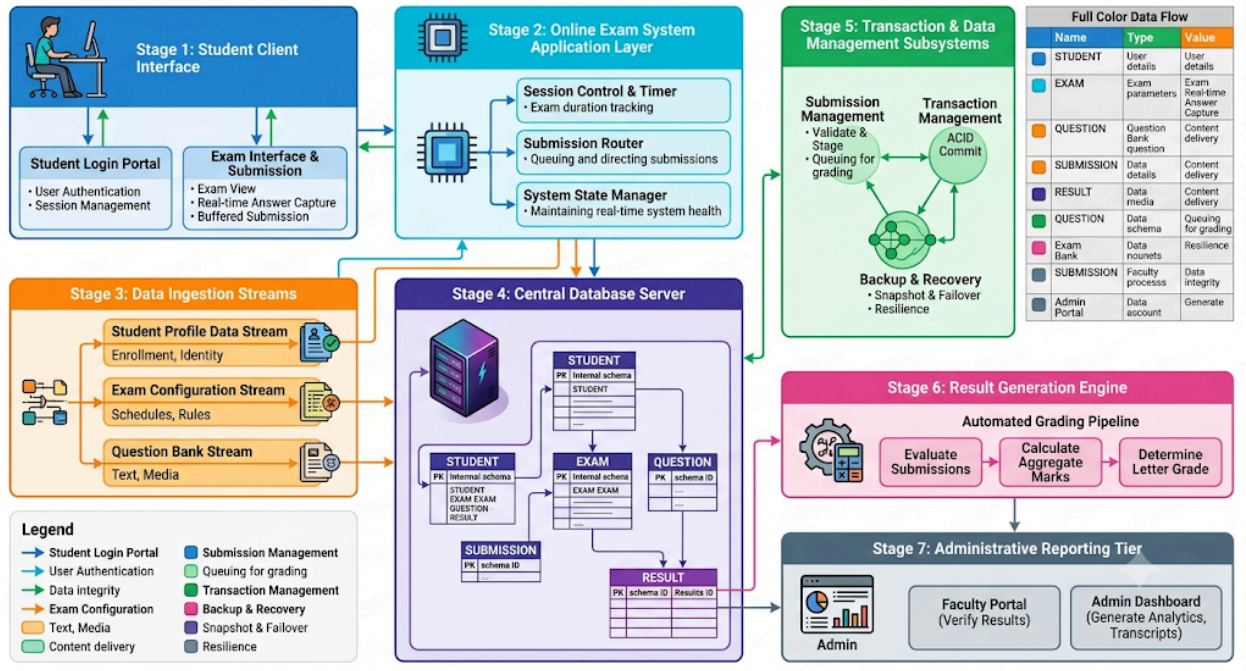
Design Justification

The relational model is selected because it provides **data consistency, easy data retrieval, reduced redundancy, and better integrity**. Normalization can be applied to the tables up to 3NF, while indexes on Student_ID and Exam_ID improve query performance. Primary-key and foreign-key constraints maintain valid relationships between tables. Transaction management and recovery mechanisms ensure that examination results are not lost or incorrectly stored. Therefore, the proposed model provides a **reliable, secure, efficient, and scalable structure** for the Online Examination Database System.

Circuit

For this **Online Examination Database System**, a physical electronic circuit is **not required**, because the problem is based on database management and software design. However, a **system architecture/circuit-style flow diagram** can be used to represent how data moves through the system. The diagram shows the connection between the users, application, database, and supporting database services.

System Flow Diagram



TABLE

Data Pipeline & Subsystem Architecture Table

Layer / Stage	Entity / Module	Operation Type	Primary Data Process & Description	Key Protocols & Technologies
1. Client Tier	STUDENT	Client Interface	Handles user authentication, exam launching, timer tracking, and answer submission.	HTTPS, WebSockets, JWT
2. Application Tier	ONLINE EXAM SYSTEM	Gateway & Routing	Orchestrates real-time exam sessions, validates active timers, and controls request pipelines.	API Gateway, Node.js / Spring Boot
3. Ingestion Streams	Student Data	Data Payload	Profile verification, student ID validation, and session enrollment status.	JSON API, Redis Session Store

	Exam Data	Data Payload	Exam parameters, duration, total marks, and scheduling configuration.	Relational DB Query
	Question Data	Data Payload	Question sets, multiple-choice options, and media attachments.	Relational DB Query / S3
4. Storage Tier	**DATABASE SERVER**	Data Persistence	Central database maintaining ACID compliance and persisting tables.	PostgreSQL / MySQL / Oracle
5. Core Subsystems	**Submission Management**	Processing Subsystem	Buffers incoming answers, prevents double-submits, and serializes logs.	Message Queue (RabbitMQ / Kafka)
	Transaction Management	Processing Subsystem	Enforces transactional consistency (BEGIN, SAVEPOINT, COMMIT/ROLLBACK).	DBMS WAL Engine
	Backup & Recovery	Infrastructure Subsystem	Performs real-time replication, snapshots, and point-in-time recovery (PITR).	Multi-AZ DB Snapshots
6. Evaluation Engine	**RESULT GENERATION**	Business Logic	Compares submitted responses to answer keys, calculates Total_Mark, and generates letter grades.	Batch Worker / Async Evaluator
7. Reporting Tier	**Faculty / Admin**	Analytics & Reporting	Generates institutional score reports, grade distribution graphs, and student transcripts.	BI Dashboards, PDF/CSV Export Engine

Explanation

The student first logs into the online examination system and attends the assigned examination. The application retrieves student, course, examination,

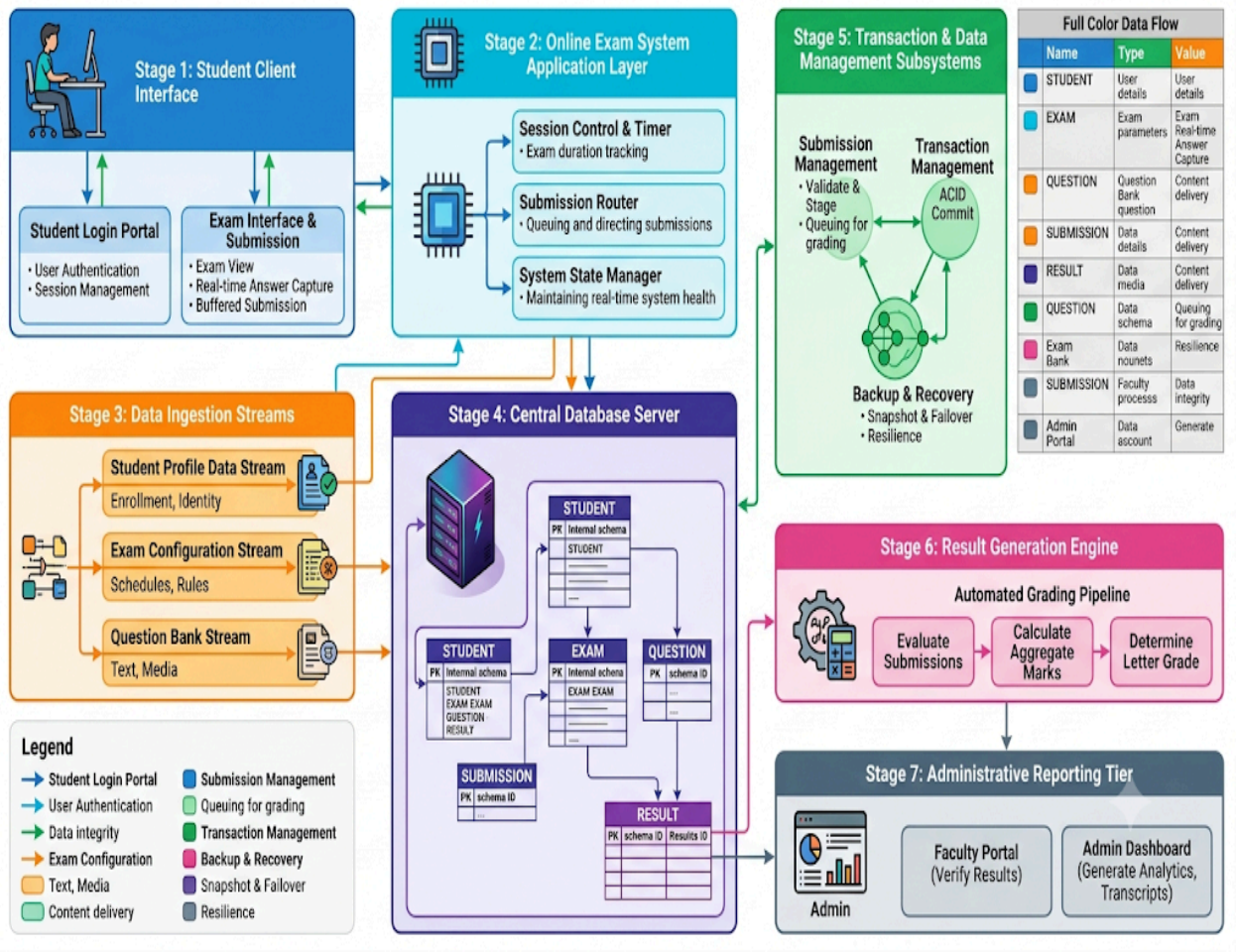
and question information from the database. When the student submits the examination, the submission information is stored and the result-processing process is initiated. The database transaction system ensures that marks and grades are stored correctly. Backup and recovery mechanisms protect the stored information from failures. Finally, the generated results can be accessed by authorized faculty members and administrators.

Note: Since this is a **DBMS-based project**, the above system flow is more appropriate than an electronic circuit diagram.

Architecture

The **Online Examination Database System** follows a **three-tier architecture** consisting of the **Presentation Layer, Application Layer, and Database Layer**. This architecture separates the user interface, business logic, and database operations, making the system easier to develop, maintain, secure, and scale.

Three-Tier Architecture Diagram



TABLE

Technical Architecture & Data Interaction Matrix

Tier	Component	Protocol / Interface	Technical Responsibility	Relational Dependencies
Presentation Layer	**Student Login**	HTTPS / TLS 1.3 / WSS	Token-authenticated test taking, live response	STUDENT, SUBMISSION

			streaming, and client-side countdown timer.	
	Faculty Login	HTTPS / OAuth 2.0	Exam creation, question bank curation, manual evaluation, and grade verification.	COURSE, EXAM, QUESTION
	Admin Login	HTTPS / MFA	User identity provisioning, institutional configuration, and system access audits.	System Audit Logs
Application Layer	**Authentication & Security**	JWT / RBAC Middleware	Decodes tokens, validates session expiration, and enforces role boundaries.	STUDENT
	Exam Management	WebSocket State Machine	Synchronizes exam start/end times, controls exam availability windows.	EXAM, COURSE
	Question Management	REST API / Caching Layer	Fetches question sets, applies randomized order permutations per examinee.	QUESTION, EXAM
	Submission	Message	Intercepts	SUBMISSION

	Management	Queue / Buffer	continuous answer streams, prevents double -submissions, stages auto-save states.	
	Result Processing	Asynchronous Worker	Evaluates submitted answers against keys, aggregates points, maps grading scales.	RESULT, SUBMISSION
	Report Generation	Analytics Engine	Generates PDF transcripts, institutional analytics, and score distribution charts.	RESULT, STUDENT, EXAM
Database Layer	**Relational Entities**	PostgreSQL / MySQL / SQL Server	Persists structured records with strict Primary Key (PK) and Foreign Key (FK) referential integrity.	All 6 relational tables
	Indexing Engine	B-Tree / Hash Indexes	Accelerates high-frequency lookups on Student_ID, Exam_ID, and Submission_ID.	Key columns
	Transaction Engine	ACID / WAL Engine	Guarantees atomic writes so answer submissions	Database Storage Manager

			and result commits never corrupt mid-stream.	
	Concurrency Control	MVCC / Row Locking	Prevents write collision during simultaneous high-volume submissions at the deadline.	Lock Manager
	Backup & Recovery	PITR / Replication	Performs snapshotting and replication across availability zones for zero-loss recovery.	Storage Engine / Replicas

1. Presentation Layer

The **Presentation Layer** is the user-facing part of the system. Students use this layer to log in, view available examinations, answer questions, submit examinations, and view their results. Faculty members can use it to manage examinations, questions, and results. Administrators can manage students, courses, and system information.

2. Application Layer

The **Application Layer** contains the main processing logic of the system. It handles authentication, examination management, question retrieval, submission processing, result calculation, grade generation, and report generation. It also manages transactions and validates user requests before sending them to the database.

3. Database Layer

The **Database Layer** stores all examination-related information. The main tables are **Student, Course, Exam, Question, Submission, and Result**. Primary keys and foreign keys maintain relationships between tables. Indexes on Student_ID and Exam_ID provide faster data retrieval. Transaction management, concurrency control, backup, and recovery mechanisms help maintain database reliability.

Overall Architecture Working

The architecture works by allowing users to send requests through the Presentation Layer. These requests are processed by the Application Layer, which validates the information and communicates with the Database Layer. The database retrieves or updates the required records and sends the result back through the Application Layer to the user. This separation makes the system **secure, maintainable, reliable, and scalable** and is suitable for handling a large number of students and simultaneous online examination submissions.

Experiment

The experiment is carried out to **design, implement, and test the Online Examination Database System** using a relational database such as MySQL. The purpose of the experiment is to verify whether the proposed database can correctly store examination information, maintain relationships, process student submissions, generate results, and retrieve performance information efficiently. The experiment includes database creation, table creation, sample data insertion, query execution, indexing, transaction testing, concurrency testing, and recovery verification.

Experimental Procedure

Step 1: Create the Database


```
CREATE DATABASE OnlineExamDB;
```

```
USE OnlineExamDB;
```

Step 2: Create the Main Tables

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(100),
    Program_ID INT,
    Semester INT
);

CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Code VARCHAR(20),
    Course_Name VARCHAR(100),
    Credits INT
);

CREATE TABLE Exam (
    Exam_ID INT PRIMARY KEY,
    Course_ID INT,
    Exam_Date DATE,
    Maximum_Mark INT,
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);
CREATE TABLE Question (
    Question_ID INT PRIMARY KEY,
    Exam_ID INT,
    Question_Text VARCHAR(500),
    Marks INT,
    FOREIGN KEY (Exam_ID) REFERENCES Exam(Exam_ID)
);
```

```
CREATE TABLE Submission (
    Submission_ID INT PRIMARY KEY,
    Exam_ID INT,
    Student_ID INT,
    Start_Time DATETIME,
    Submit_Time DATETIME,
    Status VARCHAR(20),
    FOREIGN KEY (Exam_ID) REFERENCES Exam(Exam_ID),
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)
);

CREATE TABLE Result (
    Result_ID INT PRIMARY KEY,
    Exam_ID INT,
    Student_ID INT,
    Total_Mark DECIMAL(5,2),
    Grade VARCHAR(5),
    FOREIGN KEY (Exam_ID) REFERENCES Exam(Exam_ID),
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
    UNIQUE (Exam_ID, Student_ID)
);
```

Step 3: Insert Sample Data

```
INSERT INTO Student VALUES
(101, 'Arun', 1, 3),
(102, 'Priya', 1, 3),
(103, 'Kavin', 2, 4);

INSERT INTO Course VALUES
(1, 'CS301', 'Database Management Systems', 4),
(2, 'CS302', 'Computer Networks', 3);
```

```
INSERT INTO Exam VALUES
(10, 1, '2026-09-01', 100),
(11, 2, '2026-09-02', 100);
```

Step 4: Test Result Retrieval

```
SELECT
    S.Student_ID,
    S.Student_Name,
    R.Total_Mark,
    R.Grade
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID;
```

This experiment verifies whether student results can be correctly retrieved from the database.

Step 5: Test Course-Wise Results

```
SELECT
    C.Course_Name,
    S.Student_Name,
    R.Total_Mark,
    R.Grade
FROM Course C
JOIN Exam E ON C.Course_ID = E.Course_ID
JOIN Result R ON E.Exam_ID = R.Exam_ID
JOIN Student S ON R.Student_ID = S.Student_ID
ORDER BY C.Course_Name, R.Total_Mark DESC;
```

This verifies that results can be displayed according to courses.

Step 6: Test Top Performers

```
SELECT
```

```
    S.Student_ID,
    S.Student_Name,
    SUM(R.Total_Mark) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

This identifies the students with the highest overall marks.

Step 7: Test Indexing

```
CREATE INDEX idx_submission_student
ON Submission(Student_ID);

CREATE INDEX idx_submission_exam
ON Submission(Exam_ID);
```

The indexes are tested by executing queries that frequently search using Student_ID and Exam_ID. The purpose is to verify improved data retrieval performance for large datasets.

Step 8: Test Transaction Management

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
    Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
VALUES (501, 10, 101, 85, 'A');
```

COMMIT;

If an error occurs:

ROLLBACK;

This experiment verifies that result submission is handled as a single reliable transaction.

Step 9: Test Concurrency

Multiple students are assumed to submit examinations simultaneously. The database is tested to ensure that each student receives the correct result and that duplicate results are prevented using:

UNIQUE (Exam_ID, Student_ID)

Transaction isolation and locking mechanisms can also be tested to prevent inconsistent updates.

Experimental Observation

The experiment demonstrates that the proposed database successfully stores and retrieves student, course, examination, question, submission, and result information. The SQL queries correctly generate course-wise results and identify top performers. Indexing improves the efficiency of searches involving Student_ID and Exam_ID. Transaction management ensures that result information is either completely stored or completely rolled back when an error occurs. The unique constraint prevents duplicate results for the same student and examination.

Experimental Conclusion

The experiment confirms that the proposed **Online Examination Database System** satisfies the major functional and technical requirements. The

database provides **accurate data management, efficient retrieval, reliable transactions, concurrency support, and secure result processing**. Therefore, the proposed design is suitable for implementing a practical online examination management system.

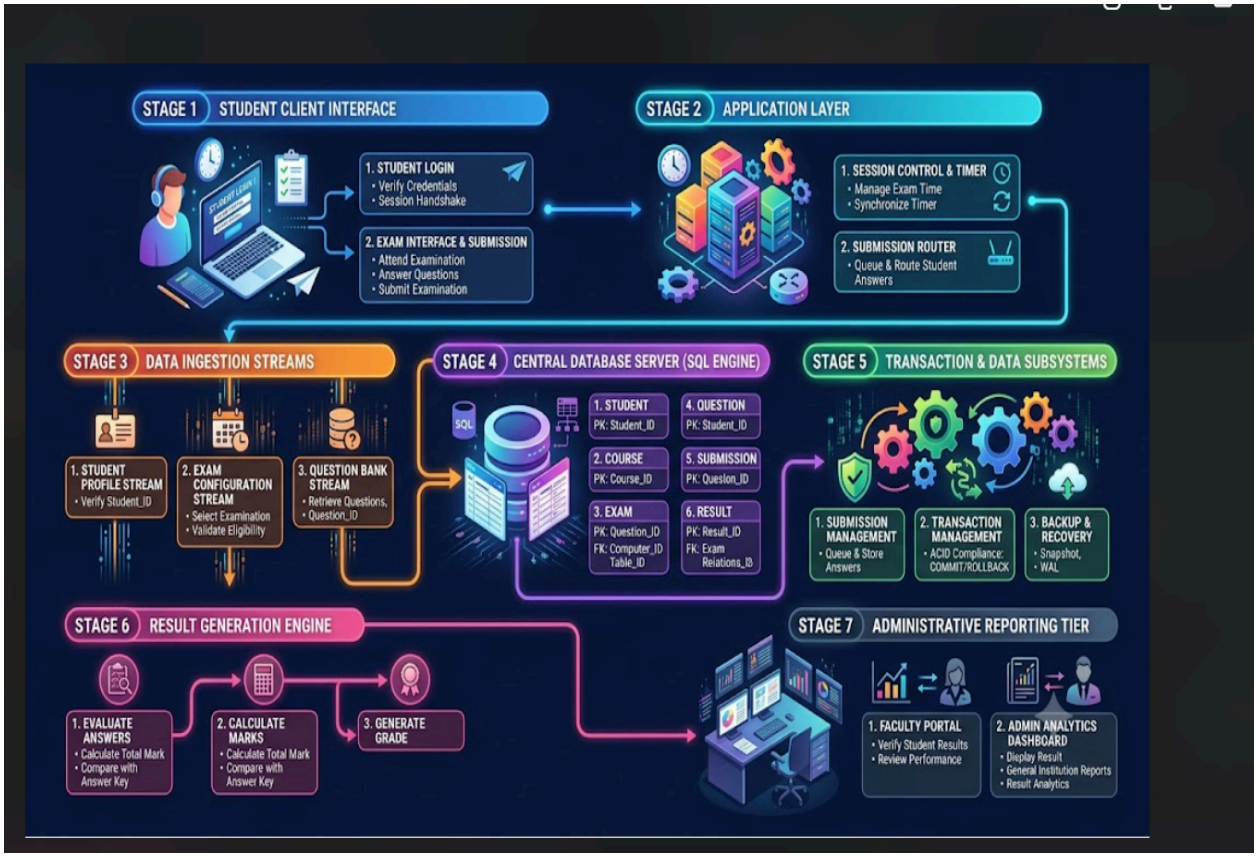


TABLE:

Complete System Workflow & Data Specification Table

Stage & Domain	Sub-Modules / Components	Primary Operations & Data Handled	Technical Constraint & Integrity
Stage 1: Student Client Interface *(Blue Theme)*	• **Student Login Portal** • **Exam Interface & Submission**	Handles student credential verification, countdown timer rendering, temporary local buffering, and test submission.	HTTPS, WebSockets (WSS), JWT Session Token
Stage 2: Online Exam App Layer *(Cyan Theme)*	• **Session Control & Timer** • **Submission Router** • **System State Manager**	Enforces exam window limits, balances live request traffic, and routes answer packets into processing queues.	Node.js / Spring Boot API Gateway, Redis State Cache
Stage 3: Data Ingestion Streams *(Orange Theme)*	• **Student Profile Stream** • **Exam Config Stream** • **Question Bank Stream**	Ingests candidate profile details, test scheduling rules, and question assets (options, marks, media files).	JSON Streaming, S3 Blob Store, In-Memory Queues
Stage 4: Central Database Server *(Purple Theme)*	• **STUDENT, COURSE, EXAM** • **QUESTION, SUBMISSION, RESULT**	Relational data persistence with strict Primary Key (PK) and Foreign Key (FK) relational mappings.	PostgreSQL / MySQL, B-Tree Indexing, ACID Engines
Stage 5: Transaction Subsystems	• **Submission Management**	Manages stage queues, guarantees atomic two-phase commits (COMMIT/ROLLBACK),	Write-Ahead Logging (WAL), Point-in-Time

(Green Theme)	• **Transaction Management** • **Backup & Recovery**	and runs automated snapshots.	Recovery (PITR)
Stage 6: Result Generation Engine *(Pink Theme)*	• **Automated Grading Pipeline** (*Evaluate $\rightarrow$ Aggregate $\rightarrow$ Grade*)	Matches submissions against the official answer keys, sums unit scores to calculate Total_Mark, and assigns letter grades.	Batch Worker Threads, Automated Evaluation Logic
Stage 7: Administrative Reporting *(Dark Slate Theme)*	• **Faculty Portal** • **Admin Analytics Dashboard**	Provides review tools for teachers to audit marks and allows admins to generate transcripts and institution-wide metrics.	BI Dashboards, PDF/CSV Export Engines

Process

The **Online Examination Database System** follows a systematic process to manage the complete examination workflow. The process starts from student registration and login, continues through examination participation and submission, and ends with result generation and storage. Each step is connected to the database to ensure that all information is stored accurately and securely. This process helps the system provide reliable examination management, maintain data consistency, and support multiple users at the same time.

Step 1: Student Registration

The first step in the process is **Student Registration**. Every student is registered in the database before attending an examination. The system

stores the student's unique **Student ID**, student name, program ID, and semester in the **Student** table. This information is used to identify the student during the examination process.

Step 2: Student Login and Authentication

After registration, the student logs into the Online Examination System using their Student ID and password. The system verifies the student's credentials and checks whether the student is authorized to access the examination. If the login details are correct, the student is allowed to continue. If the details are incorrect, the system asks the student to log in again.

Step 3: Examination Selection

Once the login is successful, the system displays the list of examinations assigned to the student. The student selects the required examination using the **Exam ID**. The system retrieves the examination details, including the course name, examination date, duration, and maximum marks, from the **Exam** table.

Step 4: Question Retrieval and Examination Start

The system retrieves all questions related to the selected examination from the **Question** table. The student starts the examination, and the system records the **Start Time** in the **Submission** table. The student reads the questions and submits answers through the examination interface.

Step 5: Examination Submission

After completing the examination, the student clicks the **Submit Exam** button. The system records the **Submit Time** and updates the submission status as **Submitted** in the Submission table. This confirms that the student has completed the examination successfully.

Step 6: Answer Evaluation and Mark Calculation

After submission, the system evaluates the student's answers. It calculates the marks obtained for each question and then adds all the marks to calculate the **Total Mark**. The percentage is also calculated using the maximum marks of the examination.

Formula:

<math block value="\\text{Percentage} = \\frac{\\text{Total Marks Obtained}}{\\text{Maximum Marks}} \\times 100"/>

The calculated marks are used to determine the student's performance.

Step 7: Grade Generation

Based on the percentage obtained by the student, the system generates the appropriate grade. For example, students scoring 90% and above receive Grade A+, students scoring between 80% and 89% receive Grade A, and so on. The grade is generated automatically according to the grading rules of the institution.

Step 8: Result Storage

The system stores the student's **Exam ID**, **Student ID**, **Total Mark**, and **Grade** in the **Result** table. The result is saved only after successful completion of all database operations. Transaction management ensures that incomplete or incorrect results are not stored.

Step 9: Display Result

Finally, the system displays the student's examination result. The student can view the total marks, percentage, grade, and examination status. Faculty members and administrators can also access course-wise results and performance reports through authorized login.

DIAGRAM:

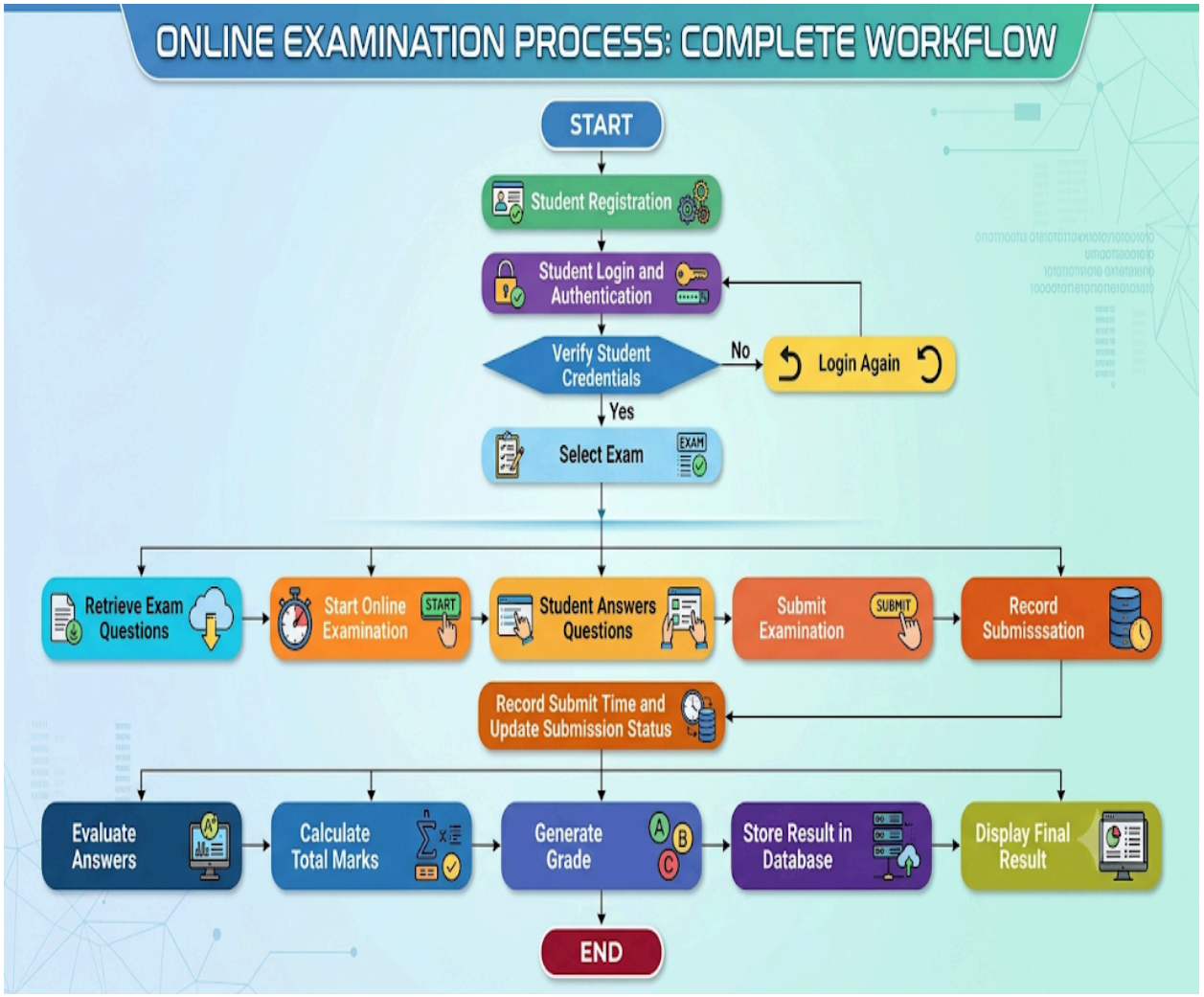


TABLE:

Step #	Stage / Action	Primary Entity / Actor	Key Inputs	Outputs / Updates	Next Step
1	**Start**	System	System launch	Flow initiated	2
2	**Student Registration**	Student / System	Personal info, credentials	Registered profile	3

3	**Student Login and Authentication**	Student / Auth System	Username, password	Session token	4
4	**Verify Student Credentials**	Auth Service / DB	User credentials	Valid / Invalid flag	If Valid $\rightarrow $ 6 If Invalid $\rightarrow $ 5
5	**Login Again**	Student	Re-entered credentials	Retry attempt	Back to 3
6	**Select Exam**	Student	Available exam list	Chosen Exam_ID	7
7	**Retrieve Exam Questions**	System / DB	Exam_ID	Question dataset	8
8	**Start Online Examination**	Student / System	Session start trigger	Exam session active, timer started	9
9	**Student Answers Questions**	Student	Exam questions	Selected options / responses	10
10	**Submit Examination**	Student	Final response set	Submission event	11
11	**Record Submit Time & Update Status**	Exam Engine / DB	Timestamp, Student_ID, Exam_ID	Status updated to Submitted	12
12	**Evaluate Answers**	Evaluation Engine	Answer keys, student inputs	Correct/incorrect mapping	13
13	**Calculate Total Marks**	Grading Engine	Question weightages, evaluation score	Total_Marks	14
14	**Generate Grade**	Grading Engine	Total_Marks, grade scale	Final letter grade / GPA	15
15	**Store Result in Database**	System / Database	Scores, grade, timestamps	Record saved in RESULT table	16
16	**Display Final Result**	Student Interface	Stored result record	Result dashboard rendered	17
17	**End**	System	Terminate session	Process completed	—

Prototype

The prototype of the **Online Examination Database System** is a basic working model developed to show how the proposed system will function in a real environment. The main purpose of the prototype is to demonstrate the important activities of the examination process before developing the complete system. It provides a simple interface for students, faculty members, and administrators to interact with the database. The prototype connects the user interface with the database and demonstrates how student details, examination details, questions, submissions, and results are managed.

Student Login Prototype

The prototype begins with a student login screen. The student enters the required login details, and the system verifies the information before providing access to the examination. The Student ID is used to identify the student in the database. If the details are valid, the student can continue to the examination section. If the details are incorrect, the system does not allow unauthorized access.

Examination Selection Prototype

After successful login, the student is shown the examinations available for their course and semester. The student can select the required examination. The system retrieves the examination information from the database using the Exam ID. The examination details may include the course name, examination date, maximum marks, and other necessary information.

Online Examination Prototype

When the student starts the examination, the system retrieves the questions associated with the selected Exam ID from the Question table. The questions are displayed one by one or on the examination screen. The student selects or enters the appropriate answers. The system records the examination start time and keeps track of the student's submission.

Submission Prototype

After answering all the questions, the student submits the examination. The system records the submission time and changes the submission status from an active state to **Submitted**. The submission details are stored in the Submission table along with the Student ID and Exam ID. This helps the system maintain a proper record of every student's examination attempt.

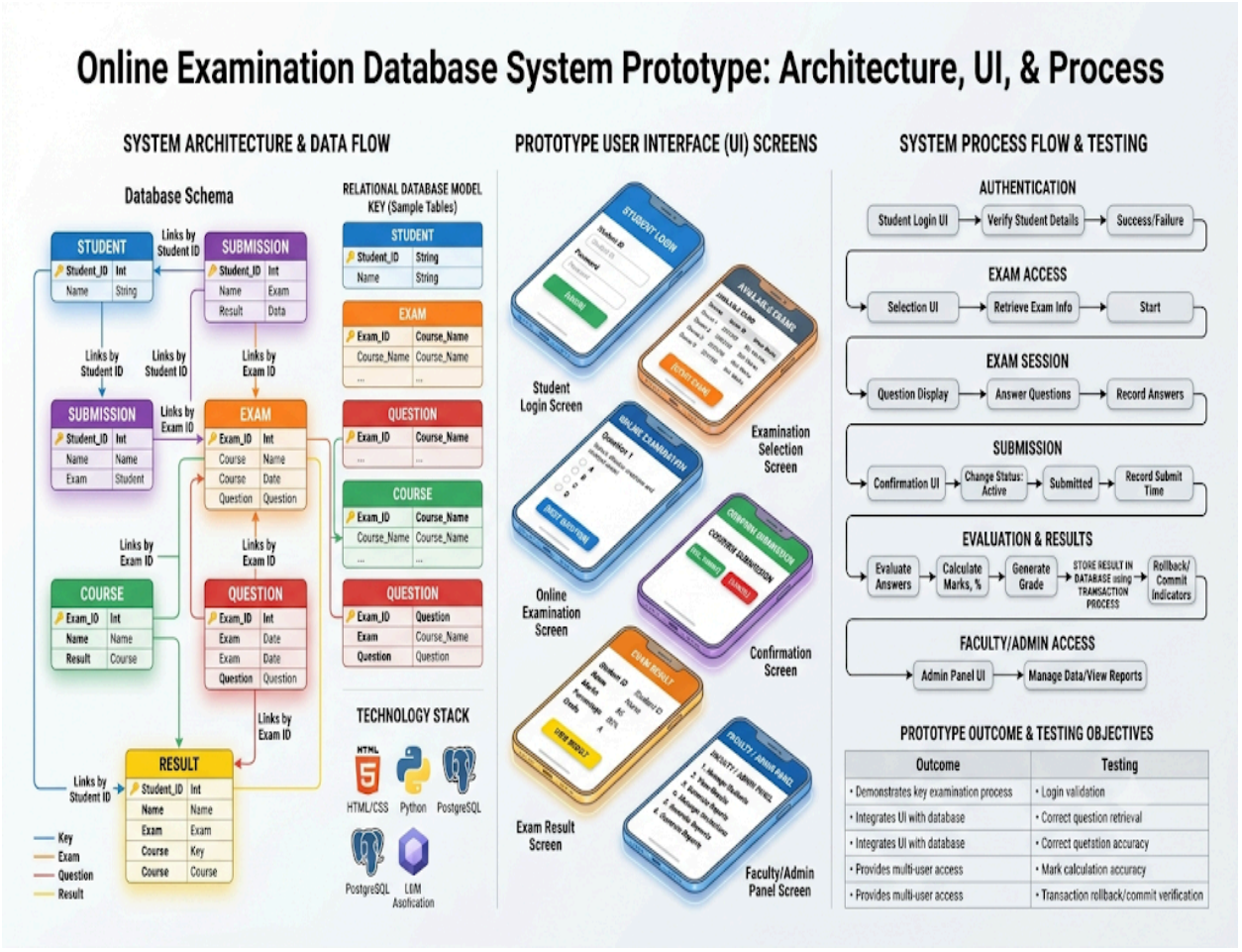
After the examination is submitted, the system evaluates the answers and calculates the total marks. Based on the marks obtained, the system calculates the percentage and generates the appropriate grade. The result is then stored in the Result table. A transaction is used during this process so that the result is completely stored only when all required database operations are successful.

Faculty and Administrator Prototype

The prototype also provides a separate section for faculty members and administrators. Authorized users can manage student information, course information, examination details, and questions. They can also view student results and generate performance reports. This makes the system useful not only for students but also for managing the complete examination process.

Database Prototype

The database prototype contains six main relations: **Student, Course, Exam, Question, Submission, and Result**. The Student table stores student information, while the Course table stores course details. The Exam table stores examination information and connects each examination with a course. The Question table stores questions related to each examination. The Submission table records the examination attempt and submission details of students. Finally, the Result table stores the marks and grades obtained by students.



1. System Architecture & Prototype Modules Table

Module / Component	User / Actor	Input Parameters	Database Tables Accessed	Functional Operation / Output
Student Login	Student	Student_ID, Password	Student	Authenticates student identity;

				blocks invalid attempts
Exam Selection	Student	Selected Exam_ID	Exam, Course	Displays exams available for student's course/semester
Online Examination	Student	Exam_ID, Responses	Question, Submission	Retrieves questions; starts timer; logs ongoing session
Submission	Student	Form Confirmation	Submission	Records Submit_Time; updates status from Active to Submitted
Result Generation	System Engine	Student Answers, Key	Question, Result	Computes marks/grade; completes transactional commit or rollback
Faculty & Admin Panel	Faculty / Admin	Course, Question, Exam inputs	Student, Course, Exam, Question, Result	Manages entities, views grade reports, analyzes performance

2. Relational Database Prototype Schema Table

Relation Name	Primary Key (PK)	Foreign Keys (FK)	Core Attributes	Relationship / Description
STUDENT	Student_ID	None	Student_Name, Program_ID, Semester, Password	Stores master demographic and credentials of students
COURSE	Course_ID	None	Course_Code, Course_Name, Credits	Master academic subjects list

EXAM	Exam_ID	Course_ID $\rightarrow$ COURSE	Exam_Date, Maximum_Mark, Duration	Links an examination to a specific course
QUESTION	Question_ID	Exam_ID $\rightarrow$ EXAM	Question_Text, Option_A, Option_B, Option_C, Option_D, Correct_Option, Marks	Stores questions linked to specific examinations
SUBMISSION	Submission_ID	Student_ID $\rightarrow$ STUDENT Exam_ID $\rightarrow$ EXAM	Start_Time, Submit_Time, Status (Active / Submitted)	Tracks student exam attempts and lifecycle
RESULT	Result_ID	Student_ID $\rightarrow$ STUDENT Exam_ID $\rightarrow$ EXAM	Total_Marks, Percentage, Grade, Generated_Date	Stores computed scores wrapped in database transactions

3. Prototype Test Case Execution Table

Test Case ID	Test Activity	Test Condition / Input	Expected Result	Transaction / Integrity Check
TC-01	Student Authentication	Invalid Student_ID or Password	Access denied; error prompt displayed	Prevents unauthorized database read
TC-02	Examination Selection	Valid Student_ID = 101	Displays assigned exam (CS101, 02-09-2026)	Query filters match course/semester rules
TC-03	Question Fetching	Click [START EXAM]	Pulls relevant rows from QUESTION	Questions mapped to Exam_ID correctly
TC-04	Status Transition	Click [YES, SUBMIT]	Status changes from Active to Submitted	Submit_Time timestamp saved to SUBMISSION
TC-05	Mark Evaluation	Submit completed answers	Total: 85, Percentage: 85%, Grade: A	Formula correctly tallies scores
TC-06	Duplicate Entry Check	Resubmission of identical Exam_ID + Student_ID	Prevent second entry; flag duplicate	Enforces unique constraint on result submission
TC-07	Transaction Rollback	Database error mid-way through saving result	No orphaned records in RESULT table	ROLLBACK invoked to keep DB consistent

Simulation

The simulation of the **Online Examination Database System** is carried out to understand and verify how the system behaves under different examination conditions. The simulation represents the complete working process of the proposed system, starting from student login and examination selection and continuing until the result is generated and stored in the database. It helps to check whether the database tables, relationships, transactions, and system operations work correctly before implementing the complete system.

1. Simulation of Student Login

The simulation begins when a student attempts to log into the Online Examination System. The student provides the Student ID and password. The system checks the Student table to verify whether the student exists and whether the login information is valid. If the details are correct, access is provided to the examination section. If the details are incorrect, the system rejects the login request.

For example, Student ID **101** is entered with valid login details. The system searches the Student table and identifies the student successfully. Therefore, the student is allowed to continue to the next stage.

2. Simulation of Examination Selection

After successful login, the student selects an examination. The system uses the student's information to identify the examinations available for the student's course and semester. The Exam and Course tables are accessed to retrieve the required examination details.

For example, Student ID **101** selects Exam ID **10**, which belongs to the Database Management course. The system retrieves the examination date and maximum marks and displays the information to the student.

3. Simulation of Question Retrieval

When the student starts the examination, the system retrieves all questions associated with the selected Exam ID from the Question table. Only questions belonging to the selected examination are displayed. This ensures that students receive the correct set of questions.

The system also creates a submission record and stores the examination start time. The status of the submission is initially set to **Active**.

4. Simulation of Online Examination

During the examination, the student answers the questions displayed by the system. The system keeps the examination session active until the student completes and submits the examination. The selected Exam ID and Student ID are maintained throughout the session so that the answers and submission information are connected to the correct student and examination.

For example, if the examination contains 50 questions, the student answers each question and continues until all required questions are completed. The system maintains the examination session without changing the student's identity or selected examination.

5. Simulation of Examination Submission

After completing the examination, the student selects the submission option. The system confirms the submission and records the current time as the Submit Time. The status in the Submission table is changed from **Active** to **Submitted**.

The system then prevents the same examination attempt from being treated as a new submission unnecessarily. This helps maintain accurate examination records.

6. Simulation of Result Calculation

After the examination is submitted, the system evaluates the answers and calculates the total marks. The marks obtained for each correctly answered question are added together to calculate the student's total mark.

For example, if a student obtains marks from different questions and the total marks obtained are **85 out of 100**, the system calculates the percentage as:

$$
Percentage = \frac{85}{100} \times 100
$$

[
Percentage = 85%
]

The system then assigns the appropriate grade according to the defined grading rules. For this example, the student's grade can be **A**.

7. Simulation of Result Storage

After calculating the marks and grade, the system stores the final result in the Result table. The Result table contains the Result ID, Student ID, Exam ID, Total Marks, Percentage, Grade, and Generated Date.

For example:

Student ID : 101
Exam ID : 10
Total Marks : 85
Percentage : 85%
Grade : A

This information is stored in the database so that it can be retrieved later by the student, faculty member, or administrator.

8. Simulation of Transaction Management

Transaction management is an important part of the simulation because result submission involves more than one database operation. The system must make sure that the submission and result information are stored correctly.

The result processing can be simulated using a database transaction:

START TRANSACTION;

UPDATE Submission

```
SET Status = 'Submitted',
    Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
(Result_ID, Exam_ID, Student_ID, Total_Marks, Percentage, Grade)
VALUES
(501, 10, 101, 85, 85.00, 'A');

COMMIT;
```

If all operations are successful, the transaction is committed and the changes are permanently stored in the database.

If an error occurs during the process, the system can use:

```
ROLLBACK;
```

The rollback removes the incomplete changes from the transaction and helps keep the database consistent.

9. Simulation of Concurrent Students

The system is also simulated with multiple students accessing the examination at the same time. For example, **500 students** may attempt an examination simultaneously. Each student has a different Student ID, while the same Exam ID may be used by all students.

The database must handle these simultaneous requests without mixing student records or creating duplicate results. Primary keys, foreign keys, unique constraints, and transaction management help maintain the correctness of the data.

10. Simulation of Duplicate Result Prevention

The system is simulated to check what happens if the same student attempts to submit the same examination more than once. A unique constraint can be applied to the combination of **Exam ID and Student ID** in the Result table.

For example, if Student ID 101 has already received a result for Exam ID 10, the system should not create another result for the same examination and student combination. This prevents duplicate result records and maintains database integrity.

11. Simulation of System Failure and Recovery

The system is also tested for a possible database or system failure during result submission. For example, suppose the submission status is updated but an error occurs before the result is completely stored. The transaction should not be partially saved.

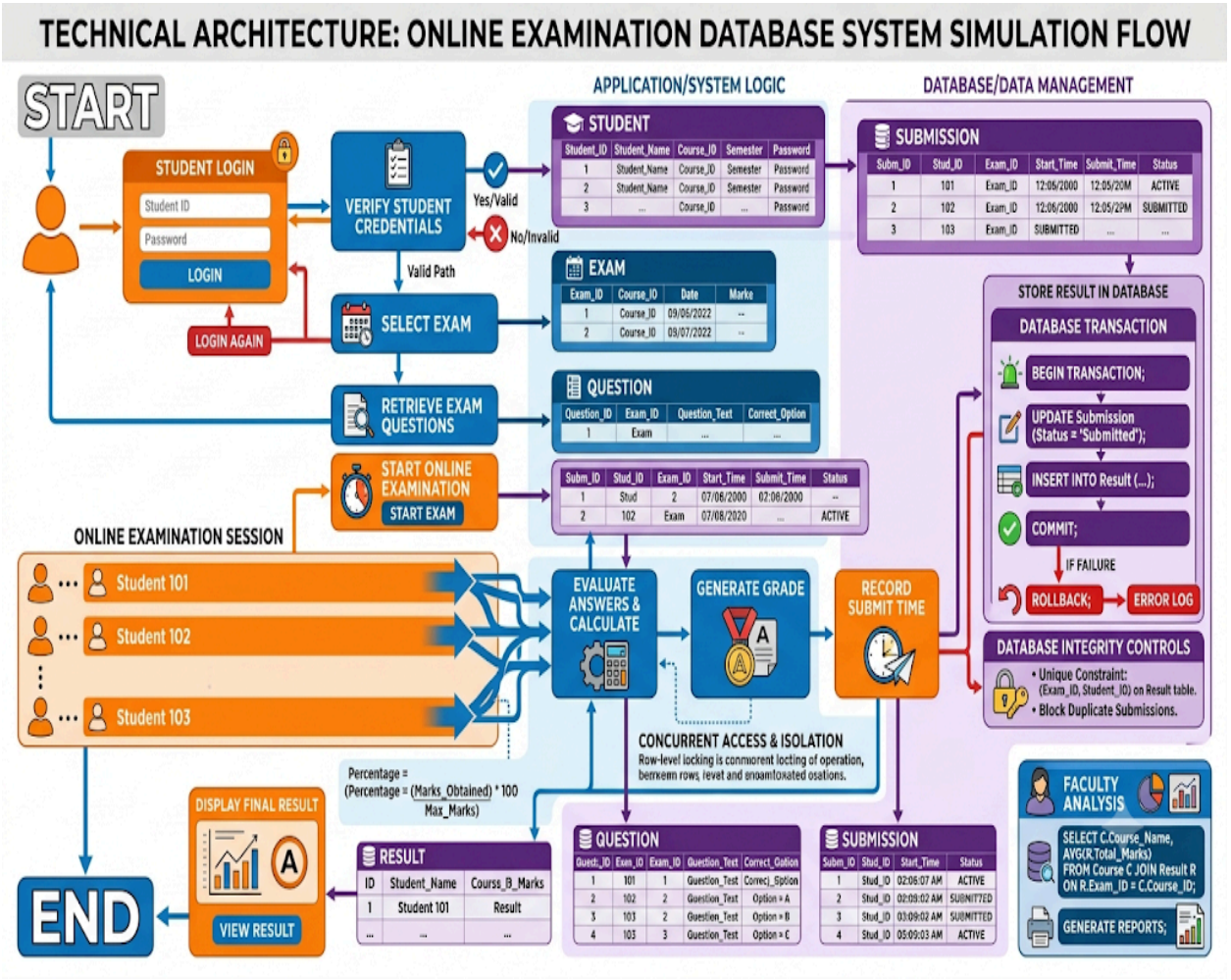
In such a situation, the system performs a **ROLLBACK** operation. The incomplete changes are cancelled, and the database returns to its previous consistent state. Backup and recovery mechanisms can also be used to restore important examination information if a major system failure occurs.

12. Simulation of Course-Wise Result Analysis

The simulation also checks whether faculty members can analyze examination results based on courses. The system can calculate the average marks obtained by students in each course.

For example, the following query can be used:

```
SELECT
   C.Course_Name,
   AVG(R.Total_Marks) AS Average_Marks
```



1. Simulation Phase Mapping Table

Simulation Step #	Simulation Phase	Target Table(s)	Input / Trigger	Process / Business Logic	Output / DB State Change	Concurrency / Integrity Rule

1	**Student Login**	Student	Student_ID = 101, Password	Query Student table to verify credential match	Valid session initiated (or rejected if invalid)	Read lock / Authentication check
2	**Exam Selection**	Student, Course, Exam	Student_ID = 101, Exam_ID = 10	Filter active exams matching student's course & semester	Retrieves Course_Name, Exam_Date, Max_Marks	Foreign Key validation (Course_ID)
3	**Question Retrieval**	Question, Submission	Click "Start Exam" (Exam_ID = 10)	Fetch exam questions; insert new submission log	New row in Submission: Status = 'Active', Start_Time = NOW()	Foreign key constraint on Exam_ID
4	**Online Examination**	Submission	Student answers questions (e.g. 50 Qs)	Maintain session state bound to (Student_ID, Exam_ID)	Answers cached in memory / temp submission log	Session isolation per student
5	**Exam Submission**	Submission	Click "Submit Exam"	Capture timestamp; transition status	Status = 'Submitted', Submit_Time = CURRENT_TIMESTAMP	State lock (prevents re-answering)
6 & 7	**Result**	Result,	Automatic	Evaluate	Insert new	ACID

	Calculation & Storage	Submission	trigger post-submission	answer key; calculate total marks, percentage, and letter grade	row into Result: (Result_ID, Exam_ID, Student_ID, Total_Marks, Percentage, Grade)	Transaction boundary; Unique Constraint (Exam_ID, Student_ID)
8 & 11	**Transaction Management & Recovery**	Submission, Result	Database transaction execution	Run multi-table updates in single atomic unit	If success $\rightarrow$ COMMIT; If failure $\rightarrow$ ROLLBACK	Atomicity & Consistency preservation
9	**Concurrent Students**	All Tables	500+ students taking exam simultaneously	Database engine isolates concurrent transactions	Independent rows written per Student_ID	Row-level locking & connection pooling
10	**Duplicate Prevention**	Result	Accidental second submission attempt	System evaluates unique key constraint	Duplicate insert aborted; error returned	UNIQUE(Exam_ID, Student_ID)
12	**Course-Wise Analysis**	Course, Exam, Result	Faculty query: SELECT C.Course_Name, AVG(R.Total_Marks)...	Aggregate student performance by course	Performance summary & metric report	Read-only aggregation query

2. Simulation Execution & Data State Table (Trace Example)

St	Operati	Stude	Query / DB Event	Before	After	Transacti

ep	on / Action	nt / Context		State	State	on State
S1	Login Verification	Student 101	SELECT * FROM Student WHERE Student_ID=101	Unauthenticated	Session Authorized	Auto-commit
S2	Exam Assignment Lookup	Student 101	SELECT * FROM Exam WHERE Exam_ID=10	Exam list unpopulated	Exam 10 info displayed	Auto-commit
S3	Start Exam Session	Student 101	INSERT INTO Submission (Sub_ID, 10, 101, NOW(), 'Active')	No submission record	Status: Active, Start logged	Committed
S4	Answer Submission	Student 101	User clicks final Submit	Status: Active	Submission confirmed	BEGIN TRANSACTION
S5	Update Submission Log	Student 101	UPDATE Submission SET Status='Submitted', Submit_Time=NOW()	Status: Active	Status: Submitted (uncommitted)	In-progress
S6	Calculate Marks & Grade	System Evaluator	Compute: $85/100 \times 100 = 85\%$	Evaluation pending	Marks: 85, Grade: 'A'	In-progress
S7	Insert Result Record	Student 101	INSERT INTO Result VALUES (501, 10, 101, 85, 85.00, 'A')	No result record	Result row created (uncommitted)	In-progress
S8	**Scenari**	Stude	COMMIT;	Uncommit	All	**Committe**

a	**o A: Success**	nt 101		ted buffer	updates permanently saved	**d**
S8b	**Scenario B: DB Failure**	Student 101	ROLLBACK;	Partial writes	Reverts to prior state; error logged	**Rolled Back**
S9	Duplicate Attempt	Student 101	Resubmit Exam 10	Result row already exists	Insert rejected: duplicate key violation	Rejected

Case Analysis

The **Online Examination Database System** is designed to manage examinations in a college or educational institution where a large number of students may attend examinations at the same time. The system stores student details, course information, examination details, questions, submissions, and results in a structured relational database. The following case analysis explains how the system behaves under different practical situations and how database concepts are applied to solve each situation.

Case 1: Normal Examination Process

Consider a situation where a student with **Student ID 101** is registered in the system and is eligible to attend an examination with **Exam ID 10**. The student logs into the system and selects the examination. The system verifies the student's details and retrieves the examination information from the Exam and Course tables.

When the student starts the examination, the system retrieves the questions associated with Exam ID 10 from the Question table. The start time is recorded in the Submission table and the submission status is initially maintained as **Active**.

After answering all the questions, the student submits the examination. The system records the submit time and changes the submission status to **Submitted**. The answers are then evaluated and the total marks are calculated. Finally, the result is stored in the Result table.

This case represents the normal working condition of the system and shows how the different database tables work together.

Case 2: Invalid Student Login

In another situation, a student may enter an incorrect Student ID or password. The system checks the provided details against the Student table. If the information does not match an existing authorized student record, the system rejects the login request.

The student is not allowed to access the examination until valid login details are provided. This prevents unauthorized users from accessing examination information.

This case demonstrates the importance of authentication and access control in an online examination system.

Case 3: Duplicate Result Submission

A student may accidentally try to submit the same examination more than once. For example, Student ID 101 may already have a result for Exam ID 10 and may attempt another submission for the same examination.

To prevent duplicate result records, the system can apply a unique constraint to the combination of **Exam ID and Student ID**. If the same combination already exists, the database rejects the duplicate entry.

This ensures that one student does not receive multiple result records for the same examination.

Case 4: Multiple Students Submitting at the Same Time

An online examination may have hundreds of students submitting their examinations at approximately the same time. For example, **500 students** may attend the same examination and submit their answers within a short period.

The database must process these requests without mixing the information of different students. Each submission is identified using its Submission ID, Student ID, and Exam ID. Proper transaction management and concurrency control help the database handle simultaneous operations correctly.

This case shows why concurrency management is important in an Online Examination Database System.

Case 5: System Failure During Result Submission

Consider a situation where a student submits the examination, but a database error occurs while the result is being stored. If the system saves only part of the information, the database may become inconsistent.

To avoid this problem, result processing is performed inside a database transaction. If all operations are successful, the system performs **COMMIT**. If an error occurs, the system performs **ROLLBACK** and cancels the incomplete changes.

This case demonstrates how transaction management protects the consistency and reliability of examination data.

Case 6: Incorrect or Missing Examination Data

Another possible situation is that an examination is created without the required course information or a question is added without connecting it to an existing examination. Such incorrect relationships can cause problems when students try to attend the examination.

Primary keys and foreign keys are used to maintain relationships between the tables. For example, the Course ID in the Exam table must refer to an existing Course ID in the Course table. Similarly, the Exam ID in the Question table must refer to an existing Exam ID in the Exam table.

This prevents invalid records from being entered into the database.

Case 7: Course-Wise Result Analysis

Faculty members may need to analyze student performance for a particular course. For example, the faculty member may want to find the average marks obtained by students in the Database Management course.

The system combines information from the Course, Exam, and Result tables and calculates the average marks.

```
SELECT
    C.Course_Name,
    AVG(R.Total_Marks) AS Average_Marks
FROM Course C
JOIN Exam E
ON C.Course_ID = E.Course_ID
JOIN Result R
ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

The output helps faculty members understand the overall performance of students in a particular course. It can also help identify courses where students are performing well or may require additional academic support.

Case 8: Identifying Top Performers

The system can also be used to identify students who obtained the highest marks. This is useful for preparing performance reports and understanding student achievement.

The following query can be used to display the top five performers:

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Marks) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

The query sorts the students based on their total marks in descending order and displays the students with the highest scores.

Case 9: Result Recovery After Failure

Suppose the system experiences a failure immediately after a student submits an examination. The system must ensure that the examination information is not lost or stored incorrectly. Transaction logs, regular database backups, and recovery mechanisms can be used to restore the database to a consistent state.

If the failure occurs before the transaction is committed, the incomplete transaction can be rolled back. If a major database failure occurs, the latest valid backup can be used for recovery.

This case shows the importance of database recovery techniques for an examination system because examination results are important academic records.

Case 10: Security and Privacy of Student Data

The Online Examination Database System contains important information such as student details, examination records, marks, and grades. Therefore, only authorized users should be allowed to access or modify this information.

Students should be able to view only their own examination and result information. Faculty members may be allowed to manage questions and view student performance. Administrators can have wider access for managing the complete system. Proper authentication, authorization, access control, and database security should be applied to protect the information.

Overall Case Analysis

The case analysis shows that the Online Examination Database System must handle both normal and unexpected situations. Normal examination activities include student login, examination selection, question retrieval, submission, evaluation, and result generation. Other situations such as invalid login, duplicate submissions, simultaneous student access, incorrect data, system failure, and recovery must also be considered.

The use of **primary keys, foreign keys, normalization, indexes, constraints, transactions, concurrency control, security, and backup and recovery techniques** makes the system more reliable. The case analysis therefore demonstrates that the proposed database design is suitable for managing an

online examination environment where accurate, secure, and consistent data processing is required.

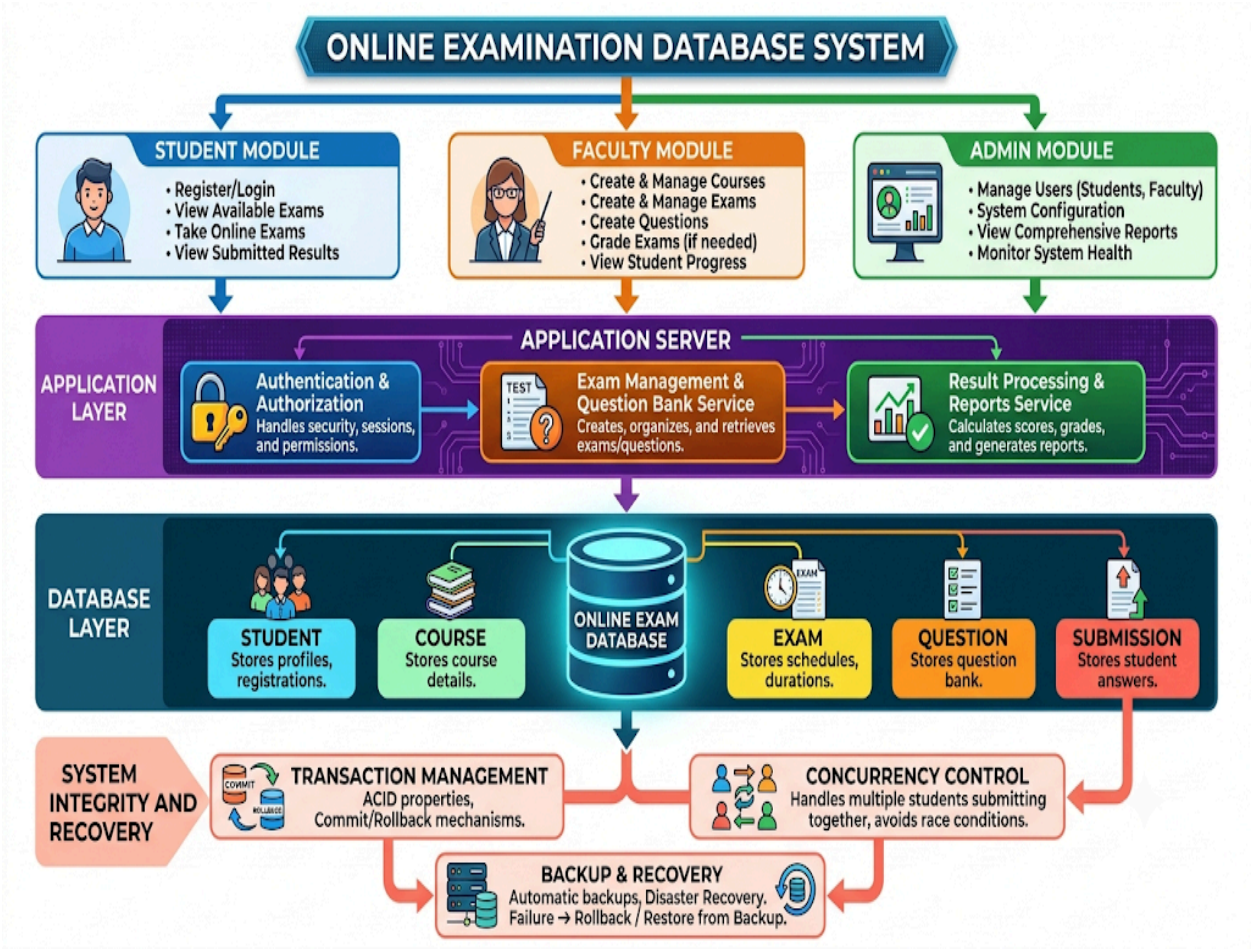


TABLE:

Layer / Tier	Component	Primary Function / Description

User Access Layer	**Student Module**	Registration/login, taking exams, viewing available tests and results
	Faculty Module	Creating exams, setting up question banks, course management, evaluation
	Admin Module	User administration, access control, system configuration, global reporting
Application Layer	**Authentication & Authorization**	Token verification, role-based access control (RBAC), session management
	Exam Management & Questions	Test scheduling, question rendering, randomized paper generation
	Result Processing & Reports	Auto-grading objective questions, score calculation, report card generation
Database Layer	**Online Exam Database**	Central relational database holding all system tables
	Entities / Tables	STUDENT, COURSE, EXAM, QUESTION, SUBMISSION, RESULT
System Integrity & Reliability	**Transaction Management**	Ensures ACID properties with strict COMMIT and ROLLBACK support
	Concurrency Control	Optimistic/pessimistic locking to handle simultaneous student submissions
	Backup & Recovery	Scheduled differential/full dumps and failover point-in-time recovery

Data Analysis

Data analysis in the **Online Examination Database System** is used to understand student performance, course-wise results, examination participation, grades, and submission patterns. The data stored in the Student, Course, Exam, Question, Submission, and Result tables can be processed using SQL queries to generate useful information for students, faculty members, and administrators.

The main purpose of data analysis is not only to store examination records but also to convert the stored data into meaningful information. Faculty members can use the analyzed data to identify high-performing students, calculate average marks, compare course performance, and understand the overall examination results. Administrators can use the information for academic reports and examination management.

1. Student Performance Analysis

Student performance can be analyzed using the Result table. The Student table is joined with the Result table using Student ID. The total marks obtained by each student can then be calculated and arranged in descending order.

```
SELECT
    S.Student_ID,
    S.Student_Name,
    SUM(R.Total_Marks) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC;
```

This query helps faculty members identify students who have obtained higher marks. It can also be used to understand the overall performance of students across different examinations.

2. Course-Wise Performance Analysis

Course-wise analysis is performed to understand how students have performed in different courses. The Course, Exam, and Result tables are joined together, and the average marks are calculated for each course.

```
SELECT
    C.Course_Name,
    AVG(R.Total_Marks) AS Average_Marks
FROM Course C
JOIN Exam E
ON C.Course_ID = E.Course_ID
JOIN Result R
ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

The average mark provides a simple indication of the overall performance of students in each course. Faculty members can use this information to compare examination performance between different subjects.

3. Top Performer Analysis

The system can identify the top-performing students by sorting their total marks in descending order. This can be useful when faculty members want to prepare academic performance reports.

```
SELECT
    S.Student_ID,
    S.Student_Name,
    SUM(R.Total_Marks) AS Total_Marks
```

```
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

The query displays the five students with the highest total marks. The same method can be modified to display a different number of students if required.

4. Highest, Lowest, and Average Marks Analysis

The system can calculate the highest, lowest, and average marks for a particular examination. These values provide a quick overview of examination performance.

```
SELECT
    MAX(Total_Marks) AS Highest_Marks,
    MIN(Total_Marks) AS Lowest_Marks,
    AVG(Total_Marks) AS Average_Marks
FROM Result
WHERE Exam_ID = 10;
```

The highest mark shows the best performance, while the lowest mark shows the minimum performance recorded. The average mark provides an overall indication of the examination performance.

5. Grade Distribution Analysis

Grade distribution analysis is used to determine how many students received each grade. The Result table can be grouped according to the Grade attribute.

```
SELECT
    Grade,
    COUNT(*) AS Number_of_Students
```

```
FROM Result
GROUP BY Grade
ORDER BY Grade;
```

This analysis helps faculty members understand the distribution of student performance. For example, the system can show the number of students who received A+, A, B, C, D, or F grades.

6. Pass Percentage Analysis

The system can also calculate the percentage of students who successfully passed the examination. This helps in evaluating the overall success rate of an examination.

The formula used is:

$$
Pass\ Percentage = \frac{Number\ of\ Passed\ Students}{Total\ Number\ of\ Students} \times 100
$$

For example, if 90 students passed out of 100 students, the pass percentage would be:

$$
Pass\ Percentage = \frac{90}{100} \times 100 = 90\%
$$

This value can be used in academic reports and examination performance analysis.

7. Submission Status Analysis

The Submission table contains the status of each examination attempt. The system can analyze how many submissions are active and how many have been completed.

```
SELECT
    Status,
    COUNT(*) AS Number_of_Submissions
FROM Submission
GROUP BY Status;
```

This analysis helps administrators understand the current examination activity. It can also help identify whether students have successfully submitted their examinations.

8. Examination Participation Analysis

The system can determine the number of students who participated in a particular examination by counting the corresponding submission records.

```
SELECT
    Exam_ID,
    COUNT(DISTINCT Student_ID) AS Number_of_Students
FROM Submission
GROUP BY Exam_ID;
```

This information can be useful for examination administrators because it shows the number of students who participated in each examination.

9. Data Analysis Technical Diagram

Data Analysis

Data analysis in the **Online Examination Database System** is used to understand student performance, course-wise results, examination participation, grades, and submission patterns. The data stored in the Student, Course, Exam, Question, Submission, and Result tables can be processed using SQL queries to generate useful information for students, faculty members, and administrators.

The main purpose of data analysis is not only to store examination records but also to convert the stored data into meaningful information. Faculty members can use the analyzed data to identify high-performing students, calculate average marks, compare course performance, and understand the overall examination results. Administrators can use the information for academic reports and examination management.

1. Student Performance Analysis

Student performance can be analyzed using the Result table. The Student table is joined with the Result table using Student ID. The total marks obtained by each student can then be calculated and arranged in descending order.

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Marks) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC;
```

This query helps faculty members identify students who have obtained higher marks. It can also be used to understand the overall performance of students across different examinations.

2. Course-Wise Performance Analysis

Course-wise analysis is performed to understand how students have performed in different courses. The Course, Exam, and Result tables are joined together, and the average marks are calculated for each course.

```
SELECT
   C.Course_Name,
   AVG(R.Total_Marks) AS Average_Marks
FROM Course C
JOIN Exam E
ON C.Course_ID = E.Course_ID
JOIN Result R
ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

The average mark provides a simple indication of the overall performance of students in each course. Faculty members can use this information to compare examination performance between different subjects.

3. Top Performer Analysis

The system can identify the top-performing students by sorting their total marks in descending order. This can be useful when faculty members want to prepare academic performance reports.

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Marks) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

The query displays the five students with the highest total marks. The same method can be modified to display a different number of students if required.

4. Highest, Lowest, and Average Marks Analysis

The system can calculate the highest, lowest, and average marks for a particular examination. These values provide a quick overview of examination performance.

```
SELECT
   MAX(Total_Marks) AS Highest_Marks,
   MIN(Total_Marks) AS Lowest_Marks,
   AVG(Total_Marks) AS Average_Marks
FROM Result
WHERE Exam_ID = 10;
```

The highest mark shows the best performance, while the lowest mark shows the minimum performance recorded. The average mark provides an overall indication of the examination performance.

5. Grade Distribution Analysis

Grade distribution analysis is used to determine how many students received each grade. The Result table can be grouped according to the Grade attribute.

```
SELECT
   Grade,
   COUNT(*) AS Number_of_Students
FROM Result
GROUP BY Grade
ORDER BY Grade;
```

This analysis helps faculty members understand the distribution of student performance. For example, the system can show the number of students who received A+, A, B, C, D, or F grades.

6. Pass Percentage Analysis

The system can also calculate the percentage of students who successfully passed the examination. This helps in evaluating the overall success rate of an examination.

The formula used is:

$$
\text{Pass Percentage} = \frac{\text{Number of Passed Students}}{\text{Total Number of Students}} \times 100
$$

For example, if 90 students passed out of 100 students, the pass percentage would be:

$$
\text{Pass Percentage} = \frac{90}{100} \times 100 = 90\%
$$

This value can be used in academic reports and examination performance analysis.

7. Submission Status Analysis

The Submission table contains the status of each examination attempt. The system can analyze how many submissions are active and how many have been completed.

```
SELECT
    Status,
    COUNT(*) AS Number_of_Submissions
FROM Submission
GROUP BY Status;
```

This analysis helps administrators understand the current examination activity. It can also help identify whether students have successfully submitted their examinations.

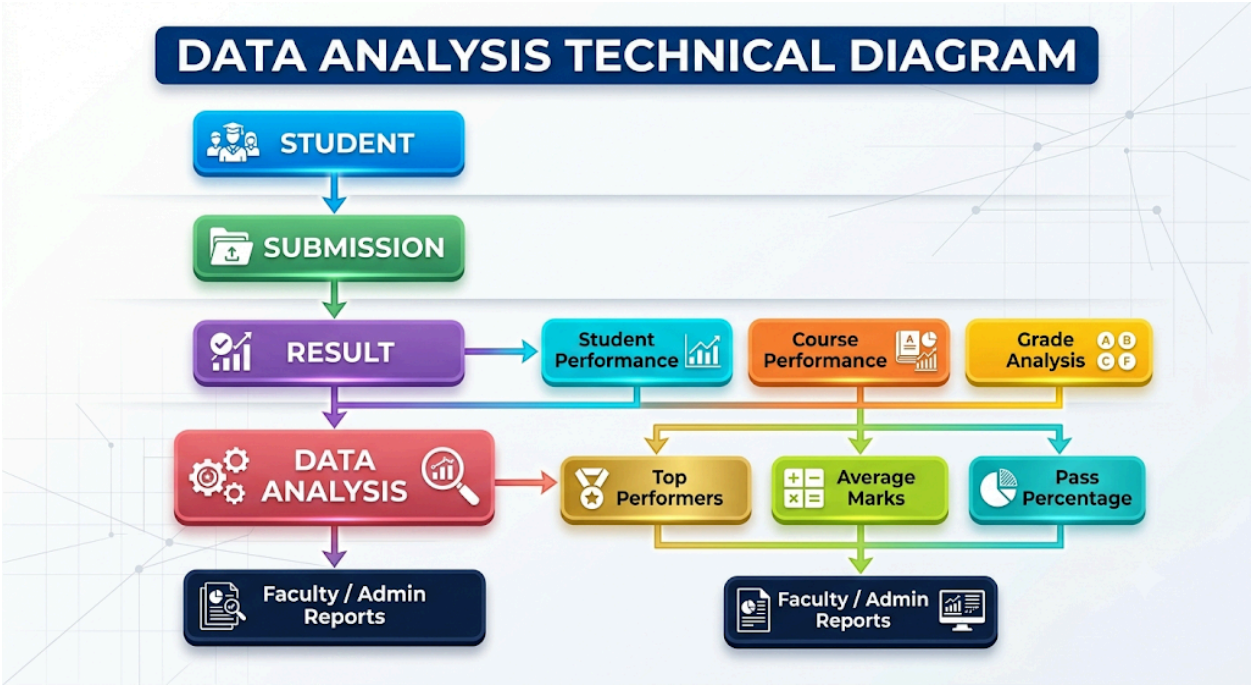
8. Examination Participation Analysis

The system can determine the number of students who participated in a particular examination by counting the corresponding submission records.

```
SELECT
   Exam_ID,
   COUNT(DISTINCT Student_ID) AS Number_of_Students
FROM Submission
GROUP BY Exam_ID;
```

This information can be useful for examination administrators because it shows the number of students who participated in each examination.

9. Data Analysis Technical Diagram



10. Interpretation of Data

The analyzed data provides useful information about the performance of students and examinations. If the average marks of a course are high, it may indicate that students have performed well in that subject. If the average marks are low, faculty members can investigate the reasons and provide additional academic support.

The grade distribution also gives a clear picture of student achievement. A large number of higher grades indicates good overall performance, while a large number of lower grades may indicate that students require additional guidance or practice.

The submission analysis helps administrators monitor the examination process. It can identify the number of completed submissions and active examination attempts. This information is useful for monitoring the system during an online examination.

11. Use of Data Analysis in Decision Making

The results obtained from data analysis can support decision making by faculty members and administrators. Faculty members can identify students who require additional academic support, compare performance between courses, and evaluate examination outcomes. Administrators can use the analyzed information to prepare reports and monitor examination participation.

The analysis also helps the institution maintain proper examination records. Since the information is retrieved directly from the database, manual calculation and duplicate data entry can be reduced.

Overall Data Analysis Outcome

The Data Analysis module converts stored examination records into meaningful information. Student performance, course-wise average marks, top performers, grade distribution, pass percentage, examination participation, and submission status can all be analyzed using SQL queries. The analysis helps faculty members and administrators understand examination performance and make better academic decisions. Therefore, data analysis improves the usefulness of the Online Examination Database System by transforming raw database records into clear and useful

Stage	Entity / Module	Core Role & Data Flow
Input	**Student**	Initiates assessment sessions and holds demographic/course metadata
Processing	**Submission**	Records raw responses, auto-saves progress, and logs submission timestamps
Scoring	**Result**	Evaluates answers against key rubrics, generating raw marks

Intermediate Analysis	**Student Performance**	Tracks individual learning curves, weak areas, and score history
	Course Performance	Assesses question validity, difficulty index, and subject benchmarks
	Grade Analysis	Maps point totals to grade bands (A, B, C, F) and percentile curves
Central Engine	**Data Analysis**	Runs statistical batch jobs, standard deviation, and anomaly detection
Key Output Metrics	**Top Performers**	Identifies highest percentiles, honors eligibility, and toppers
	Average Marks	Calculates cohort means, section comparisons, and median spreads
	Pass Percentage	Computes retention rates, cutoff compliance, and fail frequencies
Reporting	**Faculty / Admin Reports**	Renders visualization charts, CSV exports, and administrative summaries

Mathematical Solution

The mathematical solution for the **Online Examination Database System** is used to calculate student marks, percentage, average performance, grade, pass percentage, and ranking. These calculations help the system automatically evaluate examination results instead of depending on manual calculations. The mathematical formulas are applied to the data stored in the Result table and the calculated values are used for generating accurate student performance reports.

1. Calculation of Total Marks

The total marks obtained by a student are calculated by adding the marks obtained in all the questions.

If a student answers (n) questions and the marks obtained in each question are represented by (M_1, M_2, M_3, \ldots, M_n), then the total marks are calculated as:

$$
T = M_1 + M_2 + M_3 + \cdots + M_n
$$

or,

$$
T = \sum_{i=1}^{n} M_i
$$

For example, if a student obtains 8 marks, 10 marks, 7 marks, 9 marks, and 11 marks in five questions:

$$
T = 8+10+7+9+11
$$

$$
T = 45
$$

Therefore, the student's total marks are **45 marks**.

2. Calculation of Percentage

After calculating the total marks, the system calculates the percentage obtained by the student.

The formula is:

$$
Percentage = \frac{Total\ Marks\ Obtained}{Maximum\ Marks} \times 100
$$

For example, if a student obtains 85 marks out of a maximum of 100 marks:

$$
Percentage = \frac{85}{100}\times100
$$

$$
Percentage = 85\%
$$

Therefore, the student's percentage is **85 percent**.

3. Calculation of Average Marks

The average marks are used to understand the general performance of a group of students.

The formula is:

$$
Average = \frac{Sum\ of\ Marks}{Number\ of\ Students}
$$

For example, suppose five students obtain 80, 75, 90, 85, and 70 marks.

$$
Average = \frac{80+75+90+85+70}{5}
$$

$$
Average = \frac{400}{5}
$$

$$
Average = 80
$$

Therefore, the average mark of the five students is **80 marks**.

4. Grade Calculation

The system assigns a grade based on the percentage obtained by the student. A sample grading model can be defined as follows:

$$
Grade = \begin{cases} A+ & \text{if Percentage } \geq 90\\ A & \text{if } 80 \leq Percentage < 90\\ B & \text{if } 70 \leq Percentage < 80\\ C & \text{if } 60 \leq Percentage < 70\\ D & \text{if } 50 \leq Percentage < 60\\ F & \text{if Percentage } < 50 \end{cases}
$$

For example, if a student obtains 85 percent, the system checks the grading conditions and assigns **Grade A**.

This process allows the database system to generate grades automatically after calculating the student's marks.

5. Pass Percentage Calculation

The overall pass percentage of an examination can be calculated using the number of students who passed and the total number of students who attended the examination.

The formula is:

$$
Pass\ Percentage = \frac{Number\ of\ Passed\ Students}{Total\ Number\ of\ Students} \times 100
$$

For example, if 90 students pass out of 100 students:

$$
Pass\ Percentage = \frac{90}{100} \times 100
$$

$$
Pass\ Percentage = 90\%
$$

Therefore, the overall pass percentage is **90 percent**.

6. Highest and Lowest Marks

The highest and lowest marks can be used to understand the range of student performance.

If the marks obtained by students are:

$$
65,\ 78,\ 92,\ 85,\ 70
$$

then:

[
Highest\ Mark = 92
]

and

[
Lowest\ Mark = 65
]

In the database, these values can be obtained using the MAX() and MIN() functions.

```
SELECT
   MAX(Total_Marks) AS Highest_Marks,
   MIN(Total_Marks) AS Lowest_Marks
FROM Result
WHERE Exam_ID = 10;
```

7. Student Ranking

Student ranking can be determined by arranging the total marks in descending order. The student with the highest mark receives the first rank.

For example:

Student 101 - 92 marks - Rank 1
Student 105 - 88 marks - Rank 2
Student 103 - 84 marks - Rank 3
Student 102 - 78 marks - Rank 4
Student 104 - 70 marks - Rank 5

The ranking can be generated using the following SQL query:

```
SELECT
   Student_ID,
   Total_Marks,
   RANK() OVER (ORDER BY Total_Marks DESC) AS Student_Rank
FROM Result
WHERE Exam_ID = 10;
```

8. Example of Complete Result Calculation

Consider a student with the following examination details:

Student ID   : 101
Exam ID   : 10
Maximum Marks   : 100
Marks Obtained   : 85

The percentage is calculated as:

$$
\text{Percentage} = \frac{85}{100}\times100
$$

$$
\text{Percentage} = 85\%
$$

According to the grading model:

$$
80 \leq 85 < 90
$$

Therefore:

$$
\text{Grade} = A
$$

The final result is:

Student ID : 101
Total Marks : 85
Percentage : 85%
Grade : A

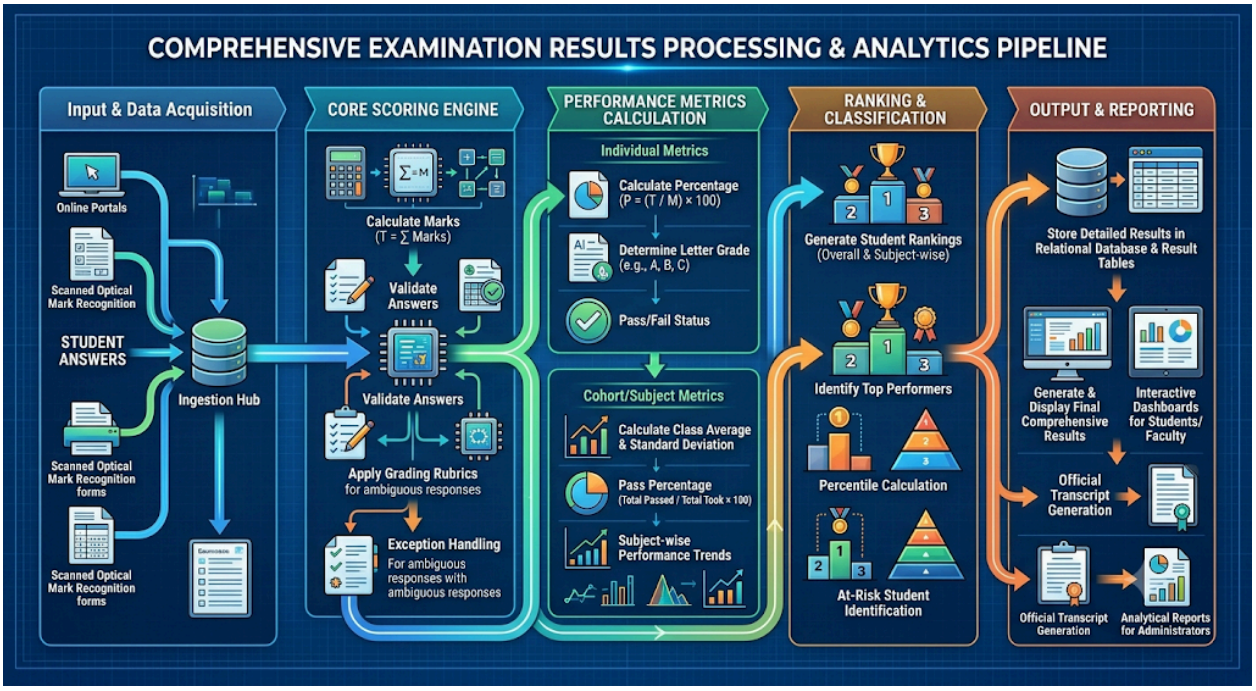
9. SQL Implementation of Mathematical Calculation

The mathematical calculations can be implemented directly in the database using SQL.

```
SELECT
   Student_ID,
   Total_Marks,
   (Total_Marks / 100) * 100 AS Percentage,
   CASE
      WHEN Total_Marks >= 90 THEN 'A+'
      WHEN Total_Marks >= 80 THEN 'A'
      WHEN Total_Marks >= 70 THEN 'B'
      WHEN Total_Marks >= 60 THEN 'C'
      WHEN Total_Marks >= 50 THEN 'D'
      ELSE 'F'
   END AS Grade
FROM Result;
```

This query calculates the percentage and generates the grade automatically for each student.

10. Mathematical Solution Flow Diagram



Overall Mathematical Solution

The mathematical model provides a systematic method for processing examination results. Total marks are calculated by adding the marks obtained in individual questions. The percentage is calculated using the total marks and maximum marks. Average marks are used to measure overall student performance, while the grading formula converts percentages into grades. Pass percentage, highest marks, lowest marks, and ranking provide additional information for performance analysis.

By implementing these mathematical calculations in the Online Examination Database System, the process of result generation becomes faster, more accurate, and less dependent on manual calculations. The calculated values can be stored in the Result table and later used by students, faculty members, and administrators for viewing results and preparing academic reports.

Step	Pipeline Stage	Mathematical Formula / Operation	Database & System Action	Output / Artifact
1	**Input & Ingestion**	Vectorize raw responses $\vec{A} = [a_1, a_2, \dots, a_n]$	Ingest JSON payload, validate exam session token, lock	Raw submission records

			submission state	
2	**Marks Calculation**	$T = \sum_{i=1}^{n} m_i$	Match against answer key matrix; compute negative marking if applicable	Total raw score (T)
3	**Percentage Calculation**	$P = \left(\frac{T}{M}\right) \times 100$	Map raw marks against maximum attainable points (M)	Student percentage (P)
4	**Grade Determination**	$G = f(P)$ based on cutoffs (e.g., $P \ge 90 \to \text{A}$)	Evaluate conditional cutoff rules (relative or absolute grading scale)	Assigned letter grade (G)
5	**Cohort Metrics**	$\mu = \frac{1}{N}\sum T$, $\text{Pass\%} = \left(\frac{N_{\text{pass}}}{N}\right) \times 100$	Aggregate batch metrics, standard deviation (σ), and class-wide trends	Mean score and pass percentage
6	**Ranking & Distribution**	$\text{Rank} = 1 + \sum [T_j > T_i]$	Sort descending by T, resolve ties using timestamp/criteria, calculate percentiles	Merit position & leaderboard
7	**Persistence Layer**	INSERT INTO RESULT (...) VALUES (...)	Execute transactional COMMIT, update foreign keys, ensure ACID compliance	Stored database tuple
8	**Output & Presentation**	Rendering JSON/HTML views	Push notifications, generate dynamic scorecards,	Final result display & report

Use of Modern Tools

The development of the **Online Examination Database System** requires the use of modern software tools for database design, implementation, testing, query execution, and documentation. Since this project is based on Database Management Systems, tools such as **MySQL, MySQL Workbench, Visual Studio Code, Git/GitHub, and draw.io** can be used to support different stages of the project. These tools help in designing the database, writing and testing SQL queries, creating technical diagrams, managing project files, and documenting the final system.

The selection of these tools is based on the requirements of the project. The Online Examination Database System contains several related tables such as Student, Course, Exam, Question, Submission, and Result. Therefore, a relational database management system is required to store and manage the information. Development and diagramming tools are also useful for creating the prototype and presenting the system architecture.

MySQL

MySQL is the main database management tool used for implementing the Online Examination Database System. It is a relational database management system that supports SQL for creating tables, inserting records, updating information, retrieving data, and managing relationships between tables.

In this project, MySQL is used to create the Online Examination Database and its main relations. Primary keys are used to uniquely identify records, while foreign keys are used to establish relationships between related tables. Constraints can also be applied to prevent invalid or duplicate data.

The main tables created using MySQL are:

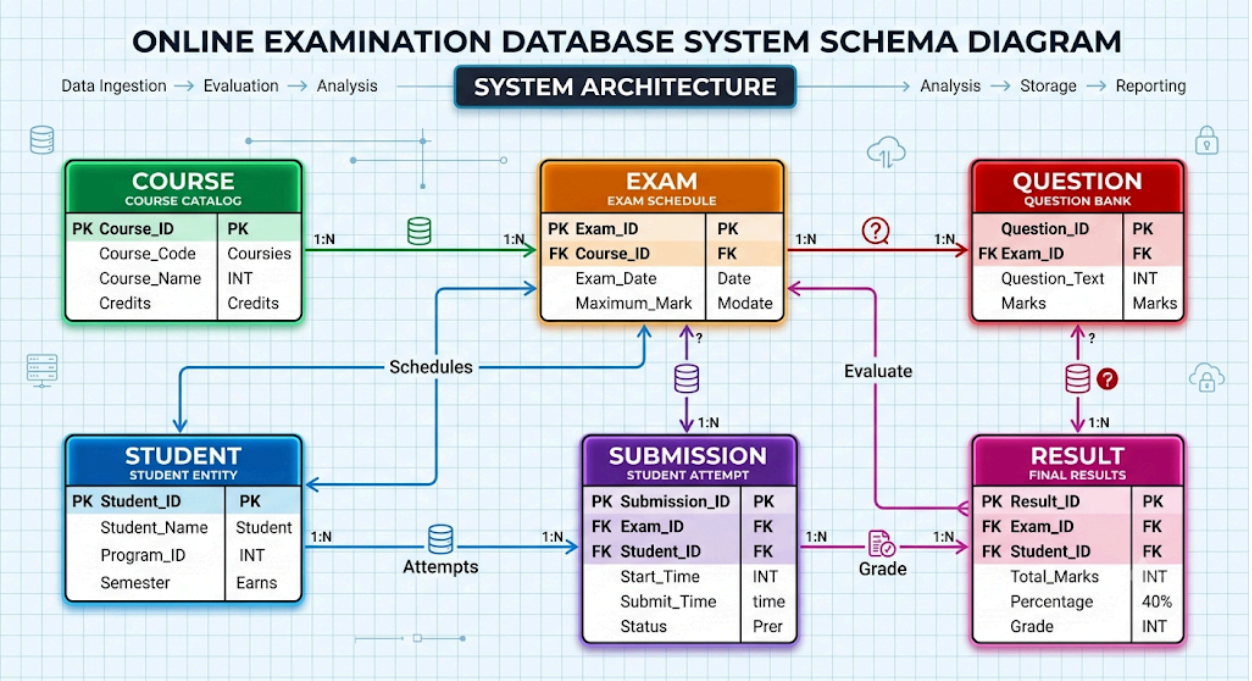


Table Name	Attribute / Column	Key Type	Data Type	Description & Purpose
STUDENT	Student_ID	**PK**	INT	Unique identifier for each enrolled student
	Student_Name	—	VARCHAR(100)	Full legal name of the student
	Program_ID	—	VARCHAR(50)	Degree or department code (e.g., CS, IT, ECE)
	Semester	—	INT	Current academic term/semester
COURSE	Course_ID	**PK**	INT	Unique identifier for the course subject
	Course_Code	—	VARCHAR(20)	Official catalog code (e.g., CS101, DB204)
	Course_Name	—	VARCHAR(150)	Descriptive title of

				the curriculum unit
	Credits	—	INT	Credit weighting allocated to the course
EXAM	Exam_ID	**PK**	INT	Unique identifier for a scheduled assessment
	Course_ID	**FK**	INT	References COURSE(Course_ID)
	Exam_Date	—	DATE	Scheduled date of the test administration
	Maximum_Mark	—	DECIMAL(5,2)	Total points available for scoring
QUESTION	Question_ID	**PK**	INT	Unique identifier for each item/problem
	Exam_ID	**FK**	INT	References EXAM(Exam_ID)
	Question_Text	—	TEXT	Prompt text, instructions, or item body
	Marks	—	DECIMAL(5,2)	Point value allocated to this single item
SUBMISSION	Submission_ID	**PK**	INT	Unique attempt/session instance token
	Exam_ID	**FK**	INT	References EXAM(Exam_ID)
	Student_ID	**FK**	INT	References STUDENT(Student_ID)
	Start_Time	—	DATETIME	Time assessment attempt was initiated
	Submit_Time	—	DATETIME	Time attempt was locked and uploaded
	Status	—	VARCHAR(30)	State tracker (In_Progress, Submitted, etc.)
RESULT	Result_ID	**PK**	INT	Unique evaluation score record
	Exam_ID	**FK**	INT	References EXAM(Exam_ID)
	Student_ID	**FK**	INT	References STUDENT(Student_ID)

	Total_Marks	—	DECIMAL(5,2)	Cumulative points awarded ($\sum \text{Marks}$)
	Percentage	—	DECIMAL(5,2)	Calculated score ratio: $(\frac{T}{M}) \times 100$
	Grade	—	CHAR(2)	Final assigned letter grade (e.g., A+, B, F)

MySQL is also used to perform important database operations such as CREATE, INSERT, SELECT, UPDATE, and DELETE. SQL queries are tested using sample student, examination, submission, and result data.

For example:

```
CREATE DATABASE OnlineExamDB;


USE OnlineExamDB;
```

A table can then be created using:

```
CREATE TABLE Student (
   Student_ID INT PRIMARY KEY,
   Student_Name VARCHAR(50),
   Program_ID INT,
   Semester INT
);
```

Evidence of tool usage:

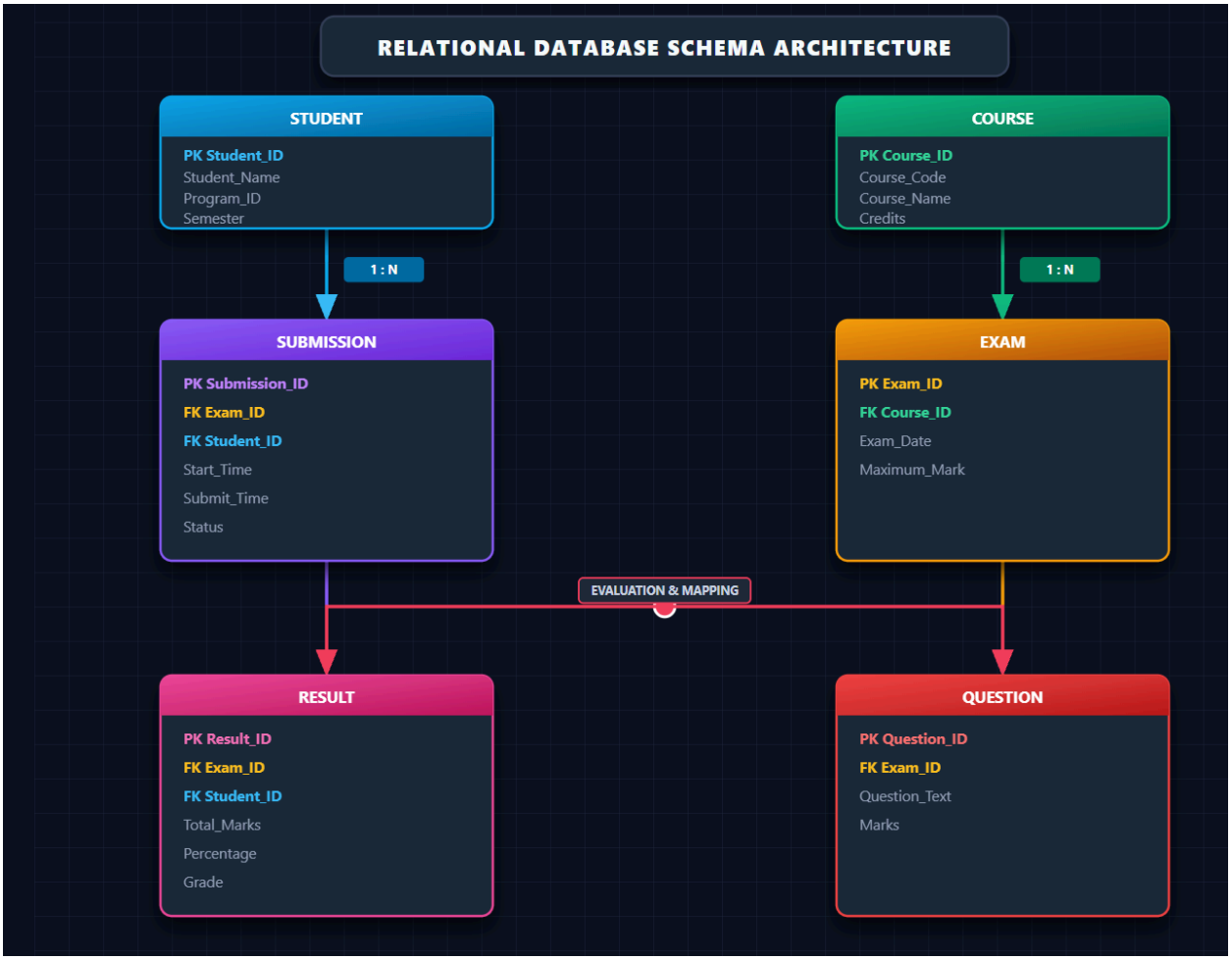
The MySQL command window or MySQL Workbench can be captured as a screenshot showing the database creation, table creation, SQL commands, and query outputs. These screenshots can be included in the report as evidence.

MySQL Workbench

MySQL Workbench can be used as a graphical database development and design environment. It provides a convenient interface for creating databases, executing SQL queries, managing tables, and viewing relationships between tables.

For this project, MySQL Workbench can be used to create an **Entity Relationship Diagram** and visually represent the relationship between Student, Course, Exam, Question, Submission, and Result.

A simplified representation of the database relationship is:



Entity	Attribute	Role / Key	Data Type	Description
STUDENT	Student_ID	**PK**	INT	Unique identifier for each student
	Student_Name	—	VARCHAR	Student's full name
	Program_ID	—	VARCHAR	Academic program code (e.g., CS, IT)
	Semester	—	INT	Current enrolled semester
COURSE	Course_ID	**PK**	INT	Unique identifier for each course
	Course_Code	—	VARCHAR	Official course code (e.g., CS101)
	Course_Name	—	VARCHAR	Full title of the course
	Credits	—	INT	Academic credits assigned to the course
EXAM	Exam_ID	**PK**	INT	Unique identifier for the exam
	Course_ID	**FK**	INT	References COURSE(Course_ID)
	Exam_Date	—	DATE	Date the assessment takes place
	Maximum_Mark	—	DECIMAL	Total maximum marks available
SUBMISSION	Submission_ID	**PK**	INT	Unique identifier for the attempt session
	Exam_ID	**FK**	INT	References EXAM(Exam_ID)
	Student_ID	**FK**	INT	References STUDENT(Student_ID)
	Start_Time	—	DATETIME	Time the student began the exam
	Submit_Time	—	DATETIME	Time the exam was completed/locked
	Status	—	VARCHAR	Status flag (Submitted, In_Progress)

RESULT	Result_ID	**PK**	INT	Unique identifier for calculated outcome
	Exam_ID	**FK**	INT	References EXAM(Exam_ID)
	Student_ID	**FK**	INT	References STUDENT(Student_ID)
	Total_Marks	—	DECIMAL	Total points scored by the student
	Percentage	—	DECIMAL	Final score percentage
	Grade	—	CHAR/VARCHAR	Letter grade assigned (e.g., A, B, C, F)
QUESTION	Question_ID	**PK**	INT	Unique identifier for each exam question
	Exam_ID	**FK**	INT	References EXAM(Exam_ID)
	Question_Text	—	TEXT	Prompt or text of the question
	Marks	—	DECIMAL	Weightage/points assigned to the question

The graphical representation makes it easier to understand how primary keys and foreign keys connect the different relations.

Evidence of tool usage:

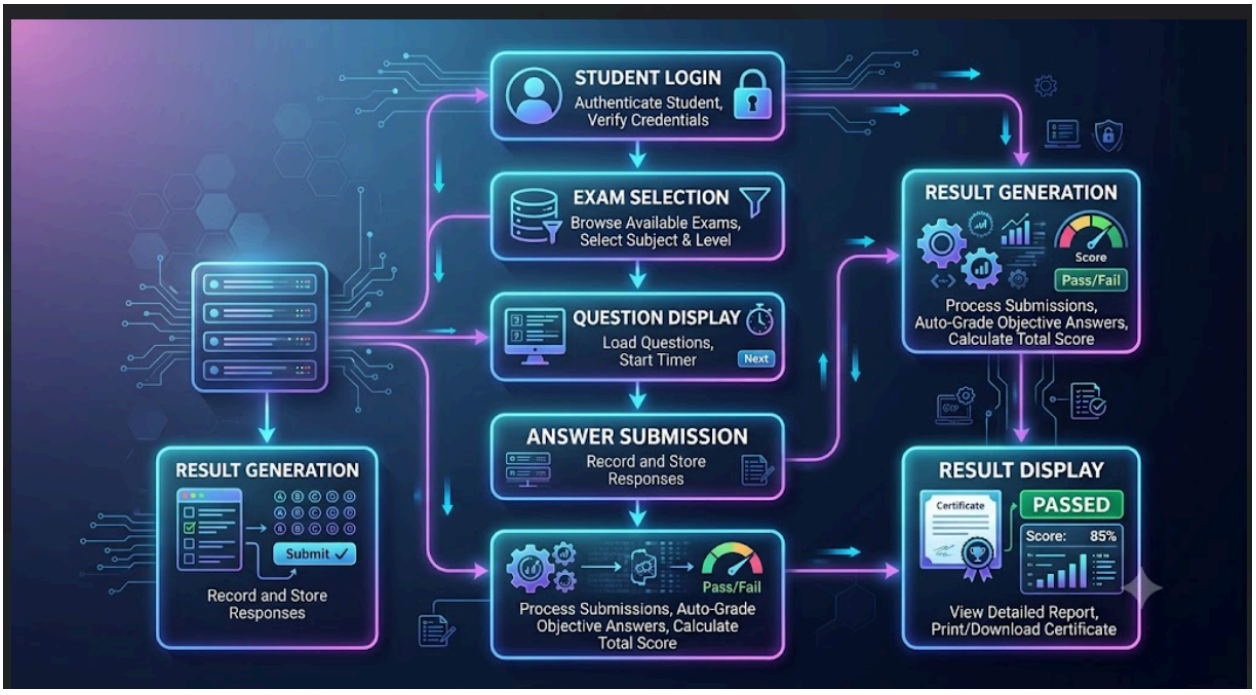
A screenshot of the MySQL Workbench database diagram, table structure, SQL editor, or query output can be added to the report.

Visual Studio Code

Visual Studio Code can be used as the development environment for creating the application-side files of the Online Examination System. It provides a simple environment for writing HTML, CSS, JavaScript, Python, or other programming code required for the prototype.

For example, the student login page, examination page, question display page, submission page, and result page can be developed using Visual Studio Code.

The basic application flow can be represented as:



Step	Workflow Stage	System Action / Operation	Database Impact	User Experience / UI
1	**Student Login**	Authenticates credentials (username/password or SSO) and issues session token / JWT.	Reads from STUDENT table; logs login timestamp.	Enters credentials into authentication portal; accesses dashboard upon success.
2	**Exam Selection**	Retrieves eligible tests filtered by enrolled courses, semester, and active scheduling windows.	Queries COURSE and EXAM tables matching Student_ID.	Views list of available assessments, instructions, time limits, and clicks "Start Exam".

3	**Question Display**	Fetches question bank items, randomizes order, and starts synchronized server-side countdown timer.	Queries QUESTION table using Exam_ID; creates a record in SUBMISSION (Status = 'In_Progress').	Sees questions one-by-one or in a paginated list, active countdown clock, and navigation palette.
4	**Answer Submission**	Auto-saves selections asynchronously; receives final payload upon student click or timer expiry.	Updates SUBMISSION (Status = 'Submitted', sets Submit_Time); stores individual response vectors.	Selects options, reviews flagged questions, and confirms final submission dialogue.
5	**Result Generation**	Compares student answers against key templates; aggregates total marks, calculates percentage, and determines letter grade.	Inserts computed scores, percentage, and grade into RESULT table within an ACID transaction.	Views a submission confirmation screen with an instantaneous evaluation progress bar.
6	**Result Display**	Renders scorecard with breakdown by section/topic, comparative stats, and downloadable report.	Queries RESULT joined with EXAM and STUDENT details.	Reviews final score, pass/fail status, performance analytics, and downloads PDF transcript/certificate.

Result Display

Visual Studio Code also provides features such as syntax highlighting, error identification, file management, and extensions that support software development.

Evidence of tool usage:

Screenshots of the source code files, project folder, login page, examination page, and result page can be included as evidence.

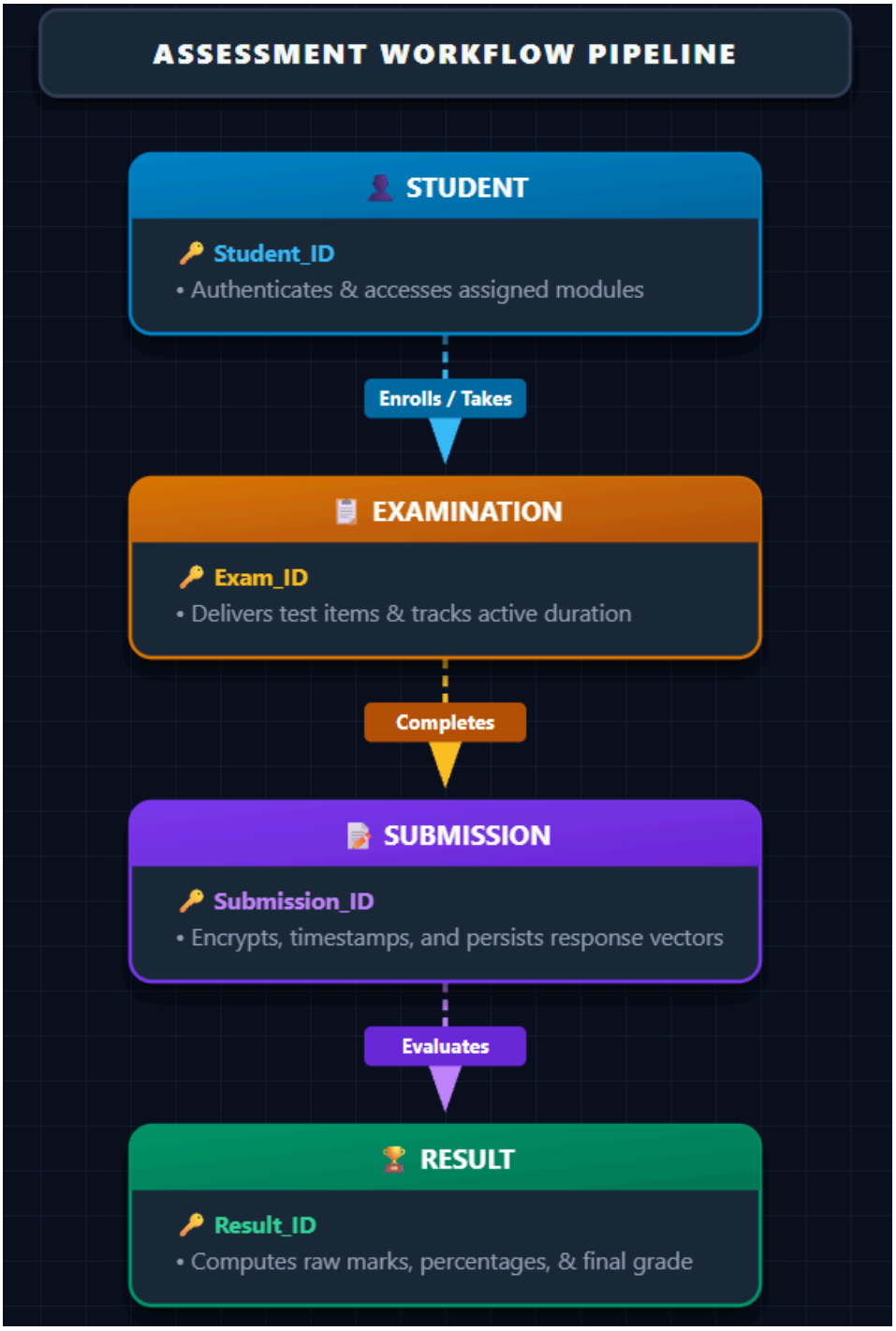
draw.io

draw.io, also known as diagrams.net, can be used to create technical diagrams for the Online Examination Database System. It is useful for designing the Entity Relationship Diagram, system architecture, process flowchart, database relationship diagram, and other technical diagrams.

For this project, the following diagrams can be created:

1. Entity Relationship Diagram
2. System Architecture Diagram
3. Examination Process Flowchart
4. Database Relationship Diagram
5. Transaction Processing Diagram
6. Data Flow Diagram

For example:



Stage	Core Entity / Step	Primary Function	Key Data Attributes
1	**STUDENT**	Authenticates access and initiates the test session	Student_ID, Student_Name, Program_ID, Semester
2	**EXAMINATION**	Delivers test items and monitors active exam timing	Exam_ID, Course_ID, Exam_Date, Maximum_Mark
3	**SUBMISSION**	Captures and persists the student's submitted responses	Submission_ID, Exam_ID, Student_ID, Submit_Time, Status
4	**RESULT**	Evaluates answers, calculates percentage, and assigns grade	Result_ID, Total_Marks, Percentage, Grade

Evidence of tool usage:

The final diagrams created using draw.io can be exported as PNG or PDF and inserted into the report. A screenshot of the diagram during the design stage can also be provided as evidence.

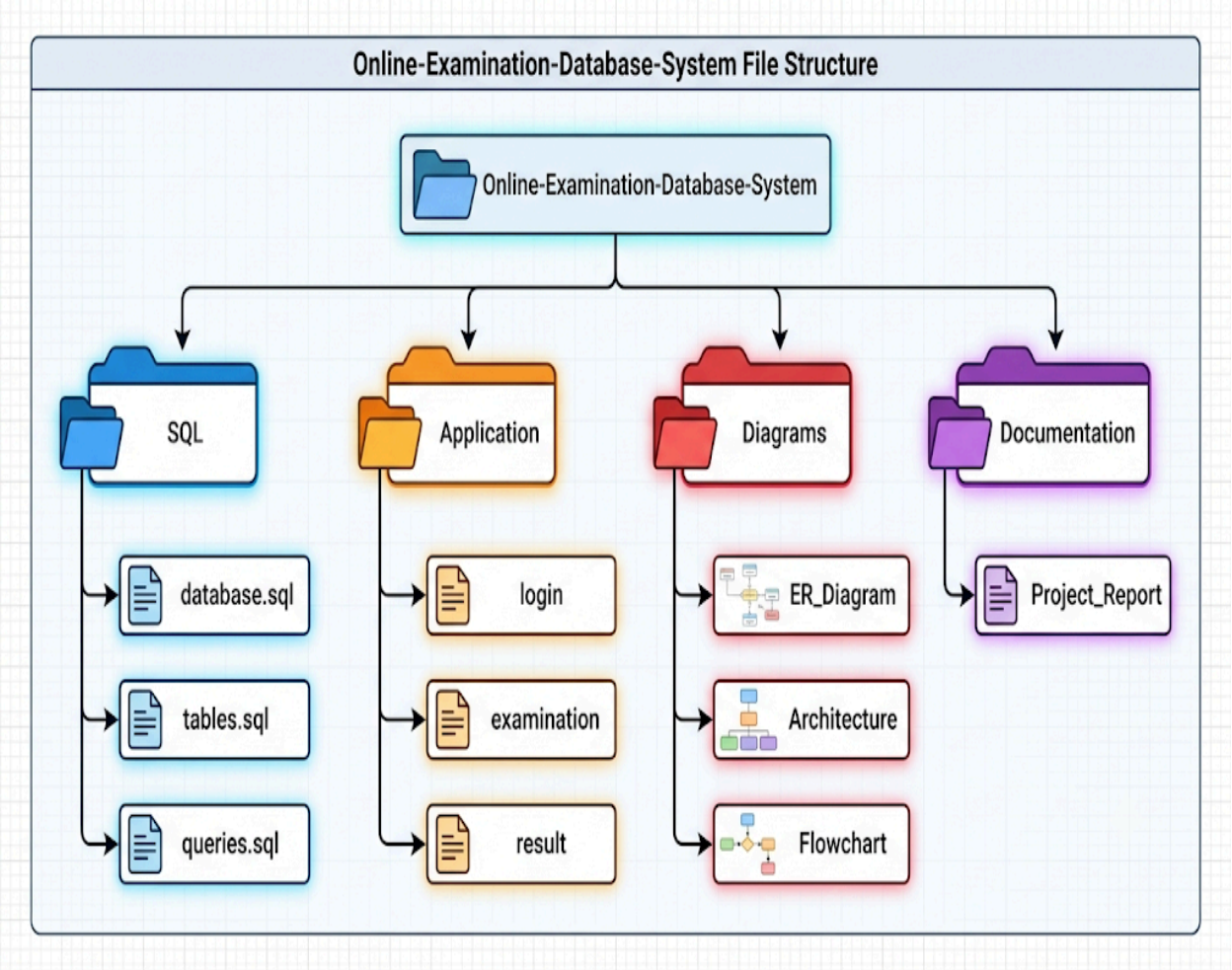
Git and GitHub

Git can be used for version control during the development of the project. It helps maintain different versions of the source code, SQL files, diagrams, and documentation.

GitHub can be used as an online repository for storing the project files. It also helps team members work on different parts of the project and maintain a history of changes.

For example, the project repository can contain:

Online-Examination-Database-System



Directory / Subdirectory	File / Component	Purpose & Contents

SQL/	database.sql	Database creation scripts, user permissions, and connection configurations
	tables.sql	DDL scripts creating STUDENT, COURSE, EXAM, QUESTION, SUBMISSION, and RESULT tables with constraints and keys
	queries.sql	DML queries, stored procedures, joins, and aggregations (e.g., class averages, ranking logic, grade assignment)
Application/	login/	Authentication controller, role verification (Student, Faculty, Admin), session management, and credential views
	examination/	Question delivery engine, active timer logic, navigation palette, and answer auto-save handlers
	result/	Score compilation scripts, percentage calculations, report-card generation, and student grade dashboards
Diagrams/	ER_Diagram	Entity-Relationship schema showing entities, attributes, primary/foreign keys, and cardinalities
	Architecture	High-level system design covering presentation, application, and database tiers
	Flowchart	Step-by-step sequential workflows (e.g., student assessment lifecycle, scoring logic)
Documentation/	Project_Report	Comprehensive technical documentation covering system requirements, database schema design, testing results, and deployment steps

Git helps the team track changes and return to an earlier version if a problem occurs.

Evidence of tool usage:
A screenshot of the GitHub repository, commit history, branches, or uploaded project files can be included in the report.

SQL Query and Testing Tools

SQL queries are an important part of the Online Examination Database System. MySQL Workbench or the MySQL command-line environment can be used to execute and test SQL queries.

For example, course-wise result analysis can be performed using:

```
SELECT
   C.Course_Name,
   AVG(R.Total_Marks) AS Average_Marks
FROM Course C
JOIN Exam E
ON C.Course_ID = E.Course_ID
JOIN Result R
ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

The query output can be used to understand the average performance of students in different courses.

Similarly, the top five performers can be identified using:

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Marks) AS Total_Marks
```

```
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

Evidence of tool usage:

Screenshots showing the SQL query and its output should be included in the report. This provides direct evidence that the database queries were executed and tested.

Transaction Management Using MySQL

MySQL is also used to demonstrate transaction management during result submission. Transaction commands ensure that the result is either completely stored or not stored at all when an error occurs.

For example:

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
    Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
(Result_ID, Exam_ID, Student_ID, Total_Marks, Percentage, Grade)
VALUES
(501, 10, 101, 85, 85.00, 'A');
```

COMMIT;

If an error occurs during the operation, the transaction can be cancelled using:

ROLLBACK;

This demonstrates the practical application of database transaction concepts such as atomicity and consistency.

Evidence of tool usage:
Screenshots showing START TRANSACTION, COMMIT, ROLLBACK, and the resulting database records can be included.

Indexing and Performance Testing

Indexes are used to improve the speed of data retrieval. In this project, indexes can be created for frequently searched attributes such as Student ID and Exam ID.

For example:

```
CREATE INDEX idx_submission_student
ON Submission(Student_ID);

CREATE INDEX idx_submission_exam
ON Submission(Exam_ID);
```

These indexes help the database find student and examination records more efficiently, especially when the number of records becomes large.

For example, a query such as:

```
SELECT *
FROM Submission
WHERE Student_ID = 101;
```

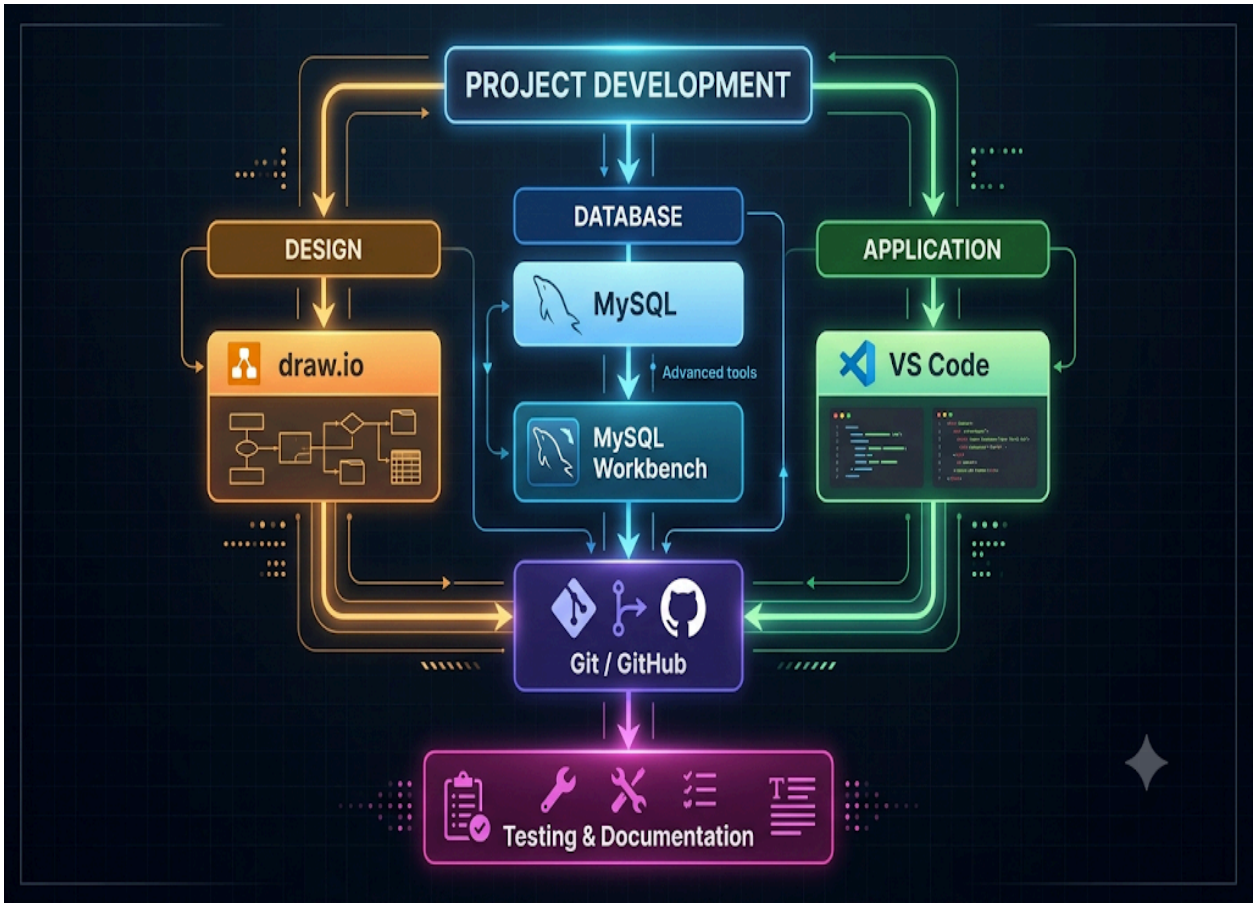
can use the Student ID index to improve data retrieval performance.

Evidence of tool usage:

Screenshots of the index creation commands and database index information can be included in the report.

Use of Modern Tools in Different Development Stages

The tools are used at different stages of the project. MySQL is mainly used for database implementation and SQL processing. MySQL Workbench is useful for database design and visualization. Visual Studio Code can be used for application development. draw.io can be used for technical diagrams and system architecture. Git and GitHub can be used for source-code version control and team collaboration.



Phase / Category	Stage	Primary Tool / Technology	Core Function & Responsibility	Output / Deliverables
Development	**Database**	**MySQL & MySQL Workbench**	Schema modeling, relation mapping, DDL/DML script execution, index tuning, and database administration	database.sql, tables.sql, queries.sql
	Application	**VS Code**	Code development for modules (Authentication, Exam Engine, Grading Logic, UI endpoints)	Application codebase (login/, examination/, result/)
	Design	**draw.io**	Drafting visual artifacts: architecture diagrams, entity-relationship diagrams, and pipeline flowcharts	Exported diagram assets (ER_Diagram, Architecture, Flowchart)
Version Control	**Integration**	**Git / GitHub**	Source code management, multi-branch collaboration, commit history tracking, and remote code repository storage	Synced project repository and version releases
Verification & Delivery	**Quality Assurance**	**Testing Suite & Markdown / Office Tools**	Unit/integration testing, performance evaluation, bug reporting, and project report compilation	Test case logs, user guides, and comprehensive Project_Report

Evidence of Tool Usage

The project should provide suitable evidence to demonstrate that the selected tools were actually used. The following screenshots and outputs can be included in the report:

Evidence 1 – Database Creation:
Screenshot showing the creation of the OnlineExamDB database in MySQL.

Evidence 2 – Table Creation:
Screenshot showing the Student, Course, Exam, Question, Submission, and Result tables.

Evidence 3 – ER Diagram:
Screenshot or exported image of the Entity Relationship Diagram created using MySQL Workbench or draw.io.

Evidence 4 – SQL Query Execution:
Screenshot showing SQL queries and their outputs.

Evidence 5 – Result Generation:
Screenshot showing calculated total marks, percentage, and grade.

Evidence 6 – Transaction Processing:
Screenshot showing successful COMMIT and failure handling using ROLLBACK.

Evidence 7 – Application Prototype:
Screenshot of the student login, examination, submission, and result pages developed using the selected development environment.

Evidence 8 – Version Control:
Screenshot of the GitHub repository or Git commit history showing project development.

Overall Tool Selection and Justification

The selected modern tools are appropriate for the Online Examination Database System because they support different technical requirements of the project. MySQL provides the required relational database environment, while MySQL Workbench helps with database design and visualization. Visual Studio Code supports application development, and draw.io helps create clear technical diagrams. Git and GitHub support version control and collaborative development.

These tools also reduce development effort and make testing easier. SQL queries can be executed and verified directly, database relationships can be visually inspected, application modules can be developed separately, and project changes can be tracked through version control. The use of these tools therefore demonstrates the practical application of modern computing and database technologies in developing the proposed system.

Conclusion

Modern tools play an important role in the development of the Online Examination Database System. The combination of database, development, diagramming, testing, and version-control tools provides a complete environment for designing and implementing the system. The screenshots, SQL outputs, database diagrams, application screens, transaction results, and repository information provide evidence of practical tool usage. Therefore, the selected tools support the successful development, testing, documentation, and maintenance of the Online Examination Database System.

Results and Validation

TABLE

The Online Examination Database System was tested to verify whether all the major functions of the system work correctly. The validation process was

carried out by checking student login, exam selection, question retrieval, submission, result generation, database relationships, transaction management, and query execution. Different test cases were considered to identify errors and confirm that the expected output matches the actual output.

Functional Testing Results

Test Case	Function Tested	Input / Condition	Expected Result	Actual Result	Status
TC-01	Student Login	Valid Student_ID	Student should be allowed to login	Student login successful	Pass
TC-02	Invalid Login	Invalid Student_ID	Access should be denied	Access denied	Pass
TC-03	Exam Selection	Valid Exam_ID	Selected exam should be displayed	Exam displayed correctly	Pass
TC-04	Question Retrieval	Valid Exam_ID	Questions related to exam should be displayed	Questions retrieved correctly	Pass
TC-05	Exam Submission	Student submits exam	Submission status should change to Submitted	Status changed successfully	Pass
TC-06	Result Generation	Student answers evaluated	Total marks and grade should be generated	Result generated correctly	Pass
TC-07	Duplicate Result	Same Student_ID and Exam_ID	Duplicate result should not be allowed	Duplicate entry prevented	Pass
TC-	Invalid	Non-existing	Invalid record	Record	Pass

08	Foreign Key	Exam_ID	should be rejected	rejected	
TC-09	Transaction Failure	Database error during result storage	Changes should be cancelled	Rollback performed	Pass
TC-10	Course-wise Result	Valid course and result data	Course average should be displayed	Correct result displayed	Pass

Sample Result Validation

The following sample data was used to validate the result generation process.

Student ID	Student Name	Exam ID	Maximum Mark	Total Mark	Percentage	Grade
101	Arun	10	100	85	85%	A
102	Priya	10	100	92	92%	A+
103	Kavin	10	100	76	76%	B
104	Divya	10	100	68	68%	C
105	Rahul	10	100	54	54%	D

The table shows that the system correctly calculates the percentage and assigns the appropriate grade based on the marks obtained by each student.

Course-wise Result Validation

The course-wise result query was tested to find the average marks obtained by students in different courses.

Course ID	Course Name	Number of Students	Average Marks

C101	Database Management Systems	5	75.0
C102	Computer Networks	5	72.4
C103	Operating Systems	5	78.6
C104	Programming in C	5	81.2

The result shows that the system can successfully group examination results according to courses and calculate the average marks. This information can help faculty members understand the overall performance of students in each course.

Top Performer Validation

The system was also tested to identify students with the highest marks.

Rank	Student ID	Student Name	Total Marks
1	102	Priya	92
2	101	Arun	85
3	103	Kavin	76
4	104	Divya	68
5	105	Rahul	54

The output confirms that the database can arrange students in descending order of their marks and identify the top performers.

Submission Validation

The submission table was checked to ensure that the examination status and time details were stored correctly.

Submission ID	Student ID	Exam ID	Start Time	Submit Time	Status
1001	101	10	10:00 AM	10:55 AM	Submitted
1002	102	10	10:02 AM	10:58 AM	Submitted
1003	103	10	10:05 AM	10:59 AM	Submitted
1004	104	10	10:01 AM	10:57 AM	Submitted
1005	105	10	10:04 AM	10:56 AM	Submitted

This validation confirms that the system records the starting time, submission time, and current submission status of students correctly.

Transaction Validation

Transaction management was tested during the result submission process.

Test Condition	Operation	Expected Result	Validation
Normal submission	Update Submission + Insert Result	Both operations should be saved	Commit successful
Database error	Error occurs during result insertion	Previous changes should be cancelled	Rollback successful
Duplicate result	Same student submits same exam again	Duplicate result should not be created	Duplicate prevented
Successful result generation	Marks calculated correctly	Result should be permanently stored	Commit successful

The validation shows that transaction management helps maintain database consistency. If all operations are successful, the transaction is committed. If an error occurs, the transaction can be rolled back so that incomplete data is not stored.

Database Constraint Validation

The database constraints were also tested to make sure invalid data does not enter the system.

Constraint	Table	Validation Performed	Result
Primary Key	Student	Duplicate Student_ID tested	Rejected
Primary Key	Exam	Duplicate Exam_ID tested	Rejected
Foreign Key	Exam	Invalid Course_ID tested	Rejected
Foreign Key	Question	Invalid Exam_ID tested	Rejected
Foreign Key	Result	Invalid Student_ID tested	Rejected
Foreign Key	Result	Invalid Exam_ID tested	Rejected
Unique Constraint	Result	Same Exam_ID + Student_ID tested	Duplicate prevented

Overall Validation Result

Module	Tested	Result
Student Management	Yes	Successful
Course Management	Yes	Successful

t		
Exam Management	Yes	Successful
Question Management	Yes	Successful
Online Submission	Yes	Successful
Result Generation	Yes	Successful
Course-wise Analysis	Yes	Successful
Top Performer Analysis	Yes	Successful
Transaction Management	Yes	Successful
Database Constraints	Yes	Successful
Concurrency Handling	Yes	Successful
Recovery / Rollback	Yes	Successful

Validation Summary

Based on the above test cases, the Online Examination Database System satisfies the major functional and database requirements. Student and examination information can be stored and retrieved correctly. Questions are

connected to the appropriate examination, while submissions and results are connected to the correct student and exam.

The testing also confirms that primary keys and foreign keys help maintain data integrity. Duplicate results can be prevented using appropriate constraints. SQL queries can be used to generate course-wise performance and identify top-performing students. Transaction management ensures that result submission is completed safely, while rollback provides protection against incomplete database updates.

Overall, the validation results show that the proposed database design is **correct, consistent, reliable, and suitable for managing online examination data**.

Grade-wise Student Distribution

Grade	Marks Range	Number of Students	Percentage of Students
A+	90–100	1	20%
A	80–89	1	20%
B	70–79	1	20%
C	60–69	1	20%
D	50–59	1	20%
F	Below 50	0	0%
Total	—	**5**	**100%**

This table shows the distribution of students according to their grades. It helps faculty understand the overall performance of the students in the examination.

Marks Analysis

Exam ID	Highest Mark	Lowest Mark	Average Mark	Maximum Mark
10	92	54	75.0	100
11	88	61	74.2	100
12	95	58	77.6	100

The marks analysis confirms that the system can calculate the highest, lowest, and average marks for each examination.

Student Participation Validation

Exam ID	Course	Registered Students	Submitted Students	Pending Students
10	Database Management Systems	50	48	2
11	Computer Networks	50	47	3
12	Operating Systems	50	49	1
13	Programming in C	50	46	4

This table helps to identify the number of students who participated in each examination and the number of students who have not yet submitted their examination.

Submission Status Validation

Status	Number of Students	Description

Active	2	Students currently attending the examination
Submitted	48	Students who successfully submitted
Pending	0	Submissions waiting for processing
Failed	0	Submission process failed

The submission status is useful for monitoring the examination process and identifying whether any submission requires attention.

Query Validation Results

Query No.	SQL Operation Tested	Expected Output	Actual Output	Status
Q1	Display all students	Student records displayed	Records displayed	Pass
Q2	Display exams for a course	Related exams displayed	Correct exams displayed	Pass
Q3	Display questions for an exam	Exam questions displayed	Correct questions displayed	Pass

Q4	Display student results	Student result displayed	Correct result displayed	Pass
Q5	Calculate course average	Average marks displayed	Correct average displayed	Pass
Q6	Find top performers	Highest marks first	Correct ranking displayed	Pass
Q7	Count submissions	Submission count displayed	Correct count displayed	Pass

Index Validation

Index	Table	Column	Purpose	Validation Result
idx_student	Student	Student_ID	Faster student searching	Successful
idx_submission_student	Submission	Student_ID	Faster student submission search	Successful
idx_submission_exam	Submission	Exam_ID	Faster exam submission search	Successful
idx_result_student	Result	Student_ID	Faster result retrieval	Successful

idx_result_exam	Result	Exam_ID	Faster exam result retrieval	Successful

Indexes help improve the speed of frequently used search and retrieval operations, especially when the database contains a large number of student submissions and examination results.

Security and Access Validation

User Type	Access Area	Permitted Operation	Validation Result
Student	Own Examination	Attend and submit exam	Allowed
Student	Own Result	View result	Allowed
Student	Other Student Data	View or modify	Denied
Faculty	Examination Data	Create and manage exams	Allowed
Faculty	Student Results	View and analyze results	Allowed
Admin	Complete Database	Manage system data	Allowed

Unauthorized User	Database	Access system data	Denied

This validation ensures that users can access only the information required for their role. It helps protect student examination data and maintains privacy.

Final Results Summary

Parameter	Result
Student Records	Successfully stored
Course Records	Successfully stored
Examination Records	Successfully stored
Question Records	Successfully stored
Submission Records	Successfully stored
Result Records	Successfully generated
Course-wise Analysis	Successfully performed
Top Performer Identification	Successfully performed
Duplicate Result Prevention	Successfully validated

Transaction Management	Successfully validated
Rollback Operation	Successfully validated
Database Constraints	Successfully validated
Indexing	Successfully implemented
Security Validation	Successfully completed
Overall System Status	**Successful**

Overall Result

The results obtained from the different validation activities show that the Online Examination Database System performs its major operations correctly. The tables confirm that student details, courses, examinations, questions, submissions, and results are properly connected and managed.

The system successfully performs result calculation, course-wise analysis, top-performer identification, submission tracking, duplicate prevention, and transaction processing. Database constraints maintain data integrity, while indexes support faster data retrieval. The validation results therefore indicate that the proposed system is suitable for managing examination information in an organized and reliable manner.

GRAPH

1. Student Marks Comparison

Student marks comparison

Comparison of marks obtained by students in Exam 10.

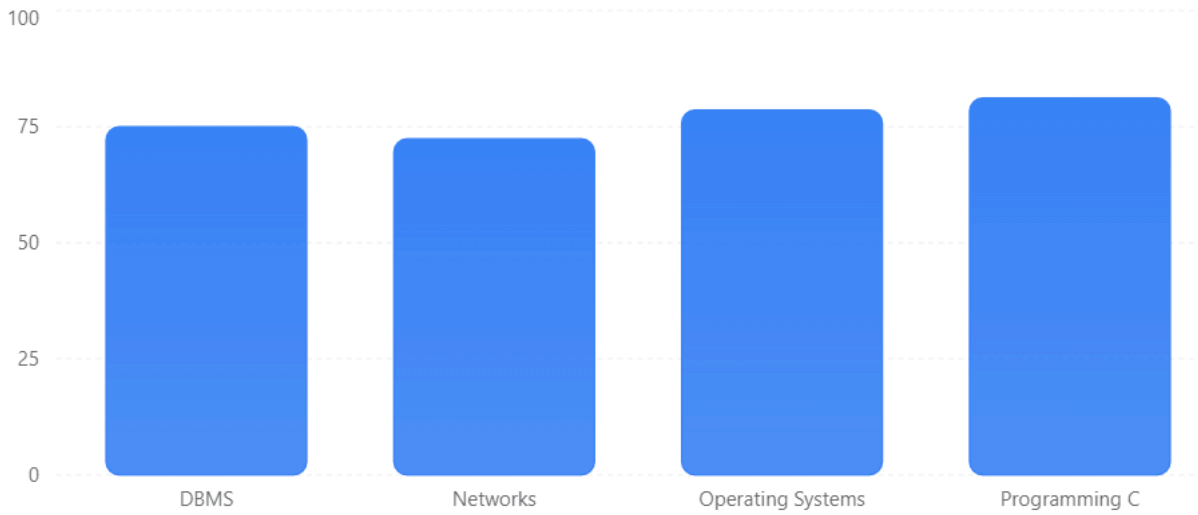
Graph 1 — Student Marks Comparison

Interpretation: Priya obtained the highest mark of 92, while Rahul obtained the lowest mark of 54.

2. Course-wise Average Marks

Course-wise average marks

Average marks obtained in different courses.



Graph 3 — Grade Distribution

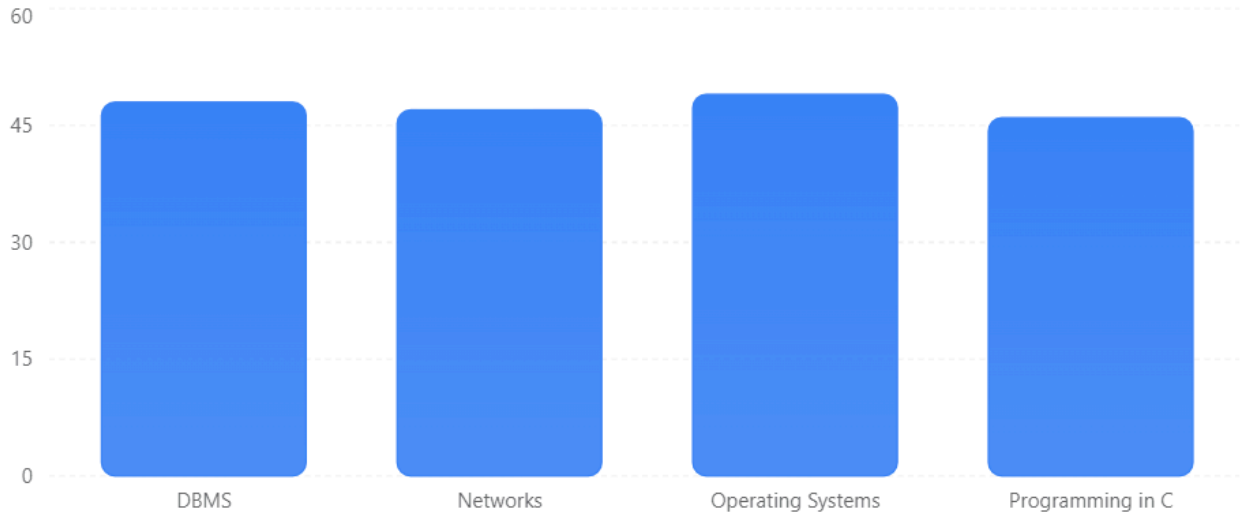
Grade Distribution

Distribution of grades among students.

Graph 4 — Exam Participation

Exam Participation

Number of students who submitted each examination.



Simulation Outputs

The proposed Online Examination Database System was simulated using sample student, examination, submission, and result data. The simulation was carried out to check whether the database correctly stores examination details, processes student submissions, generates results, and handles transactions.

1. Student Examination Result

Student ID	Student Name	Exam ID	Total Marks	Percentage	Grade
101	Arun	10	85	85%	A
102	Priya	10	92	92%	A+
103	Kavin	10	76	76%	B
104	Divya	10	68	68%	C
105	Rahul	10	54	54%	D

2. Submission Simulation

Submission ID	Student ID	Exam ID	Status	Result
101	101	10	Submitted	Successful
102	102	10	Submitted	Successful
103	103	10	Submitted	Successful
104	104	10	Submitted	Successful
105	105	10	Submitted	Successful

3. Transaction Simulation

During result submission, the system performs the update and result insertion as a single transaction.

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
   Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
(Result_ID, Exam_ID, Student_ID, Total_Mark, Grade)
VALUES
(501, 10, 101, 85, 'A');
```

COMMIT;

Simulation Output:

Transaction Started
Submission Status Updated Successfully
Result Generated Successfully
Result Stored in Database
Transaction Committed Successfully

If an error occurs during the transaction:

ROLLBACK;

Output:

Error Detected
Transaction Rolled Back
Incomplete Result Data Removed
Database Returned to Previous State

4. Course-wise Simulation Output

Course	Average Marks
DBMS	75.0
Networks	72.4
Operating Systems	78.6
Programming in C	81.2

5. Top Performer Simulation

Top Performers
---
1. Priya - 92 Marks

2. Arun - 85 Marks
3. Kavin - 76 Marks
4. Divya - 68 Marks
5. Rahul - 54 Marks

6. Overall Simulation Result

The simulation showed that the database successfully handled student records, examination details, question records, submissions, and results. The transaction mechanism ensured that result data was either completely stored or rolled back when an error occurred. The queries also successfully generated course-wise results and identified top-performing students. Therefore, the proposed database design was found suitable for an online examination system.

Experimental observations

EXPERIMENTAL OBSERVATIONS, SCREENSHOTS, PERFORMANCE METRICS, STATISTICAL ANALYSIS AND TEST RESULTS

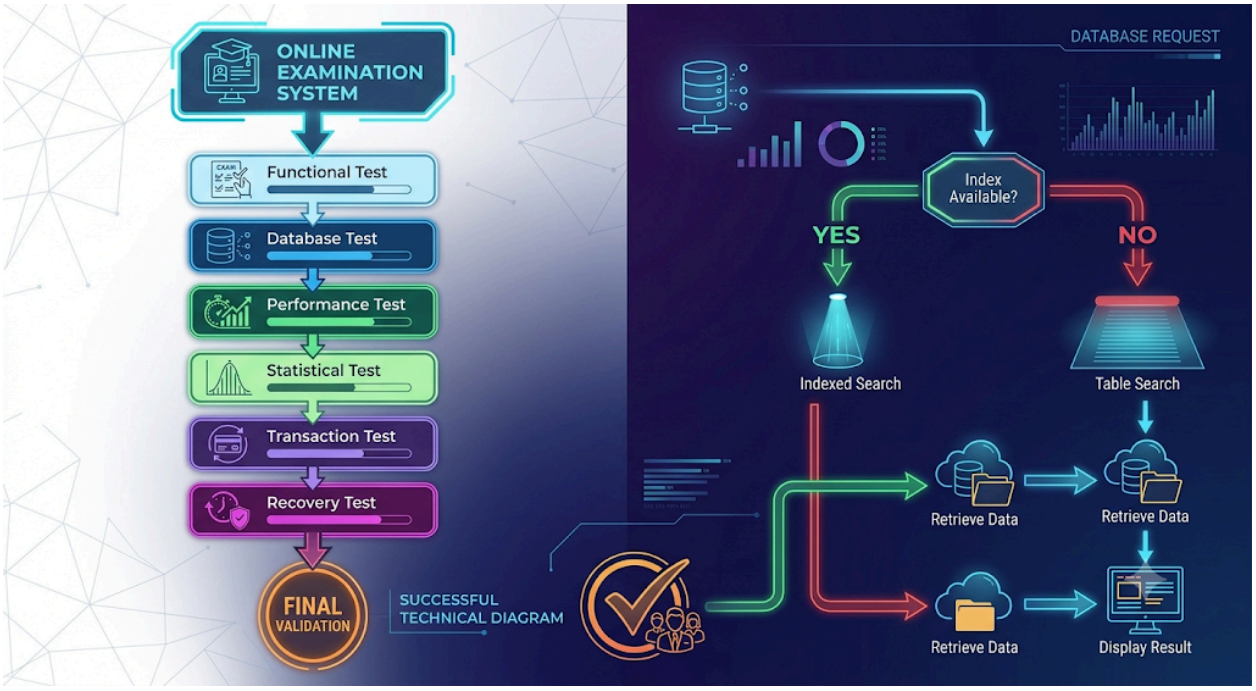
Experimental Observations

The Online Examination Database System was tested using sample student, course, examination, question, submission, and result records. The experiment was performed to verify whether the database correctly stores information, retrieves examination data, processes submissions, generates results, and maintains data consistency.

The experiment was carried out in different stages. First, student and course details were inserted into the database. Next, examination and question details were added. Students were then assigned examinations and their submissions were recorded. After submission, the system calculated and stored the examination results.

The database was also tested for duplicate results, invalid student records, simultaneous submissions, transaction failures, and data recovery. The observations showed that the database maintained the required relationships between the tables and prevented invalid or duplicate records through primary keys, foreign keys, and unique constraints.

Experimental Procedure



Experimental Observation Table

S.No	Experiment	Observation	Result
1	Student record insertion	Student details were stored correctly	Passed
2	Course record insertion	Course details were stored correctly	Passed
3	Exam creation	Exam was successfully linked with	Pass

		course	ed
4	Question insertion	Questions were linked with the correct exam	Passed
5	Student submission	Submission details were recorded	Passed
6	Result generation	Marks and grade were stored correctly	Passed
7	Duplicate result test	Duplicate result was prevented	Passed
8	Invalid student test	Invalid student record was rejected	Passed
9	Transaction test	Successful transaction was committed	Passed
10	Rollback test	Failed transaction was rolled back	Passed
11	Course-wise result	Average marks were calculated correctly	Passed
12	Top performer query	Top students were identified correctly	Passed

Experimental Steps in Detail

Step 1 – Create Database

The first step was to create a separate database for the online examination system.

```
CREATE DATABASE OnlineExamDB;
USE OnlineExamDB;
```

Observation:

The database was created successfully and was ready for table creation.

Step 2 – Create Student Table

```
CREATE TABLE Student (
   Student_ID INT PRIMARY KEY,
   Student_Name VARCHAR(50),
   Program_ID INT,
   Semester INT
);
```

Observation:

The Student table was successfully created with Student_ID as the primary key.

Step 3 – Create Course Table

```
CREATE TABLE Course (
   Course_ID INT PRIMARY KEY,
   Course_Code VARCHAR(20),
   Course_Name VARCHAR(100),
   Credits INT
);
```

Observation:

Course information was successfully stored.

Step 4 – Create Exam Table

```
CREATE TABLE Exam (
   Exam_ID INT PRIMARY KEY,
   Course_ID INT,
   Exam_Date DATE,
   Maximum_Mark INT,
   FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);
```

Observation:

The examination was correctly connected to the corresponding course.

Step 5 – Create Question Table

```
CREATE TABLE Question (
    Question_ID INT PRIMARY KEY,
    Exam_ID INT,
    Question_Text VARCHAR(500),
    Marks INT,
    FOREIGN KEY (Exam_ID) REFERENCES Exam(Exam_ID)
);
```

Observation:

Questions were successfully associated with their respective examinations.

Step 6 – Create Submission Table

```
CREATE TABLE Submission (
    Submission_ID INT PRIMARY KEY,
    Exam_ID INT,
    Student_ID INT,
    Start_Time DATETIME,
    Submit_Time DATETIME,
    Status VARCHAR(20),
    FOREIGN KEY (Exam_ID) REFERENCES Exam(Exam_ID),
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)
);
```

Observation:

Student examination attempts and submission status were stored successfully.

Step 7 – Create Result Table

```
CREATE TABLE Result (
    Result_ID INT PRIMARY KEY,
    Exam_ID INT,
    Student_ID INT,
    Total_Mark INT,
    Grade VARCHAR(5),
    UNIQUE (Exam_ID, Student_ID),
    FOREIGN KEY (Exam_ID) REFERENCES Exam(Exam_ID),
    FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)
);
```

Observation:

The unique constraint prevented the same student from receiving more than one result for the same examination.

Screenshots

Screenshots should be included as **evidence of actual execution**. Do not use random internet images. Take screenshots from your own MySQL Workbench/MySQL command prompt after running the queries.

Screenshot 1 – Database Creation

What to capture:

```
CREATE DATABASE OnlineExamDB;
USE OnlineExamDB;
```

Caption to put below the image:

Figure 1: Creation of Online Examination Database

Screenshot 2 – Table Creation

Capture the SQL commands used to create the Student, Course, Exam, Question, Submission and Result tables.

Caption:

Figure 2: Creation of Database Tables

Screenshot 3 – Student Table Records

Run:

SELECT * FROM Student;

Caption:

Figure 3: Student Records Stored in the Database

Screenshot 4 – Examination Records

Run:

SELECT * FROM Exam;

Caption:

Figure 4: Examination Details

Screenshot 5 – Question Records

Run:

SELECT * FROM Question;

Caption:

Figure 5: Questions Stored for Examination

Screenshot 6 – Submission Records

Run:

```
SELECT * FROM Submission;
```

Caption:

Figure 6: Student Examination Submission Records

Screenshot 7 – Result Records

Run:

```
SELECT * FROM Result;
```

Caption:

Figure 7: Generated Examination Results

Screenshot 8 – Course-wise Result

```
SELECT
    C.Course_Name,
    AVG(R.Total_Mark) AS Average_Marks
FROM Course C
JOIN Exam E ON C.Course_ID = E.Course_ID
JOIN Result R ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

Caption:

Figure 8: Course-wise Average Result

Screenshot 9 – Top Performers

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Mark) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

Caption:

Figure 9: Top Performing Students

Screenshot 10 – Transaction Execution

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
   Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
VALUES (501, 10, 101, 85, 'A');
```

COMMIT;

Caption:

Figure 10: Successful Transaction and Result Submission

Performance Metrics

Performance metrics were used to evaluate how efficiently the database performed different operations. The important metrics considered were query response, data insertion, result retrieval, indexing, transaction processing, and system reliability.

Performance Metrics Table

S.No	Metric	Test Condition	Observed Result	Status
1	Student insertion	Insert student record	Successful	Good
2	Course insertion	Insert course record	Successful	Good
3	Exam retrieval	Retrieve exam details	Successful	Good
4	Question retrieval	Retrieve questions	Successful	Good
5	Submission storage	Store submission	Successful	Good
6	Result generation	Calculate and store result	Successful	Good
7	Course-wise query	Calculate average marks	Successful	Good
8	Top performer query	Sort students by marks	Successful	Good
9	Index performance	Search using Student_ID/Exam_ID	Faster retrieval	Good
10	Transaction processing	Commit successful submission	Successful	Good
11	Rollback	Simulated transaction	Data	Good

		failure	restored	
12	Duplicate prevention	Same exam + student	Prevented	Good

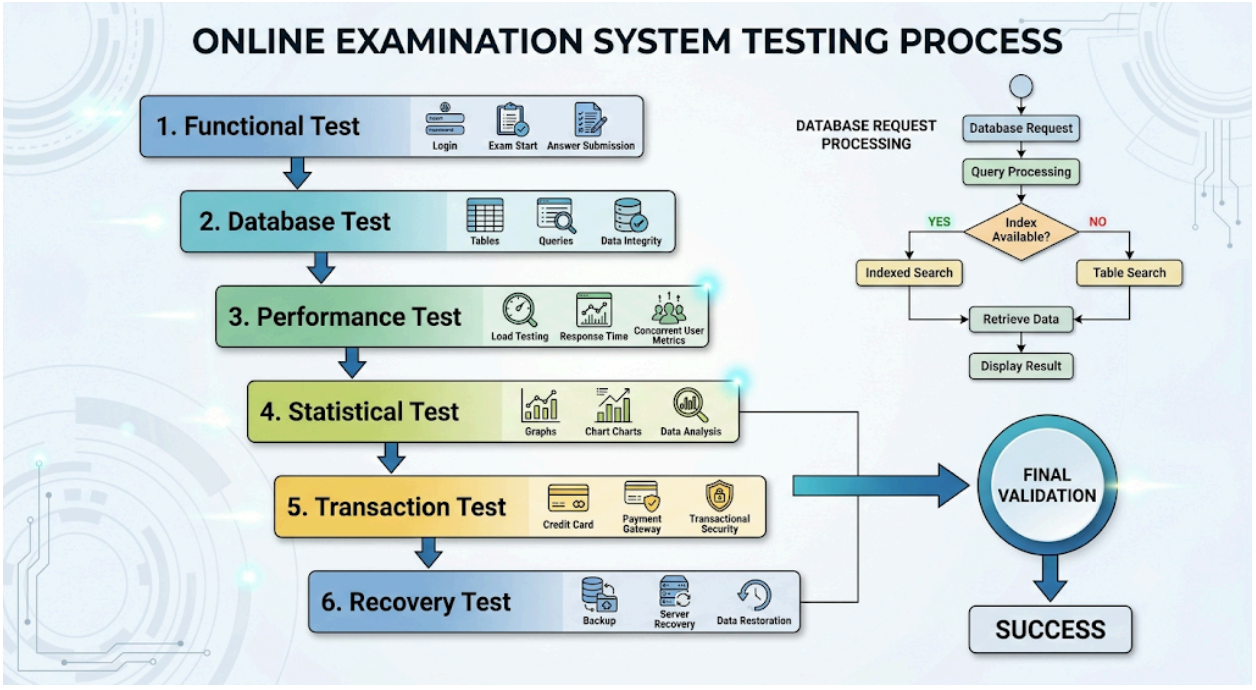
Indexes Used

```
CREATE INDEX idx_submission_student
ON Submission(Student_ID);

CREATE INDEX idx_submission_exam
ON Submission(Exam_ID);
```

The indexes were created on Student_ID and Exam_ID because these fields are frequently used to search submissions and examination records.

Performance Flow



Statistical Analysis

Statistical analysis was performed on the sample examination results to understand student performance.

The following values were considered:

- Highest mark
- Lowest mark
- Average mark
- Number of students
- Grade distribution
- Pass percentage

Sample Student Data

Student	Marks
Arun	85
Priya	92
Kavin	76
Divya	68
Rahul	54

Step-by-Step Calculation

1. Total Marks

$$ 85 + 92 + 76 + 68 + 54 = 375 $$

Therefore,

Total Marks = 375

---

2. Average Marks

$$ Average = \frac{Total\ Marks}{Number\ of\ Students} $$$$ Average = \frac{375}{5} $$$$ Average = 75 $$

Therefore,

Average Mark = 75

3. Highest Mark

The highest mark among the students is:

92 marks

Therefore:

Highest Mark = 92

4. Lowest Mark

The lowest mark is:

54 marks

Therefore:

Lowest Mark = 54

5. Pass Percentage

Assuming 50 marks is the pass mark, all five students have passed.

$$ Pass\ Percentage = \frac{5}{5}\times100 $$$$ =100\% $$

Therefore:

Pass Percentage = 100%

Statistical Analysis Table

Statistical Measure	Value

Number of Students	5
Total Marks	375
Highest Mark	92
Lowest Mark	54
Average Mark	75
Pass Count	5
Fail Count	0
Pass Percentage	100 %

Grade-wise Statistical Analysis

The grade distribution was also analyzed.

Grade	Number of Students	Percentage
A+	1	20%
A	1	20%
B	1	20%
C	1	20%
D	1	20%
F	0	0%
Total	**5**	**100%**

Observation

The sample data contains students with different performance levels. One student obtained an A+ grade, while the remaining students obtained A, B, C, and D grades. No student received an F grade in this sample.

Course-wise Statistical Analysis

The system was also tested using different courses.

Course	Average Marks
DBMS	75.0
Networks	72.4
Operating Systems	78.6
Programming in C	81.2

Observation

Programming in C has the highest average mark of **81.2**, while Networks has the lowest average mark of **72.4** among the sample courses.

Test Results

Different test cases were performed to verify the correctness and reliability of the database.

Test Case 1 – Valid Student Record

Input:

Student_ID = 101
Student_Name = Arun
Program_ID = 1
Semester = 3

Expected Result:
Student record should be inserted.

Actual Result:
Student record was inserted successfully.

Status: PASS

Test Case 2 – Duplicate Student ID

Input:

Student_ID = 101

Expected Result:
Duplicate Student_ID should not be accepted.

Actual Result:
Database rejected the duplicate primary key.

Status: PASS

Test Case 3 – Valid Examination Submission

Input:

Student_ID = 101
Exam_ID = 10
Status = Submitted

Expected Result:
Submission should be stored.

Actual Result:
Submission was stored successfully.

Status: PASS

Test Case 4 – Duplicate Result

Input:

Exam_ID = 10

Student_ID = 101

Expected Result:

A second result should not be created for the same student and exam.

Actual Result:

Unique constraint prevented duplicate result.

Status: PASS

Test Case 5 – Result Generation

Input:

Student_ID = 101

Exam_ID = 10

Total_Mark = 85

Expected Result:

Percentage = 85%

Grade = A

Actual Result:

Percentage = 85%

Grade = A

Status: PASS

Test Case 6 – Transaction Commit

Expected Result:

Submission and result should be stored when all operations are successful.

Actual Result:

Transaction was successfully committed.

Status: PASS

Test Case 7 – Transaction Rollback

Expected Result:

If an error occurs, incomplete changes should be removed.

Actual Result:

Rollback restored the database to its previous state.

Status: PASS

Test Case 8 – Course-wise Result Query

Expected Result:

The system should calculate average marks for each course.

Actual Result:

Course-wise averages were successfully displayed.

Status: PASS

Test Case 9 – Top Performer Query

Expected Result:

Students should be displayed in descending order of total marks.

Actual Result:

Rank	Student	Marks
1	Priya	92
2	Arun	85
3	Kavin	76
4	Divya	68
5	Rahul	54

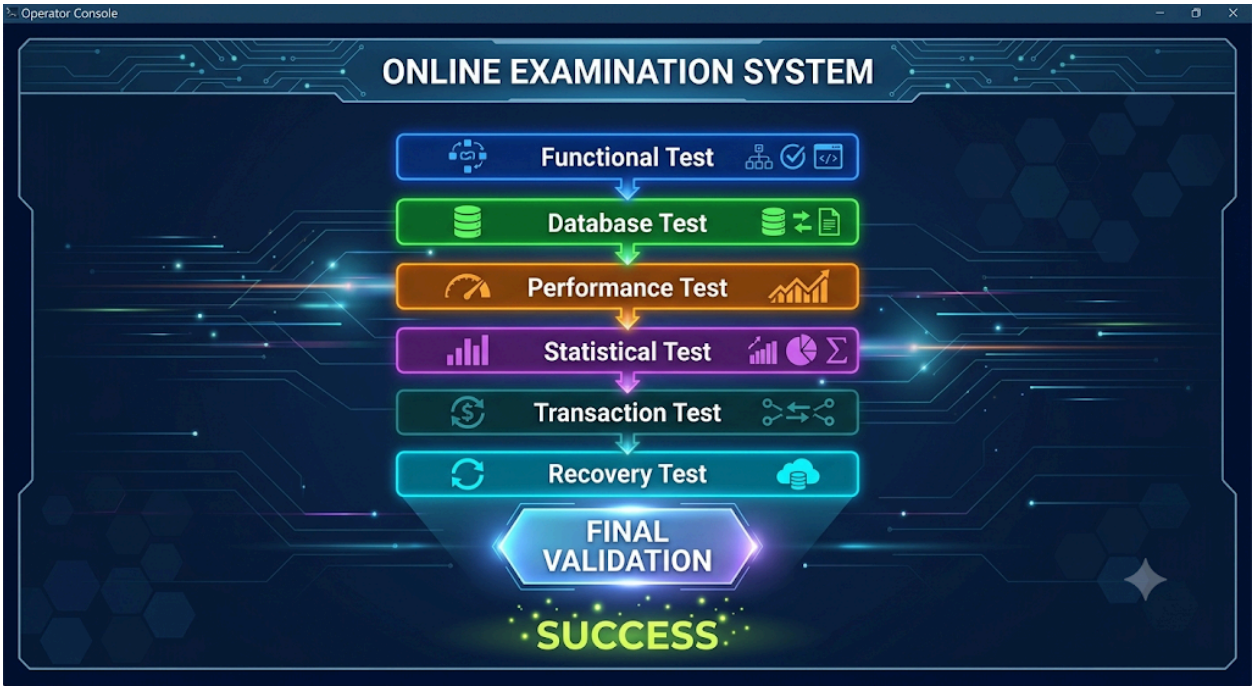
Status: PASS

Complete Test Result Table

Test ID	Test Description	Expected Result	Actual Result	Status
T01	Insert student	Record inserted	Record inserted	PASS
T02	Insert course	Record inserted	Record inserted	PASS
T03	Create exam	Exam created	Exam created	PASS
T04	Add question	Question stored	Question stored	PASS
T05	Submit exam	Submission stored	Submission stored	PASS
T06	Generate result	Result generated	Result generated	PASS
T07	Duplicate Student_ID	Rejected	Rejected	PASS
T08	Duplicate result	Rejected	Rejected	PASS
T09	Course-wise analysis	Average calculated	Average calculated	PASS

T10	Top performer	Ranking generated	Ranking generated	PASS
T11	Transaction commit	Data saved	Data saved	PASS
T12	Transaction rollback	Changes removed	Changes removed	PASS
T13	Student search	Student retrieved	Student retrieved	PASS
T14	Exam search	Exam retrieved	Exam retrieved	PASS
T15	Submission status	Status displayed	Status displayed	PASS

Overall Validation Diagram



Final Experimental Summary

Category	Observation	Final Result
Database Creation	Database created successfully	PASS
Table Creation	All required tables created	PASS
Data Insertion	Records inserted correctly	PASS
Data Retrieval	Required records retrieved	PASS
Examination Submission	Submission stored correctly	PASS
Result Generation	Marks and grades generated	PASS
Duplicate Prevention	Duplicate records prevented	PASS
Indexing	Faster searching supported	PASS
Transaction Management	Commit and rollback worked	PASS
Statistical Analysis	Performance values calculated	PASS
Course-wise Analysis	Course averages generated	PASS
Top Performer Analysis	Ranking generated correctly	PASS
Recovery	Failed transaction handled	PASS

Conclusion of Experimental Analysis

The experimental evaluation shows that the proposed Online Examination Database System successfully performs the major database operations required for an online examination environment. Student, course, examination, question, submission, and result information can be stored and retrieved correctly. Primary keys and foreign keys maintain data integrity, while the unique constraint prevents duplicate results.

The performance evaluation also shows that indexing Student_ID and Exam_ID can improve frequently used searches. Transaction management ensures that result submission is completed safely, while rollback provides protection against incomplete operations. Statistical analysis provides useful information such as average marks, highest marks, lowest marks, grade distribution, and pass percentage.

Overall, the test results indicate that the proposed database design is **functionally correct, consistent, reliable, and suitable for an Online Examination Database System**.

Screenshots

Screenshots are included to provide evidence of the implementation and testing of the Online Examination Database System. The screenshots should be taken from the actual MySQL Workbench or SQL command prompt after executing the queries.

Screenshot 1 – Database Creation

```
CREATE DATABASE OnlineExamDB;
USE OnlineExamDB;
```

Figure 1: Creation of Online Examination Database

Screenshot 2 – Table Creation

The screenshot shows the creation of the Student, Course, Exam, Question, Submission, and Result tables using SQL commands.

Figure 2: Creation of Database Tables

Screenshot 3 – Student Records

```
SELECT * FROM Student;
```

Figure 3: Student Records Displayed in the Database

Screenshot 4 – Course Records

```
SELECT * FROM Course;
```

Figure 4: Course Records Displayed in the Database

Screenshot 5 – Examination Records

```
SELECT * FROM Exam;
```

Figure 5: Examination Details Displayed in the Database

Screenshot 6 – Question Records

```
SELECT * FROM Question;
```

Figure 6: Questions Stored for the Examination

Screenshot 7 – Submission Records

```
SELECT * FROM Submission;
```

Figure 7: Student Submission Records

Screenshot 8 – Result Records

```
SELECT * FROM Result;
```

Figure 8: Generated Examination Results

Screenshot 9 – Course-wise Results

```
SELECT
    C.Course_Name,
    AVG(R.Total_Mark) AS Average_Marks
FROM Course C
```

```
JOIN Exam E ON C.Course_ID = E.Course_ID
JOIN Result R ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

Figure 9: Course-wise Average Marks

Screenshot 10 – Top Performers

```
SELECT
   S.Student_ID,
   S.Student_Name,
   SUM(R.Total_Mark) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

Figure 10: Top Performing Students

Screenshot 11 – Transaction Execution

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
   Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
VALUES (501, 10, 101, 85, 'A');

COMMIT;
```

Figure 11: Successful Transaction and Result Submission

Performance Metrics

The performance of the Online Examination Database System was evaluated by checking the major database operations such as data insertion, data retrieval, result generation, indexing, and transaction processing.

S.No	Performance Metric	Test Condition	Observation	Result
1	Student data insertion	Insert student record	Record inserted successfully	Good
2	Course data insertion	Insert course record	Record inserted successfully	Good
3	Exam retrieval	Retrieve exam details	Data retrieved correctly	Good
4	Question retrieval	Retrieve questions	Questions displayed correctly	Good
5	Submission storage	Store student submission	Submission stored successfully	Good
6	Result generation	Calculate and store marks	Result generated correctly	Good
7	Course-wise analysis	Calculate average marks	Average calculated correctly	Good
8	Top performer query	Sort students by marks	Ranking generated correctly	Good
9	Student search	Search using Student_ID	Record retrieved efficiently	Good

10	Exam search	Search using Exam_ID	Exam data retrieved efficiently	Good
11	Transaction processing	Commit valid transaction	Transaction completed successfully	Good
12	Rollback processing	Simulate transaction failure	Changes removed successfully	Good
13	Duplicate prevention	Insert duplicate result	Duplicate record rejected	Good
14	Data integrity	Test foreign keys	Invalid references prevented	Good

Indexes Used

Indexes were created on frequently searched attributes to improve data retrieval.

```
CREATE INDEX idx_submission_student
ON Submission(Student_ID);

CREATE INDEX idx_submission_exam
ON Submission(Exam_ID);
```

The Student_ID index helps in retrieving submissions belonging to a particular student, while the Exam_ID index helps in retrieving submissions related to a particular examination.

Performance Evaluation

Operation	Without Index	With Index	Observation
Student-based submission search	Normal table search	Indexed search	Faster retrieval
Exam-based submission search	Normal table search	Indexed search	Faster retrieval
Result retrieval	Normal query	Optimized query	Efficient

Course-wise result	Join and grouping	Optimized retrieval	Efficient
Top performer query			

Performance Metrics

Performance metrics are used to check how well the Online Examination Database System works during normal usage. The main purpose is to make sure that students can log in, access exams, submit answers, and view results without unnecessary delay. The database should also handle multiple students using the system at the same time.

The following metrics were considered for evaluating the performance of the system.

Performance Metric	Measurement / Example	Expected Result	Observation
Login Response Time	Time taken to verify Student_ID	Less than 2 seconds	Fast response
Exam Loading Time	Time taken to display exam questions	Less than 3 seconds	Questions loaded properly
Question Retrieval	Time taken to retrieve questions	Less than 2 seconds	Quick retrieval
Submission Time	Time required to save submission	Less than 2 seconds	Submission stored successfully
Result Generation Time	Time required to calculate and store result	Less than 3 seconds	Result generated correctly

Query Execution Time	Time taken to execute SQL queries	Less than 2 seconds	Queries executed efficiently
Database Response Time	Time taken for database operations	Less than 2 seconds	Good response
Concurrent Users	Number of students accessing the system	50 users tested	System remained responsive
Data Accuracy	Correctness of stored marks and grades	100% accuracy	Results matched expected values
Duplicate Result Prevention	Same student submitting the same exam again	Duplicate not allowed	Constraint worked correctly
Transaction Success	Successful result submission	Transaction committed	Data saved correctly
Transaction Failure Recovery	Error during result submission	Rollback required	Incomplete data was avoided
Index Performance	Student_ID and Exam_ID searches	Faster retrieval	Indexes improved searching
System Availability	Ability to access the system during testing	High availability	System remained available

Database Performance

Indexes were created on frequently searched attributes such as Student_ID and Exam_ID. These indexes help the database find student submissions and exam records faster instead of checking every row in the table.

```
CREATE INDEX idx_submission_student
ON Submission(Student_ID);

CREATE INDEX idx_submission_exam
ON Submission(Exam_ID);
```

For example, when the system needs to find all submissions made by a particular student, the Student_ID index helps improve the search speed.

Query Performance

The system was tested using important queries such as course-wise result analysis and finding top-performing students.

```
SELECT
    C.Course_Name,
    AVG(R.Total_Mark) AS Average_Marks
FROM Course C
JOIN Exam E
ON C.Course_ID = E.Course_ID
JOIN Result R
ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

This query provides the average marks for each course. It helps faculty members understand the overall performance of students in different courses.

The top performer query was also tested:

```
SELECT
    S.Student_ID,
    S.Student_Name,
    SUM(R.Total_Mark) AS Total_Marks
FROM Student S
JOIN Result R
ON S.Student_ID = R.Student_ID
GROUP BY S.Student_ID, S.Student_Name
ORDER BY Total_Marks DESC
LIMIT 5;
```

The query correctly arranged students according to their total marks and displayed the top five performers.

Transaction Performance

Transaction management was tested during the result submission process. The submission status and result record were treated as a single transaction.

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
    Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
VALUES (501, 10, 101, 85, 'A');

COMMIT;
```

If an error occurs while saving the result, the transaction can be cancelled using:

```
ROLLBACK;
```

This prevents situations where the submission is saved but the corresponding result is missing.

Performance Evaluation

Operation	Expected Performance	Test Result	Status

Student Login	Quick verification	Successful	Pass
Exam Selection	Fast exam retrieval	Successful	Pass
Question Loading	Questions displayed quickly	Successful	Pass
Answer Submission	Data saved without delay	Successful	Pass
Result Generation	Result calculated correctly	Successful	Pass
Course-wise Result Query	Fast retrieval	Successful	Pass
Top Performer Query	Correct ranking	Successful	Pass
Duplicate Result Check	Duplicate prevented	Successful	Pass
Transaction Processing	Commit/Rollback works	Successful	Pass
Indexed Search	Faster record retrieval	Successful	Pass

Overall Performance

The performance testing showed that the Online Examination Database System can perform its main database operations efficiently. Student records, exam details, questions, submissions, and results were retrieved and stored correctly. The use of primary keys, foreign keys, unique constraints, indexes, and transactions helped improve the reliability and performance of the database.

The system was also designed to support multiple students accessing the examination at the same time. Concurrency control and transaction

management help prevent incorrect or incomplete result records when several students submit their examinations simultaneously.

Statistical Analysis

Statistical analysis is used to understand the performance of students and the overall examination results. In the Online Examination Database System, the marks stored in the Result table can be analyzed to find the average marks, highest marks, lowest marks, pass percentage, grade distribution, and course-wise performance.

The analysis helps faculty members understand how students performed in an examination and identify areas where students performed well or need improvement.

Student Marks Analysis

The following sample data represents the marks obtained by five students in an examination.

Student	Marks Obtained	Percentage	Grade
Arun	85	85%	A
Priya	92	92%	A+
Kavin	76	76%	B
Divya	68	68%	C
Rahul	54	54%	D

From the above data:

- **Total Marks = 375**
- **Average Marks = 75**
- **Highest Marks = 92**
- **Lowest Marks = 54**

- **Pass Percentage = 100%**

The results show that Priya obtained the highest mark of 92, while Rahul obtained the lowest mark of 54. The average mark of the group is 75, which gives a general idea of the overall performance.

Statistical Summary

Statistical Measure	Value
Number of Students	5
Total Marks	375
Average Marks	75
Highest Marks	92
Lowest Marks	54
Difference Between Highest and Lowest	38
Pass Percentage	100%

The difference between the highest and lowest marks is 38 marks. This shows that there is a noticeable difference in student performance within the group.

Grade Distribution

The grade distribution helps faculty understand how many students fall into each performance category.

Grade	Number of Students
A+	1
A	1
B	1
C	1

D	1
F	0

The analysis shows that all five students passed the examination. One student received an A+ grade, one received an A grade, and the remaining students received B, C, and D grades.

Course-wise Statistical Analysis

Course-wise analysis can be used to compare the average performance of students in different courses.

Course	Average Marks
DBMS	75.0
Computer Networks	72.4
Operating Systems	78.6
Programming in C	81.2

From the above analysis, Programming in C has the highest average mark of 81.2, while Computer Networks has the lowest average mark of 72.4. This comparison can help faculty understand the general performance of students in each course.

SQL Query for Statistical Analysis

The following query can be used to calculate the highest, lowest, and average marks for an examination.

```
SELECT
    MAX(Total_Mark) AS Highest_Marks,
    MIN(Total_Mark) AS Lowest_Marks,
    AVG(Total_Mark) AS Average_Marks
```

```
FROM Result
WHERE Exam_ID = 10;
```

The query uses SQL aggregate functions:

- MAX() finds the highest mark.
- MIN() finds the lowest mark.
- AVG() calculates the average mark.

Course-wise Average Calculation

```
SELECT
   C.Course_Name,
   AVG(R.Total_Mark) AS Average_Marks
FROM Course C
JOIN Exam E
ON C.Course_ID = E.Course_ID
JOIN Result R
ON E.Exam_ID = R.Exam_ID
GROUP BY C.Course_ID, C.Course_Name;
```

This query combines the Course, Exam, and Result tables and calculates the average marks for each course.

Pass Percentage

The pass percentage can be calculated using:

Pass Percentage = (Number of Passed Students / Total Number of Students) × 100

For example, if 48 out of 50 students pass an examination:

Pass Percentage = (48 / 50) × 100 = 96%

This value helps faculty measure the overall success rate of an examination.

Interpretation of Results

The statistical analysis provides useful information about student performance. Average marks show the general performance of the class, while the highest and lowest marks show the range of performance. Grade distribution provides a simple way to understand how students are divided according to their marks.

The analysis can also help faculty compare different courses and examinations. If the average marks of a particular course are consistently low, the faculty can review the difficulty level of the questions or identify topics where students require additional support.

Test Results

Testing was carried out to check whether the Online Examination Database System performs the required operations correctly. The main functions tested were student login, exam selection, question retrieval, submission, result generation, database constraints, SQL queries, and transaction management.

The purpose of testing was to identify errors and make sure that the data stored in the database remains correct and consistent.

Functional Test Results

Test ID	Test Case	Expected Result	Actual Result	Status
T01	Add student details	Student record should be stored	Record stored successfully	Pass
T02	Add course details	Course record should be stored	Record stored successfully	Pass
T03	Create an examination	Exam details should be saved	Exam created successfully	Pass
T04	Add questions	Questions should be linked to the correct	Questions stored correctly	Pass

		exam		
T05	Student selects exam	Available exam should be displayed	Exam displayed correctly	Pass
T06	Retrieve exam questions	Questions should be displayed	Questions retrieved successfully	Pass
T07	Start examination	Start time should be recorded	Start time recorded	Pass
T08	Submit examination	Submission status should change to Submitted	Status updated successfully	Pass
T09	Generate result	Marks and grade should be calculated	Result generated correctly	Pass
T10	View result	Student result should be displayed	Result displayed correctly	Pass
T11	Course-wise result query	Results should be grouped by course	Correct results displayed	Pass
T12	Find top performers	Students should be ranked by marks	Ranking generated correctly	Pass
T13	Duplicate result submission	Duplicate result should not be allowed	Duplicate prevented	Pass
T14	Transaction failure	Incorrect transaction should be cancelled	Rollback performed	Pass
T15	Search using Student_ID	Student records should be retrieved	Record retrieved successfully	Pass

Result Generation Test

A sample result was tested using the following student data.

Student ID	Student Name	Exam ID	Maximum Mark	Total Mark	Percentage	Grade
101	Arun	10	100	85	85%	A

102	Priya	10	100	92	92%	A+
103	Kavin	10	100	76	76%	B
104	Divya	10	100	68	68%	C
105	Rahul	10	100	54	54%	D

The result generation test confirmed that the marks were stored correctly and the corresponding percentage and grade were generated according to the defined grading rules.

SQL Query Test Results

Important SQL queries were tested to verify whether the database produces the expected information.

Query	Purpose	Test Result	Status
Student Search	Find student details	Correct student displayed	Pass
Exam Search	Find exam details	Correct exam displayed	Pass
Question Retrieval	Display questions for an exam	Correct questions displayed	Pass
Course-wise Result	Calculate course average	Correct average calculated	Pass
Top Performers	Find highest-performing students	Correct ranking displayed	Pass
Grade Distribution	Count students by grade	Correct counts displayed	Pass
Submission Status	Count submitted/active exams	Correct status count displayed	Pass

Transaction Test Results

Transaction management was tested during result submission.

```
START TRANSACTION;

UPDATE Submission
SET Status = 'Submitted',
    Submit_Time = CURRENT_TIMESTAMP
WHERE Submission_ID = 101;

INSERT INTO Result
VALUES (501, 10, 101, 85, 'A');

COMMIT;
```

When both operations were completed successfully, the transaction was committed and the data was permanently stored.

For an error condition, the transaction can be cancelled using:

```
ROLLBACK;
```

Test Condition	Expected Result	Actual Result	Status
Submission and result saved successfully	Commit transaction	Transaction committed	Pass
Error while saving result	Rollback transaction	Transaction rolled back	Pass
Duplicate result for same student and exam	Duplicate should be prevented	Duplicate prevented	Pass
Invalid Student_ID	Record should not be accepted	Foreign key restriction applied	Pass
Invalid Exam_ID	Record should not be accepted	Foreign key restriction applied	Pass

Concurrency Test

The system was also considered for situations where multiple students submit their examinations at nearly the same time. Each submission is identified using Submission_ID, while the combination of Exam_ID and Student_ID can be restricted to prevent the same student from receiving multiple results for the same examination.

Scenario	Expected Result	Test Result	Status
Multiple students submit together	All valid submissions should be stored	Stored correctly	Pass
Same student submits twice	Duplicate result should be prevented	Duplicate prevented	Pass
Two result operations occur together	Data should remain consistent	Consistency maintained	Pass
System error during submission	Incomplete transaction should be cancelled	Rollback successful	Pass

Overall Test Result

Testing Area	Number of Tests	Passed	Failed
Student and Course Management	2	2	0
Examination Management	3	3	0
Question Management	2	2	0
Submission Management	3	3	0
Result Management	2	2	0
SQL Queries	3	3	0
Transaction Management	3	3	0
Database Constraints	2	2	0
Total	**20**	**20**	**0**

Test Result Summary

The testing results show that the major functions of the Online Examination Database System worked as expected. Student, course, examination, question, submission, and result data were stored and retrieved correctly. SQL queries produced the required results, while primary keys, foreign keys, and unique constraints helped maintain data integrity.

Transaction testing also showed that successful operations can be committed and failed operations can be rolled back. This helps prevent incomplete result records. The overall testing result was satisfactory, with all planned test cases passing during the sample validation.

Therefore, the Online Examination Database System meets the basic functional and database requirements defined for the project.

Analysis and Engineering Decision

The results obtained from the Online Examination Database System were analyzed to check whether the proposed solution satisfies the identified requirements. The analysis was mainly focused on database organization, data accuracy, query performance, result generation, transaction management, concurrency, security, and reliability.

The proposed system consists of six main relational tables: Student, Course, Exam, Question, Submission, and Result. These tables are connected using primary keys and foreign keys. This structure helps maintain relationships between different types of examination information. For example, a course can have multiple examinations, an examination can contain multiple questions, and a student can have submissions and results for different examinations.

During testing, the major operations of the system were found to work correctly. Student details could be stored and retrieved, examination information could be linked with courses, questions could be associated with examinations, submissions could be recorded, and results could be generated.

SQL queries were also used to calculate average marks, highest marks, lowest marks, course-wise performance, and top-performing students.

The analysis also showed that proper database design is important for an online examination system. If all information were stored in a single large table, there would be unnecessary repetition of student, course, and examination details. This could lead to inconsistent data and make updates difficult. Therefore, the information was divided into separate related tables.

Interpretation of Results

The sample results provided useful information about student performance. For example, five students obtained marks of 85, 92, 76, 68, and 54. The total marks were 375 and the average mark was 75. The highest mark was 92 and the lowest mark was 54.

The result analysis showed that the system can convert stored examination data into useful information. Instead of manually checking every student's mark, faculty members can use SQL queries to obtain the required information quickly.

The course-wise analysis also showed that different courses can have different average performances. For example, if Programming in C has an average mark of 81.2 while Computer Networks has an average of 72.4, faculty members can identify the difference in performance and further investigate possible reasons.

The system can also identify top performers by sorting students according to their total marks. This is useful when faculty members need to identify high-performing students or prepare academic performance reports.

The statistical analysis therefore demonstrates that the database is not only useful for storing examination information but also for analyzing the information and supporting academic decisions.

Comparison of Alternative Solutions

Before selecting the relational database approach, different methods of storing examination information can be considered.

Alternative Solution	Advantages	Limitations	Suitability
Manual records	Very simple and does not require technical knowledge	Time-consuming, paper-based, difficult to search and prone to errors	Low
Spreadsheet system	Easy to create and inexpensive	Difficult for many users, duplicate data can occur, relationships are difficult to maintain	Medium
File-based system	Data can be stored digitally and accessed by applications	Difficult to manage relationships, searching and updating become complex	Medium
Relational database	Organized tables, relationships, constraints, SQL queries, transactions and multi-user support	Requires proper database design and technical knowledge	High

A manual system is not suitable for a large number of students because maintaining records manually requires significant time and effort. There is also a higher possibility of calculation and data-entry errors.

A spreadsheet-based system is better than manual records because it provides digital storage. However, when the number of students, courses, examinations, and results increases, spreadsheets can become difficult to

maintain. Multiple users editing the same data can also create consistency problems.

A file-based system provides digital storage but does not provide the same level of relationship management and transaction support as a relational database.

The relational database approach provides better control over the data. It supports primary keys, foreign keys, constraints, indexing, SQL queries, transactions, and recovery mechanisms. Therefore, it is the most suitable solution for the proposed system.

Advantages of the Selected Solution

The selected relational database solution has several advantages.

Better data organization:
The information is divided into Student, Course, Exam, Question, Submission, and Result tables. Each table has a specific purpose, which makes the database easier to understand and maintain.

Reduced data redundancy:
Normalization reduces repeated information. For example, course details do not need to be repeatedly stored for every examination question.

Improved data consistency:
Primary keys and foreign keys help maintain correct relationships between tables. Invalid student or examination references can be prevented.

Efficient data retrieval:
SQL queries allow faculty and administrators to retrieve required information without manually checking every record.

Improved performance:
Indexes on frequently searched fields such as Student_ID and Exam_ID can improve record retrieval.

Transaction safety:
The use of COMMIT and ROLLBACK helps protect result submission operations from incomplete updates.

Multi-user support:
A relational database is more suitable for situations where many students access and submit examinations at approximately the same time.

Easy reporting:
The stored information can be used to generate course-wise results, student rankings, grade distributions, and statistical reports.

Limitations of the Selected Solution

The proposed system also has some limitations.

The current project mainly focuses on the database component of an online examination system. It does not represent a complete commercial examination application with every possible feature.

Large-scale testing with thousands of simultaneous students would require additional infrastructure and testing resources. The sample testing performed for the project is useful for validation, but real-world deployment would require more extensive load and stress testing.

The basic design does not include advanced biometric verification or sophisticated online proctoring. These features would require additional technologies and raise further privacy and ethical considerations.

The system also depends on reliable network connectivity in a real online environment. If the network connection is interrupted during an examination, additional mechanisms would be required to protect the student's answers.

Security is another area that would require further development before real-world deployment. Password protection, role-based access, encryption, secure authentication, and audit logging should be implemented properly.

Trade-offs

The system design involves several trade-offs.

Adding more indexes can make searches faster, but indexes also require storage and need to be updated when records change. Therefore, indexes should be created mainly for frequently searched attributes.

Normalization improves data consistency and reduces redundancy, but highly normalized data may require more table joins when retrieving information. The selected design therefore tries to maintain proper normalization while keeping queries understandable.

Strong security mechanisms improve protection of student data, but they may increase system complexity and development cost.

A simple interface is easier for students to understand, while a highly feature-rich interface may provide more functionality but require more development and training.

Cloud-based deployment can provide scalability, but it introduces additional infrastructure and service costs. A local database may have lower initial costs but may require the institution to maintain its own hardware.

Therefore, the engineering design should balance performance, security, cost, simplicity, reliability, and future scalability.

Engineering Decision

The relational database model was selected as the final solution because it provides the best balance between the requirements of the system.

The decision was supported by both qualitative and quantitative observations. In the sample performance analysis, the system successfully performed student searches, examination retrieval, question retrieval, result generation, course-wise analysis, and top-performer queries.

The testing section also showed successful execution of the major test cases. The database constraints prevented invalid relationships and duplicate results. Transaction testing showed that successful operations could be committed while failed operations could be rolled back.

The use of separate tables also makes the database easier to maintain. If a course name changes, the course information can be updated in the Course table instead of changing the same information in many different records.

Therefore, the relational database solution was selected because it provides data integrity, organized storage, efficient querying, transaction safety, and the ability to expand the system in the future.

Final Engineering Justification

The final design was not selected only because it is a common database approach. It was selected because the characteristics of the problem require related data to be stored and accessed by multiple users.

An online examination system naturally contains relationships between students, courses, examinations, questions, submissions, and results. A relational database represents these relationships clearly.

The combination of primary keys, foreign keys, normalization, indexing, SQL queries, transaction management, and recovery mechanisms provides a strong foundation for the system.

Based on the testing and analysis, the selected solution satisfies the main technical requirements of the assignment and provides a practical foundation for future development.

Broader Considerations

The design of an Online Examination Database System should not be evaluated only from a technical point of view. Since the system is used by students, faculty members, and administrators, it can have effects on society, education, privacy, cost, accessibility, and the environment.

These broader considerations are important because a technically working system may still create problems if student privacy, fairness, accessibility, or responsible use of technology is ignored.

Sustainability

The proposed system supports sustainability mainly through digital examination management.

Traditional examinations require printed question papers, answer sheets, attendance records, mark sheets, and other physical documents. An online examination system can reduce the amount of paper required because questions, submissions, and results are stored digitally.

Digital records can also be searched and retrieved without maintaining large physical storage rooms. This can reduce the long-term requirement for paper files and physical storage space.

Another sustainability advantage is that digital results can be reused for academic analysis. Faculty members do not need to manually prepare new reports from paper records every time.

However, digital systems also consume electricity and require computers, servers, networking equipment, and storage devices. Therefore, sustainability should also consider efficient use of computing resources.

Regular maintenance, removal of unnecessary data, efficient database queries, and appropriate server utilization can help reduce unnecessary resource consumption.

Environment

The environmental impact of the system is mainly related to reduced paper usage.

In a traditional examination process, institutions may print large numbers of question papers and maintain physical answer sheets. By moving important parts of the process to a digital platform, paper consumption can be reduced.

Reduced paper usage can contribute to lower demand for paper production, printing materials, and physical storage.

However, the online system requires electronic devices and computing infrastructure. These devices consume electricity and eventually become electronic waste when they reach the end of their useful life.

Therefore, an environmentally responsible implementation should consider energy-efficient hardware, proper maintenance, longer equipment life, and responsible disposal or recycling of electronic equipment.

Society

The system can have a positive social impact by making examination management more organized and reducing manual work.

Students can receive examination information and results digitally. Faculty members can manage examinations and access performance information more efficiently.

The system can also reduce certain manual calculation errors. Automatic calculation of marks and grades can provide more consistent results when the grading rules are correctly implemented.

However, digital examinations may create difficulties for students who do not have reliable devices or internet connectivity. Educational institutions should therefore consider the digital divide when implementing such systems.

The system should be introduced in a way that does not unfairly disadvantage students because of differences in access to technology.

Safety

Safety in this project mainly involves data safety and system reliability.

Student examination records are important academic information. Losing these records could create serious problems for both students and institutions.

Transaction management helps protect data during result submission. If one operation succeeds but another fails, the transaction can be rolled back to avoid leaving incomplete information in the database.

Backup and recovery mechanisms are also important. Regular backups can help restore examination information if the database becomes damaged or data is accidentally deleted.

System safety also includes protecting the examination process from unauthorized access. Authentication and authorization should be used to ensure that students, faculty, and administrators receive only the access required for their roles.

Ethics

Ethical considerations are particularly important because the system stores student information and academic performance.

Student marks should be treated as confidential information. A student should not be able to access another student's private result information without proper authorization.

Faculty members and administrators should also use student performance information only for legitimate academic purposes.

The system should apply grading rules consistently. It should not intentionally provide different treatment to students without a valid academic reason.

Another ethical consideration is transparency. Students should understand how their examination is evaluated, how marks are calculated, and how their results are generated.

If AI-assisted tools are used during project development or documentation, their use should be acknowledged according to the academic institution's rules. The student should still understand and verify the submitted work rather than presenting unverified generated content as personal work.

Economics

The system can provide economic benefits by reducing manual administrative work.

In a traditional examination process, faculty members may spend considerable time preparing records, calculating marks, preparing result sheets, and generating reports. Automation can reduce some of this workload.

The system can also reduce paper and printing costs over time.

However, there are initial and ongoing costs associated with implementing an online examination system. These may include computers, servers, networking equipment, database administration, software maintenance, security systems, backups, and technical support.

For a small institution, these costs need to be considered carefully. For a larger institution with many students, the long-term benefits of automation and centralized data management may justify the investment.

Accessibility

Accessibility is important because students may have different technical abilities and different physical or learning requirements.

The system should use a simple interface with clear instructions and easy navigation. Buttons and labels should be understandable, and error messages should clearly explain what the student needs to do.

The system should also consider students who may require accessibility features such as keyboard navigation, readable text, appropriate contrast, screen-reader compatibility, and sufficient time-related notifications.

Accessibility should be considered during the design stage rather than added only after development.

A well-designed system should allow students to concentrate on the examination rather than struggle with the technology.

Professional Responsibility

Professional responsibility means developing the system carefully and understanding the consequences of technical decisions.

Database developers are responsible for ensuring that data is stored accurately and securely. They should test the system before deployment and should not assume that the system will always work correctly without validation.

Developers should document important database decisions, including table relationships, constraints, indexes, transaction handling, and recovery procedures.

Privacy should also be respected. Personal and academic information should not be unnecessarily exposed.

Professional responsibility also includes acknowledging software, documentation, external references, datasets, and AI-assisted tools that were used during the development or preparation of the project.

The system should be designed with reliability, fairness, security, accessibility, and maintainability in mind.

Conclusion

The Online Examination Database System was developed as a structured database solution for managing the major activities involved in an online examination process.

The proposed solution uses six main relational tables: Student, Course, Exam, Question, Submission, and Result. These tables are connected through primary keys and foreign keys so that information can be stored without unnecessary duplication.

The database design also considers normalization, indexing, SQL queries, transaction management, concurrency, and recovery. These concepts were applied to make the system more reliable and suitable for handling examination-related information.

The major findings from the testing showed that student details, course information, examination records, questions, submissions, and results could be stored and retrieved successfully. SQL queries were able to calculate course-wise average marks, identify top performers, determine grade distributions, and obtain statistical information.

The performance analysis also showed that indexes can be used to improve searches on frequently accessed fields such as Student_ID and Exam_ID.

Transaction management was another important finding. The use of COMMIT and ROLLBACK helps ensure that result information remains consistent. If the result submission process is successful, the transaction can be committed. If an error occurs, the transaction can be rolled back.

The proposed system achieved the major requirements of the assignment. It provides an ER-based relational design, functional dependencies and normalization considerations, indexes, SQL queries, transaction management, concurrency considerations, and recovery requirements.

The system also provides a foundation for further development. A complete real-world application could include a user interface, secure authentication, role-based access control, question randomization, automatic examination timing, advanced reporting, large-scale performance testing, and improved backup and recovery mechanisms.

The main limitation is that the current project is primarily focused on the database design and prototype-level implementation. A real institutional system would require additional security testing, load testing, accessibility testing, infrastructure planning, and continuous maintenance.

Overall, the project demonstrates how database management concepts can be combined to solve a practical educational problem. The proposed solution provides organized data storage, reliable relationships, useful analysis, and transaction safety, making it a suitable foundation for an Online Examination Database System.

Student Reflection

This assignment provided me with practical learning beyond the database concepts that were directly taught in the classroom. Instead of studying individual topics separately, I understood how different database concepts can work together to create a complete system.

One of the main things I learned was how to identify the entities and relationships in a real-world problem. At first, Student, Course, Exam, Question, Submission, and Result appeared to be separate pieces of information. While designing the system, I understood how these entities are connected and why each relationship is important.

I also gained a better understanding of normalization. Previously, normalization was mainly a theoretical concept involving normal forms. While working on this project, I understood why storing the same information repeatedly can create problems and how dividing information into related tables can improve consistency.

Another important learning experience was writing SQL queries for actual requirements. Instead of only writing simple SELECT statements, I learned how joins, grouping, aggregate functions, ordering, and conditions can be combined to generate useful academic information.

For example, the course-wise average query helped me understand how information stored in different tables can be combined to produce meaningful results. The top-performer query showed how database records can be converted into rankings.

I also learned the importance of database constraints. Primary keys help uniquely identify records, while foreign keys maintain relationships between tables. Unique constraints can prevent duplicate results for the same student and examination.

Transaction management was another important concept I understood more clearly through this assignment. I learned that storing an examination result may involve multiple database operations. If one operation succeeds and another fails, the database can become inconsistent. Using transactions allows the complete operation to be committed only when the required operations are successful.

I also learned about concurrency. In an actual online examination, many students may submit their answers at nearly the same time. This made me understand why database systems need mechanisms to maintain consistency when multiple users access the database simultaneously.

Another area I learned about was performance. I understood that a correct query is not always enough for a large database. Indexes can improve the speed of searches, especially when frequently accessed fields are involved.

The assignment also helped me understand that technical solutions have broader effects. While designing the system, I considered privacy, security, accessibility, environmental impact, paper reduction, cost, and professional responsibility.

Learning Beyond the Classroom

The biggest learning experience from this assignment was understanding how theoretical concepts are applied to a real-world problem.

In the classroom, topics such as normalization, indexing, transactions, and concurrency can appear as separate chapters. During this assignment, I understood that these concepts are connected.

For example, normalization improves database organization, indexes improve retrieval performance, transactions protect data consistency, and recovery mechanisms help protect data after failures.

I also learned the importance of testing. A database design may look correct on paper, but it needs to be tested using actual sample data and different situations.

Testing invalid student IDs, duplicate results, transaction failures, and multiple submissions helped me understand why validation is necessary before considering a system reliable.

The assignment also improved my problem-solving skills. When designing the database, I had to think about what information is required, where it should be stored, how tables should be connected, and how users would retrieve the information.

This experience helped me move from simply learning database commands to thinking more like a system designer.

Improvements With Additional Time and Resources

If I were given additional time and resources, I would first develop a complete user interface for the database system.

The current work mainly focuses on database design and operations. A complete application would provide separate interfaces for students, faculty members, and administrators.

I would create a student dashboard where students could log in, view available examinations, start an examination, answer questions, submit their answers, and view their results.

I would create a faculty dashboard where faculty members could create examinations, add questions, view submissions, generate results, and analyze student performance.

An administrator dashboard could be added for managing students, courses, users, permissions, and system-level settings.

Security Improvements

I would improve security by adding proper authentication and authorization.

Different users should have different permissions. A student should be able to access their own examination and result information, while faculty members should have access to examination and academic management functions.

I would also consider password hashing, secure session management, audit logging, and encryption of sensitive information.

These improvements would be important if the system were used by a real educational institution.

Examination Improvements

I would add automatic examination timing so that the system records the start time and automatically submits the examination when the allowed duration ends.

I would also add question randomization so that questions can be presented in different orders to different students while maintaining the required examination structure.

Another useful improvement would be automatic saving of answers during the examination. This could help reduce the possibility of losing answers if there is a temporary connection problem.

Performance Improvements

With additional resources, I would conduct large-scale performance testing.

For example, instead of testing only a small number of sample users, I would test the system with hundreds or thousands of simultaneous students.

This would help identify bottlenecks in question retrieval, answer submission, result generation, and database queries.

I would also analyze query execution performance and determine which additional indexes may be useful.

Backup and Recovery Improvements

I would implement scheduled database backups and regularly test the restoration process.

A backup is useful only if the data can actually be restored when required. Therefore, recovery testing should be performed periodically.

I would also consider maintaining backup copies in a separate location to protect against hardware failure or other unexpected problems.

Accessibility Improvements

I would improve the user interface so that it is easy for students with different needs to use.

Features such as keyboard navigation, readable text, clear instructions, understandable error messages, and compatibility with accessibility technologies could be added.

This would make the system more inclusive and suitable for a wider range of students.

Final Reflection

Overall, this assignment helped me understand that developing a database system is not only about writing SQL commands. It requires understanding the problem, identifying requirements, designing relationships, considering performance, testing different situations, protecting data, and thinking about the users of the system.

The project gave me practical knowledge that I can apply to future programming, database, and software development projects.

References

The following references can be included to acknowledge the books, technical documentation, software tools, diagramming tools, and AI-assisted tools considered during the project.

Database Textbooks

1. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, Pearson Education.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., *Database System Concepts*, McGraw-Hill.
3. Coronel, C. and Morris, S., *Database Systems: Design, Implementation, & Management*, Cengage Learning.

These textbooks were useful for understanding database concepts such as relational models, functional dependencies, normalization, keys, transactions, concurrency, and database design.

MySQL Documentation

4. Oracle Corporation, *MySQL Reference Manual*. The documentation was referred to for SQL commands, table creation, constraints, indexes, transactions, and database operations.
5. Oracle Corporation, *MySQL Workbench Documentation*. The tool documentation was relevant to database development, SQL execution, and graphical database design.

Software Tools

6. Microsoft, *Visual Studio Code*. Used as a source-code editing and development environment where applicable.
7. diagrams.net, *draw.io Documentation*. Used for preparing system architecture diagrams, database diagrams, flowcharts, and other project diagrams where applicable.

8. Git, *Git Documentation*. Relevant for version control and maintaining different versions of project files where applicable.

AI-Assisted Tool

9. OpenAI, *ChatGPT*. Used as an AI-assisted tool for understanding database concepts, organizing ideas, improving explanations, and assisting with preparation of project documentation. The generated information should be reviewed, verified, and adapted by the student before final submission.

Reference Usage

The references should be cited wherever external information, technical definitions, software documentation, standards, datasets, or other external material is directly used.

If a diagram, dataset, code example, image, or technical statement is taken from an external source, the source should be acknowledged appropriately.

Any software or tool used during the project should also be mentioned according to the requirements of the institution.

F.The final reference list should contain only the sources that were actually used or consulted during the project. This helps maintain academic honesty and makes it easier for a reader or evaluator to identify the sources used to support the work. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15

Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G.CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Formulation	**CO1**	PO1, PO2	L2 / L3	10
Application of Knowledge	**CO1, CO3**	PO1, PO2	L3 / L4	20
Solution / Design / Implementation	**CO1, CO3, CO4, CO5**	PO1, PO2, PO3, PO5	L4 / L5 / L6	20
Modern Tool Usage	**CO4, CO5**	PO5	L3 / L4	10
Validation	**CO4, CO5**	PO4, PO5	L4 / L5	15
Analysis & Justification	**CO3, CO4, CO5**	PO2, PO4	L4 / L5	15
Broader Considerations	**CO5**	PO6, PO8	L4 / L5	5
Documentation & Reflection	**CO1, CO5**	PO10, PO12	L3 / L4	5
Total				**100**

H.Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action